PROJECT HANDBOOK / 2026

ADVENTURE RUNNER 3D

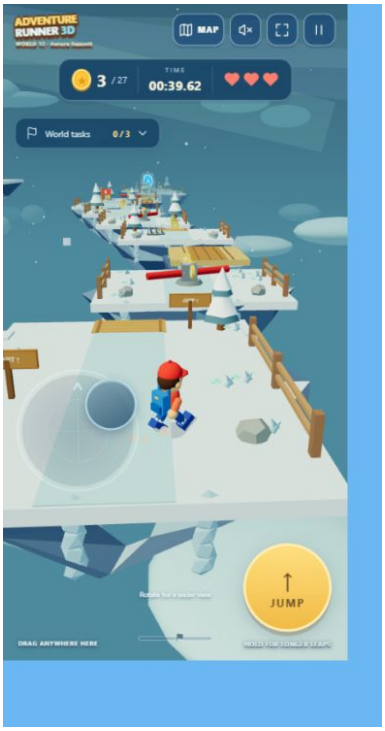
Implementation & Architecture

A complete technical recap of the twelve-world floating-island platformer: its design, code structure, gameplay, mobile controls, performance work, testing and delivery.

What is inside

Architecture diagrams and module map
Gameplay and event flows
Save-data design and world catalogue
Mobile performance decisions and limits
Run, build and hosting instructions
Complete editable-code listings + tests

Edition: source snapshot of 22 October 2026
Audience: owner, developer or technical reviewer
Companion: exact UTF-8 source ZIP embedded in this PDF



The mobile image is a development preview, not a physical-device performance measurement.

Contents

Contents 2

01 / Project scope and development recap 4

02 / Languages, tools and project files 5

03 / Runtime architecture and state ownership 7

04 / Gameplay implementation and state flow 9

05 / Events, input and user interaction 10

06 / The twelve worlds and their content 12

07 / Rendering, camera and visual feedback 14

08 / Mobile performance implementation 15

09 / Persistence, map and Continue Adventure 16

10 / Sound, haptics and browser lifecycle 17

11 / Verification, evidence and known limits 18

12 / Run, build, publish and extend 19

13 / Source snapshot and appendix guide 20

Code 01 / index.html 21

Code 02 / package.json 22

Code 03 / Play Adventure Runner.cmd 23

Code 04 / .gitignore 24

Code 05 / src/audio.js 25

Code 06 / src/batchGeometry.js 26

Code 07 / src/controls.js 27

Code 08 / src/Effects.jsx 28

Code 09 / src/game.js 29

Code 10 / src/game.test.js 33

Code 11 / src/Icon.jsx 38

Code 12 / src/main.jsx 39

Code 13 / src/MobileControls.jsx 44

Code 14 / src/preferences.js 45

Code 15 / src/Scene.jsx 46

Code 16 / src/StaticIsland.jsx 53

Code 17 / src/style.css 54

Code 18 / src/worlds.js 59

01 / Project scope and development recap

Adventure Runner 3D: The Skyward Trail is a browser-based, single-player 3D platformer built with Vite, React, React Three Fiber and Three.js. It runs its simulation and saves in the player's browser. It does not require Unity, a paid asset pack, a player account or a game server.

Visual direction and references

The project follows the supplied concept art: a red-capped explorer with a blue backpack, grassy floating islands, wooden fences and bridges, gold coins, red rotating bars, spikes, a checkpoint flag and a blue finish portal. Four reference images were visible in the conversation, although the original request mentioned five. Geometry, environment effects and synthesized sound were implemented in code; the result is an original low-poly interpretation of the references.

Development stage	Result
Core playable game	Third-person character, movement/jump, follow camera, floating-island level, health, coins, hazards, checkpoint, portal, pause, victory and local records.
World expansion	Expanded to twelve distinct routes with different environments, landmarks, moving-platform counts, hazards and coin placements.
World map and persistence	Added Map, All/Played/Completed filters, per-world tasks and best stars, next-world progression and Continue Adventure from safe ground.
Mobile and visual polish	Replaced arrows with a floating joystick; added independent jump pointers, variable jumping, responsive HUD, particles, animation, audio and haptics.
Phone performance follow-up	After feedback about mobile responsiveness, batched static scenery, removed drag-driven React updates and mobile backdrop blur, reordered simulation, and made adaptation react earlier.
Delivery	Published the game through Sites; prepared the downloadable project; explained alternative Netlify/custom-domain hosting.

Delivered scope and boundaries

Twelve worlds are available immediately. Completing one records stars and offers the next destination; it does not create an online account or unlock a multiplayer session. Progress is local to one browser origin. No server leaderboard, multiplayer, in-app purchase, native iOS/Android package or cross-device cloud save is implemented.

02 / Languages, tools and project files

Language / format	How the project uses it
JavaScript (ES modules)	Gameplay simulation, level data, controls, audio, persistence, effects and performance policies.
JSX	React components for menus/HUD and React Three Fiber components for the 3D scene. JSX is compiled by Vite; it is not a separate gameplay engine.
CSS	Responsive HUD, menus, joystick, safe-area spacing, animation and portrait/landscape rules.
HTML	The document entry point and root container.
GLSL shader strings	Animated portal spiral, waterfall surface and gradient sky inside Scene.jsx.
JSON	Package metadata, dependency lockfile, local save serialization and hosting configuration.
Windows batch	Play Adventure Runner.cmd checks Node.js, installs missing dependencies and launches the local server.

Installed dependency snapshot

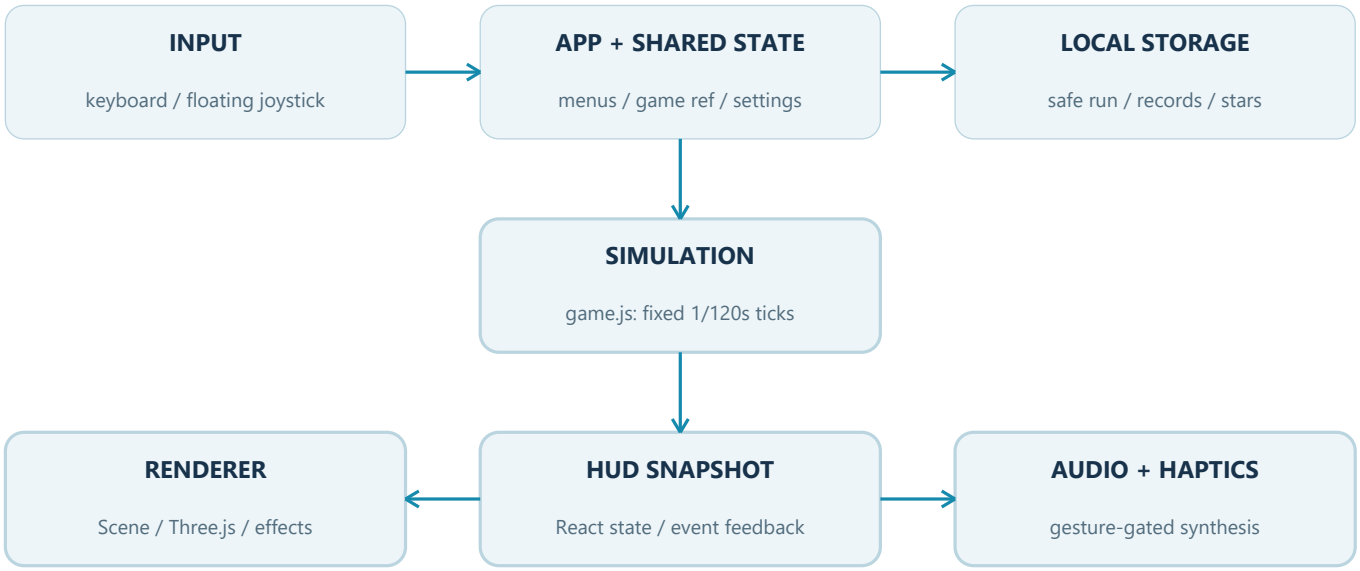
Package	Locked version	Responsibility
react	19.3.1	UI and component state
react-dom	19.3.1	Mount React in the browser
@react-three/fiber	9.8.2	React integration with Three.js and the frame loop
three	0.180.0	WebGL scene, meshes, materials, lights and cameras
vite	6.4.5	Local development and optimized production build

The project uses its own lightweight deterministic movement and collision code. A general-purpose rigid-body physics library was not added. Runtime dependencies remain React, React DOM, React Three Fiber and Three.js; the performance updates added no new runtime package.

Editable file responsibilities

File in src/	Responsibility
main.jsx	Application shell, lifecycle, keyboard input, menu/map, HUD snapshots, settings, save triggers and event dispatch.
game.js	Game model, fixed-step rules, collisions, world records, stars, safe-run serialization and restoration.
worlds.js	Ten additional deterministic routes and twelve theme palettes.
controls.js	Normalized keyboard/analog movement, joystick dead zone and separate pointer ownership.
MobileControls.jsx	Floating touch joystick, pointer capture and jump button.
Scene.jsx	Scene composition, geometry, character/camera, platform motion, portal/water shaders and frame simulation.
Effects.jsx	Pooled particles, sun glow and render-performance sampling.
preferences.js	Settings storage, device profile, vibration guards and frame-rate policy.
batchGeometry.js	Bake static local transforms and colors into one merged geometry.
StaticIsland.jsx	Create/dispose baked scenery in Smooth mode and restore originals when quality changes.
Icon.jsx	Reusable SVG HUD icons.
style.css	Responsive styling, safe areas, visual feedback and touch layouts.
audio.js	Lazy AudioContext, locally synthesized music and event sounds.
game.test.js	Node test suite, including full input-driven completions of every world.

03 / Runtime architecture and state ownership



React owns the application shell and low-frequency display state. A stable mutable game object lives in an App useRef. A shared input object carries held keys, analog axes, a held jump and a queued jump edge. The renderer and simulation read these objects directly, avoiding a React render for each physics step.

One frame, in implementation order

• Browser input handlers update the shared input object. A quick jump tap sets jumpQueued, so it survives until simulation consumes it.

• Scene useFrame runs with priority -2. While playing, it adds min(frame delta, 0.1 seconds) to an accumulator and advances game.tick in 1/120-second increments. While paused or in a menu, it clears the accumulator.

• tick updates movement, jump timing, platform carry, landing, coins, checkpoint, hazards and finish conditions. It appends event strings and bounded visual-effect records.

• The camera reads the updated position and smoothly follows it with look-ahead. Character and obstacle frame callbacks then use the latest simulation state.

• App.sync drains game.events, plays feedback, saves when necessary and updates the HUD snapshot roughly every 75 milliseconds during play, or immediately for an event.

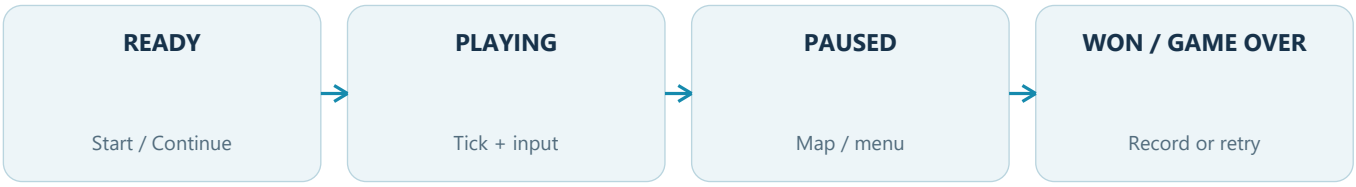
• Three.js renders the scene. FrameMonitor samples frame timing and renderer counters; a slow sample can switch to Smooth quality and a lower render density.

Model, view and world data

Object	Important fields / purpose
game	world, status, x/y/z, vx/vy/vz, facing, ground, time, health, coins Set, checkpoint, invincible, deaths, jumpBuffer, coyote, lastSafe, events and fx.
view snapshot	world, status, collected count, time, health, checkpoint, z, toast, deaths and fade. React renders this smaller projection.
world definition	platforms, coins, hazards, rotators, spawn, checkpoint, flag, portal, stages and a theme selection.
platform definition	id, x/y/z, width w, depth d and optional moving/amplitude/rate/phase. The top face is the landing surface.

Scene is memoized and receives stable callbacks. Routine HUD state changes therefore do not rebuild the entire 3D component tree. Changing world remounts Scene using its world key; the new scene uses the corresponding route and theme.

04 / Gameplay implementation and state flow



Pause resumes PLAYING. Victory and game over can restart or choose another world.

createGame creates a ready state. restart changes it to playing and queues a start event. Opening Map or pressing pause freezes simulation; resuming returns to playing. Reaching the finish sets won. Losing the final heart sets gameover. A new run resets run-specific progress, while stored best results remain.

Movement, jumping and landing

movementVector combines keyboard flags and analog axes, then normalizes diagonal input so it never exceeds full speed. Full movement speed is 6 world units/second. A jump launches with vertical speed 9.7. Gravity is 19 units/second squared. The jump system retains 0.1 seconds of coyote time and a 0.14-second input buffer. Releasing jump after the minimum 0.14-second launch window multiplies upward speed by 0.55 once, giving taps a shorter arc than a held jump.

Collision is purpose-built for the route: the player lands on rectangular platform tops while descending. platformAt computes sinusoidal horizontal motion for moving platforms. A grounded rider receives the carrier's horizontal displacement. landingPreview predicts a future landing surface and drives the green marker; it is a visual aid, not a teleport or auto-jump.

Coins, damage, checkpoint and completion

Collected coin indices live in a Set, preventing duplicate pickups and preserving them after damage. Spike hazards use distance and height checks. touchesRotator transforms the player into the current bar angle and checks along/across distances plus vertical overlap. Falling below y = -9 or hitting a hazard calls respawn. It removes one heart, increments deaths, gives two seconds of protection and returns to the start or active checkpoint with a short fade. Zero hearts ends the run.

Landing on the checkpoint island activates the checkpoint once, restores three hearts and changes the safe respawn location. Standing on the finish platform within 1.5 world units of the portal marks victory. The timer advances only during playing.

Stars and personal bests

A completed run earns one star. Collecting at least ceil(total coins x 0.6) earns a second. Finishing within the world's par time while keeping at least two hearts earns the third. Par is seven seconds per island. The end screen also displays separate coin, time and hearts medals. Previously stored highest stars never decrease. Legacy coin-only callers retain their original scoring interpretation.

The fastest completed time is stored per world. The new-personal-best message appears when the current completed time beats the previous record. Winning or game over clears the resumable-run entry, preventing an ended run from appearing as Continue.

05 / Events, input and user interaction

Simulation event	Trigger	App / feedback behavior
start	A new run begins	Startup chime; immediate save opportunity.
jump	Buffered jump launches	Jump sound and optional short vibration.
land	A descending character lands with impact	Landing sound and dust effect.
coin	A new coin is collected	Counter pop, sound, optional vibration and sparkle effect.
checkpoint	Checkpoint island is activated	Health refill, chime, vibration, effect and save.
hurt	Damage or fall calls respawn	Heart shake, damage sound, vibration and brief fade.
win	Finish portal reached	Victory sound; App records completion, best time and stars.

Game events are lightweight strings drained by App.sync. Particle records are separate objects with id, type, x/y/z and simulation time. emitEffect keeps at most 32 effect records; ParticleEffects uses a reusable instance pool rather than creating React elements for each spark.

Desktop controls

Control	Action
WASD / arrow keys	Move on the route
Space	Jump; hold for the longer arc
Escape / P	Pause or resume
M / Map button	Open the world atlas and pause an active run
R	Restart from the victory or game-over screen
Sound / fullscreen buttons	Toggle sound or request browser fullscreen

Mobile input path

A left-half movement zone captures the first joystick pointer. Its starting location becomes the movement anchor; the displayed joystick is constrained to stay visible. stickVector uses a 48-pixel radius, a radial dead zone of 0.12 and proportional speed outside that dead zone. Gentle drags walk and full drags run.

A second, independently owned pointer controls Jump. Pointer capture retains ownership when a thumb leaves a target. Releasing or cancelling one pointer does not release the other. beginJump sets both jump and jumpQueued, so a press-and-release between frames still launches. Blur, orientation changes and unmount clear touch state. Dragging paints the thumb transform directly instead of setting React state on every move.

Keyboard handlers ignore native select controls, prevent default actions for movement keys and clear held state on blur. Browser visibility changes pause an active run. Touch actions, selection and long-press menus are suppressed over the game; menus retain their own scrollable areas.

06 / The twelve worlds and their content

The first two routes are explicitly defined in game.js. worlds.js generates ten more deterministic routes from fixed lateral layouts and ferry indices. The checkpoint is always solid; a ferry index that coincides with the checkpoint is filtered out. Gaps, platform dimensions and heights are generated within the tested jump envelope.

# / world	Islands	Moving	Coins	Spikes	Bars	Par s
1 Floating Gardens	7	1	22	3	2	49
2 Ember Canyon	8	2	22	3	2	56
3 Frostfall Peaks	9	2	21	2	2	63
4 Sunstone Dunes	9	2	21	2	2	63
5 Emerald Canopy	10	2	24	2	3	70
6 Coral Coast	9	2	21	2	2	63
7 Amberwood Valley	10	2	24	2	3	70
8 Moonstone Orbit	10	3	21	2	2	70
9 Mushroom Hollow	11	2	27	3	3	77
10 Thunderhead Isles	10	2	24	2	3	70
11 Obsidian Forge	11	3	24	2	3	77
12 Aurora Summit	11	2	27	3	3	77

Environment identity

World	Visual character
Floating Gardens	Green grass, garden trees, distant castle and waterfalls.
Ember Canyon	Crystal frontier, warm rock and lava-like waterfalls.
Frostfall Peaks	Snow-covered terrain, icy cliffs and snowy trees.
Sunstone Dunes	Sandstone, cacti and pyramids.
Emerald Canopy	Jungle foliage, palms and a stepped temple.
Coral Coast	Tropical islands, turquoise water and a lighthouse.
Amberwood Valley	Autumn foliage and a turning windmill.
Moonstone Orbit	Silver terrain, stars and a ringed planet.
Mushroom Hollow	Giant mushrooms and glowing spores.
Thunderhead Isles	Storm palette, rain and electric beacons.
Obsidian Forge	Dark volcanic rock, embers and lava.
Aurora Summit	Snow, cold light and northern-light ribbons.

These are separate routes and theme variations within one platformer. Weather and landmark styling change by theme. The game does not stream a large open world or load hand-authored 3D asset packs.

07 / Rendering, camera and visual feedback

Scene composition

Canvas creates the WebGL renderer. Scene provides LowDetail and WorldTheme contexts, fog, hemisphere/directional lighting, background scenery, weather, particles, islands, coins, rotators, spikes, signs, checkpoint, portal, character, landing marker and celebration. The scene module is lazy-loaded through React Suspense; SceneBoundary presents a recovery message if the 3D scene cannot start.

Rocks are procedurally triangulated, vertex-colored island undersides. Trees, fences, flowers, signs and character parts use simple geometry. Repeated flowers/clouds and effects use instancing where appropriate. Coin rim/star details are merged into shared geometry: each coin uses one visible mesh rather than five separate detail draws.

Animation and shaders

Character limbs swing during movement, the model turns toward its facing angle, and the body squashes/stretches around jumps and landings. Coin rotation/bobbing, rotating bars and moving platforms follow simulation time. Portal GLSL draws the animated spiral; waterfall GLSL animates its surface; the sky shader blends two themed colors. Clouds drift slowly, while theme-specific point weather follows the player.

Third-person camera

The camera interpolates its position exponentially, looks toward the next platform and follows only part of vertical jump movement. Follow and Wide modes adjust distance/height. Portrait mobile uses a wider base field of view than desktop. Fast grounded movement smoothly adds a small 1.8-degree FOV kick. Camera updates run after simulation within the early frame callback.

Responsive HUD and feedback

The HUD shows coin count, timer and three hearts, with shared SVG icons for tools. Pickup count pop, heart-loss shake, checkpoint glow, a respawn fade, bounded sparkles/dust and victory stars communicate events. World Tasks is collapsible. Logo/tool sizes and layouts change by width, pointer type and landscape height; safe-area insets account for notches. Tested portrait jump targets ranged from 94 to 112 CSS pixels.

A portrait rotate hint suggests a wider view. Fullscreen, vibration and audio are guarded browser features, so unavailable APIs do not block gameplay. There is no claim that a browser fullscreen request removes all iPhone or Telegram browser chrome.

08 / Mobile performance implementation

Initial feedback found the game smooth on desktop and less responsive in an embedded mobile browser than in Safari. That observation drove a second performance pass. Desktop FPS was not treated as proof of phone performance.

Change	Implementation and reason
Static scenery batching	bakeIsland transforms each eligible static mesh into island-local coordinates, preserves vertex/material color, expands static instances and merges buffers. StaticIsland hides the originals and installs one colored mesh. Quality-change cleanup restores originals and disposes the baked resources.
Keep animated objects separate	Only selected static subtrees are batched. Character, moving hazards, portal shader and weather remain dynamic. Moving island parents still carry their baked geometry.
Reduce UI/input overhead	The joystick writes its thumb transform and shared input directly. The memoized scene stays independent of routine HUD updates. Phone backdrop blur over the moving canvas is disabled.
Update simulation first	Scene useFrame priority -2 advances physics before character/obstacle animation, avoiding the previous one-frame visual delay.
Reduce graphics cost	Smooth disables shadows and extra portal/checkpoint point lights, simplifies background detail and limits render density. Normal touch devices start at density 1.25; mobile antialias overhead is disabled.
Earlier adaptation	FrameMonitor accumulates two seconds of samples during play. Below 50 FPS, App selects Smooth and adaptive render density 1 (or .85 on a weak device). This is a one-way automatic downgrade until a manual quality change; it does not continuously raise quality.

deviceProfile considers reported device memory at or below 4 GB, hardware concurrency at or below four, and data-saving preference as low-power signals. Touch devices start in Smooth even without those signals. Missing browser hardware information is tolerated. These are heuristics, not an exact device/GPU benchmark.

Measured result and limits

Controlled render-work comparison: in the same 430 x 820 starting scene at Smooth quality, submitted drawing commands fell from 317 to 142, approximately 55% fewer. This is a reduction in graphics-driver work; it is not an iPhone FPS measurement. The temporary unbatched comparison was removed before publishing.

Physical Safari/Telegram/Android retesting remains necessary. Low Power Mode, heat, browser scheduling, embedded-browser limits and hardware differ. The implemented adaptation aims to improve responsiveness; it cannot guarantee identical graphics or a universal 60 FPS on every device.

09 / Persistence, map and Continue Adventure

localStorage key	Stored purpose
adventure-runner-worlds-v1	Per-world played/completed flags, maximum stars, maximum task coins, checkpoint task and legacy unlock marker.
adventure-runner-best-v1	Fastest completed time for World 1.
adventure-runner-best-v1-world-N	Fastest time for world index N (zero-based; worlds after the first).
adventure-runner-resume-v1	Versioned safe run: world, timer, hearts, collected coin indices, checkpoint, deaths and safe position/ground.
adventure-runner-settings-v1	Mute and haptics preferences. Quality/camera choices are session state in the current implementation.

Map opens while pausing an active run. All, Played and Completed filter the same twelve definitions. Cards show a simplified route, island/ferry counts, best stars and coin/checkpoint/portal task progress. Selecting the current paused world resumes that run; selecting a different world previews its ready state and Start New Run starts it explicitly.

Safe save and restore flow

• Autosave runs about every 1.8 seconds during play and on start/checkpoint events. Pausing, leaving or hiding the page also triggers preservation through the relevant handlers.

• lastSafe is updated only on solid, non-moving ground. An airborne or ferry save therefore restores the last safe island rather than placing the character over a gap.

• readRun validates the schema version, world, finite time, health range, unique valid coin indices, platform ID and safe bounds/height. Invalid or blocked storage returns no resumable run.

• restoreRun recreates a playing game with preserved time, coins, hearts and checkpoint; it restores solid ground and gives a brief protection period. Win/game over clear Continue; records and stars remain.

Browser storage is tied to the site origin and browser profile. Safari, Telegram and a new Netlify/custom-domain address do not automatically share these saves. The project includes no migration UI or cloud synchronization. A future domain move should warn players or implement explicit export/import first.

10 / Sound, haptics and browser lifecycle

audio.js creates AudioContext and master gain only after an interaction through unlockAudio. Sounds are oscillator/gain envelopes synthesized locally, so there are no large downloaded audio files. Event sounds cover start, jump, landing, coin, damage, checkpoint and victory. A short generated melody/bass score is scheduled while music is active, sound is enabled and the context is running.

setMusicPlaying follows the playing status. Muting or pausing clears the scheduling timer; ended oscillators/gains disconnect. App persists mute and haptics settings. Gesture restrictions, suspended contexts and missing APIs are handled defensively.

feedback checks navigator.vibrate and the user setting before invoking it. Jump, coin, hit and checkpoint have short patterns; unsupported devices simply continue without vibration. Haptics were tested with a guarded mock, not verified physically on iPhone hardware.

Failure paths and recovery

Condition	Current behavior
WebGL scene fails	SceneBoundary shows a hardware-acceleration/reload message.
Storage denied or corrupt	Reads return safe defaults; gameplay continues, but persistence can be unavailable.
Audio or vibration unavailable	Calls are guarded; game remains playable.
Fullscreen unavailable	A toast explains that play can continue in the browser.
Tab hidden / window loses focus	Input is cleared and an active run is paused/saved.
Slow render sample	Switch to Smooth/adaptive density; user can change Graphics in the menu.

11 / Verification, evidence and known limits

The latest implementation run recorded **43 passing Node tests** and a successful Vite production build. The source appendix includes the actual test suite. These checks were completed before this documentation snapshot; this PDF authoring task does not claim a new physical-device test.

Evidence category	What was verified
Simulation	Movement normalization, jumps/landing, pause, coin uniqueness, health/game over, spikes, rotating bars, platform carry and gap/jump envelope.
Complete routes	All twelve worlds completed through actual simulation inputs without teleports, damage or immunity.
Save/record behavior	Per-world fastest records, highest-star preservation, legacy records, invalid storage, safe restoration and airborne/ferry save behavior.
Touch/feel	Separate pointer owners, cancellation, analog precision, quick-tap queue, held-vs-tapped height, coyote time and buffered landing jump.
Effects/performance	Bounded effects, optional haptics, lazy audio/music lifecycle, adaptation policy, baked vertex colors and local/instance transforms.
Browser QA	Map/pause/resume, Continue after reload, graphics switching, Garden/Aurora scenes and console checks. Published version 6 loaded and retained the previous saved World 2 Continue option.

Responsive evidence by pass

The preceding polish pass recorded exact CSS sizes 375 x 812, 390 x 844, 768 x 1024 and 1920 x 1080 without document overflow. The later performance pass inspected 430 x 820 portrait and an actual 767 x 355 landscape viewport. Browser zoom changed during that pass; requested dimensions were not misreported as measured dimensions.

Known limits and technical debt

• Actual phone FPS, physical multitouch comfort, thermal behavior, vibration and embedded-browser behavior need further device testing.

• Vite emits a non-fatal large-bundle advisory for the Three.js-containing main chunk (approximately 324 KB gzip in the recorded build), plus the lazy scene chunk.

• There is no server save, account, online leaderboard, native app package, service worker/offline install flow or multiplayer implementation.

• style.css contains successive responsive overrides from development. A future cleanup should consolidate them with visual regression checks. Some older README paragraphs describe previous tuning; the latest performance section and this source-based guide supersede them.

• The API surface is modular, but some JSX/style lines are long. The appendix wraps them visually; the companion files retain exact source bytes for editing.

12 / Run, build, publish and extend

Run the copied Windows project

Open the downloaded project folder. Double-click **Play Adventure Runner.cmd**, then open the local address printed in its terminal. Keep that terminal open while playing. Node.js is required; the launcher checks for it and runs npm install only when node_modules is missing.

Command	Use
npm install	Install exact compatible dependencies for a fresh checkout; npm ci uses package-lock.json for reproducibility.
npm run dev	Run the local Vite server bound to 127.0.0.1.
npm test	Run the Node simulation/control/persistence/performance tests.
npm run build	Create the production dist folder.
npm run preview	Preview the built production output locally.

Opening index.html by double-click is not the supported source launch path. The current dev script binds to loopback, so its localhost address is not a phone-accessible LAN address. The existing hosted HTTPS link is the simplest mobile play route.

Current publication and another address

The game was published through Sites as version 6. Current recorded play link: adventure-runner-skyward-trail-kim.jollybox21.chatgpt.site

For alternative static hosting, build first and upload the **dist** folder. The user was shown Netlify Drop as a manual deployment option. A platform address can end in .netlify.app; a personally registered domain can be connected through the host. No Netlify deployment or custom-domain purchase was performed in this project. Provider features, pricing and domain availability should be checked when setting it up.

Safe extension sequence

• Add or alter a route specification in worlds.js; update WORLDS-related assumptions and add a complete input-driven route test.

• Keep checkpoint/spawn positions on solid platforms. Check gaps, jump reach, ferry timing, coins and hazard clearance together.

• Add a theme or landmark in worlds.js / Scene.jsx. Keep any animated subtree outside StaticIsland batching.

• If save format changes, version it and write migration/invalid-data tests. Preserve old best stars and make origin migration explicit.

• Re-run tests/build, inspect responsive layouts, test on real Safari/Telegram/Android hardware, then redeploy the newly built dist.

13 / Source snapshot and appendix guide

The following appendix prints every editable first-party runtime module, stylesheet, test module, HTML entry point, package configuration, launcher and ignore file. It includes the complete files, not selected snippets. Original line numbers appear on the left; a plus sign indicates a visual continuation of the same long source line. Wraps are for PDF readability only.

The companion **Adventure-Runner-Source-Companion.zip** contains byte-exact UTF-8 copies of every printed file, plus README.md, package-lock.json and SOURCE-MANIFEST.json. It is also embedded as a PDF attachment; reader support for viewing attachments varies. Generated dist bundles and third-party node_modules are excluded from printed code because they can be rebuilt or installed. The repository download includes the complete project and production build.

Git source snapshot: 0fddc412febb01c4ea610f91db5c6336b8351120

The companion ZIP contains the matching source files and checksums.

Function index: gameplay and storage

getWorld, readProgress, writeProgress, starsForRun, runMedals, recordRunProgress, completeWorld, clearRun, saveRun, readRun, restoreRun, platformAt, createGame, restart, emitEffect, respawn, touchesRotator, tick, formatTime, readBest, saveBest, landingPreview.

Source provenance and integrity

The PDF was assembled from the local project snapshot, its verification reports and the development conversation. World counts and installed dependency versions were read from code/lockfile. The SHA-256 manifest in the companion ZIP allows byte-level integrity comparison. The printed source matches the files in the companion ZIP.

Helpful external documentation

React - https://react.dev/

React Three Fiber - https://r3f.docs.pmnd.rs/

Three.js - https://threejs.org/docs/

Vite - https://vite.dev/guide/

Netlify manual deploy guide - https://docs.netlify.com/deploy/create-deploys/

External links are navigation references. The implementation statements in this report are based on the included local source and recorded checks.

Code 01 / index.html

2 original lines | 480 bytes | complete file

```
1  <!doctype html><html lang="en"><head><meta charset="UTF-8"/><meta name="viewport"
+  content="width=device-width,initial-scale=1.0,viewport-fit=cover"/><meta name="theme-color"
+  content="#132c37"/><meta name="description" content="Explore the floating gardens in Adventure Runner 3D,
+  a complete low-poly browser platformer."/><title>Adventure Runner 3D · The Skyward
+  Trail</title></head><body><div id="root"></div><script type="module"
+  src="/src/main.jsx"></script></body></html>
2
```

Code 02 / package.json

1 original lines | 366 bytes | complete file

```
1  {"name":"adventure-runner-3d","version":"1.0.0","private":true,"type":"module","scripts":{"dev":"vite
+  --host 127.0.0.1","build":"vite build","test":"node --test src/game.test.js","preview":"vite preview
+  --host 127.0.0.1"},"dependencies":{"@react-three/fiber":"^9.0.0","react":"^19.0.0","react-dom":"^19.0.0","
+  three":"^0.180.0"},"devDependencies":{"vite":"^6.1.0"}}
```

Code 03 / Play Adventure Runner.cmd

19 original lines | 422 bytes | complete file

```
@echo off
cd /d "%~dp0"
where node >nul 2>nul
if errorlevel 1 (
  echo Please install Node.js from https://nodejs.org then run this file again.
  pause
  exit /b 1
)
if not exist node_modules (
  call npm.cmd install
  if errorlevel 1 (
    pause
    exit /b 1
  )
)
echo Open the Local address below to play Adventure Runner 3D.
echo Keep this window open while playing. Press Ctrl+C to stop.
call npm.cmd run dev
pause
```

Code 04 / .gitignore

3 original lines | 38 bytes | complete file

```
1  node_modules/
2  .openai/*.local.*
3  *.log
```

Code 05 / src/audio.js

34 original lines | 2424 bytes | complete file

Lazy AudioContext, locally synthesized music and event sounds.

```
// AudioContext and the complete score are created only after a user gesture.
let context,master,timer,step=0,enabled=true,musicActive=false;
export function audioStatus(){return {created:!!context,enabled,musicActive,scheduled:!!timer};}
export function enableSound(value){enabled=value;if(master&&context)master.gain.setTargetAtTime(value?.72:0,context.currentTime,.08);scheduleMusic();}
export function unlockAudio(){
 try{
  if(!context){context=new(window.AudioContext||window.webkitAudioContext)();master=context.createGain();master.gain.value=enabled?.72:0;master.connect(context.destination);}
  if(context.state==='suspended')context.resume().then(scheduleMusic).catch(()=>{});else scheduleMusic();
 }catch{}
}
export function setMusicPlaying(value){musicActive=value;scheduleMusic();}
function note(frequency,start,duration,type='sine',volume=.065){
 if(!enabled||!context||context.state!=='running')return;
 const osc=context.createOscillator(),gain=context.createGain();osc.type=type;osc.frequency.setValueAtTime(frequency,context.currentTime+start);
 gain.gain.setValueAtTime(.0001,context.currentTime+start);gain.gain.linearRampToValueAtTime(volume,context.currentTime+start+.015);gain.gain.exponentialRampToValueAtTime(.0001,context.currentTime+start+duration);
 osc.connect(gain);gain.connect(master);osc.start(context.currentTime+start);osc.stop(context.currentTime+start+duration+.03);
 osc.onended=()=>{osc.disconnect();gain.disconnect();};
}
const melody=[523.25,659.25,783.99,659.25,587.33,698.46,880,698.46,523.25,659.25,783.99,1046.5,880,783.99,659.25,587.33];
function scheduleMusic(){
 if(timer){clearTimeout(timer);timer=null;}
 if(!musicActive||!enabled||!context||context.state!=='running')return;
 note(melody[step%melody.length],.02,.48,'sine',.018);
 if(step%4===0)note([130.81,146.83,130.81,174.61][Math.floor(step/4)%4],0,1.6,'triangle',.014);
 step++;timer=setTimeout(scheduleMusic,430);
}
export function playSound(event){
 if(event==='coin'){note(988,0,.13);note(1480,.06,.2);}
 if(event==='jump'){note(260,0,.08,'triangle',.04);note(390,.045,.09,'triangle',.035);}
 if(event==='land')note(100,0,.09,'triangle',.02);
 if(event==='hurt'){note(150,0,.18,'triangle');note(90,.08,.3,'triangle');}
 if(event==='checkpoint'||event==='start')[523,659,784].forEach((f,i)=>note(f,i*.12,.3,'triangle'));
 if(event==='win')[523,659,784,1047,784,1047].forEach((f,i)=>note(f,i*.15,.4,'triangle'));
}
```

Code 06 / src/batchGeometry.js

40 original lines | 2067 bytes | complete file

Bake static local transforms and colors into one merged geometry.

```
import * as THREE from 'three';
import { mergeGeometries } from 'three/addons/utils/BufferGeometryUtils.js';

// Bake only static island scenery. Animated obstacles and characters stay separate.
export function bakeIsland(root) {
  root.updateWorldMatrix(true, true);
  const inverse = root.matrixWorld.clone().invert();
  const parts = [], originals = [], instance = new THREE.Matrix4();
  root.traverse(object => {
    const material = object.material;
    if (!object.isMesh || !object.visible || !material?.isMeshStandardMaterial || material.transparent ||
Array.isArray(material)) return;
    const add = matrix => {
      const geometry = object.geometry.index ? object.geometry.toNonIndexed() : object.geometry.clone();
      geometry.applyMatrix4(matrix);
      const position = geometry.getAttribute('position'), colors = geometry.getAttribute('color');
      const values = new Float32Array(position.count * 3);
      for (let i = 0; i < position.count; i++) {
        values[i * 3] = material.color.r * (material.vertexColors && colors ? colors.getX(i) : 1);
        values[i * 3 + 1] = material.color.g * (material.vertexColors && colors ? colors.getY(i) : 1);
        values[i * 3 + 2] = material.color.b * (material.vertexColors && colors ? colors.getZ(i) : 1);
      }
      for (const name of Object.keys(geometry.attributes)) if (!['position', 'normal'].includes(name))
geometry.deleteAttribute(name);
      geometry.setAttribute('color', new THREE.BufferAttribute(values, 3));
      parts.push(geometry);
    };
    const local = new THREE.Matrix4().multiplyMatrices(inverse, object.matrixWorld);
    if (object.isInstancedMesh) {
      for (let i = 0; i < object.count; i++) {
        object.getMatrixAt(i, instance);
        add(new THREE.Matrix4().multiplyMatrices(local, instance));
      }
    } else add(local);
    originals.push(object);
  });
  if (!parts.length) return null;
  const geometry = mergeGeometries(parts, false);
  parts.forEach(part => part.dispose());
  geometry.computeBoundingSphere();
  return { geometry, originals };
}
```

Code 07 / src/controls.js

27 original lines | 1688 bytes | complete file

Normalized keyboard/analog movement, joystick dead zone and separate pointer ownership.

```
// Radial dead zone prevents thumb drift, while proportional speed allows careful positioning.
export function stickVector(dx,dy,radius=48){
 const distance=Math.hypot(dx,dy);
 if(!Number.isFinite(distance)||radius<=0)return {x:0,z:0,thumbX:0,thumbY:0};
 const fraction=Math.min(1,distance/radius),visual=distance?Math.min(distance,radius)/distance:0;
 const strength=fraction<=.12?0:(fraction-.12)/.88;
 return
{x:distance?dx/distance*strength:0,z:distance?dy/distance*strength:0,thumbX:dx*visual,thumbY:dy*visual};
}
export function movementVector(input){
 const axis=value=>Number.isFinite(value)?Math.max(-1,Math.min(1,value)):0;
 let x=Number(!!input.right)-Number(!!input.left)+axis(input.axisX);
 let z=Number(!!input.back)-Number(!!input.forward)+axis(input.axisZ);
 const length=Math.hypot(x,z);if(length>1){x/=length;z/=length;}
 return {x,z,strength:Math.min(1,length)};
}
// A separate pointer owns each control so lifting the jump thumb never stops steering.
export function createTouchController(input){
 let stickPointer=null,jumpPointer=null;
 return {
  beginStick(id){if(stickPointer!==null)return false;stickPointer=id;return true;},
  moveStick(id,x,z){if(id!==stickPointer)return false;input.axisX=x;input.axisZ=z;return true;},
  endStick(id){if(id!==stickPointer)return;stickPointer=null;input.axisX=0;input.axisZ=0;},
  beginJump(id){if(jumpPointer!==null)return
false;jumpPointer=id;input.jump=true;input.jumpQueued=true;return true;},
  endJump(id){if(id!==jumpPointer)return;jumpPointer=null;input.jump=false;},
  reset(){stickPointer=null;jumpPointer=null;input.axisX=0;input.axisZ=0;input.jump=false;input.jumpQueued
=false;},
 };
}
```

Code 08 / src/Effects.jsx

43 original lines | 3216 bytes | complete file

Pooled particles, sun glow and render-performance sampling.

```
import React,{useMemo,useRef} from 'react';
import {useFrame} from '@react-three/fiber';
import * as THREE from 'three';
import {performanceStep} from './preferences';
export function ParticleEffects({game,low=false}){
 const mesh=useRef(),cursor=useRef(0),serial=useRef(0),lastTime=useRef(0);
 const count=low?100:200;
 const pool=useMemo(()=>Array.from({length:count},()=>({life:0,born:-100,x:0,y:0,z:0,vx:0,vy:0,vz:0,size:0
,color:new THREE.Color()})),[count]);
 const dummy=useMemo(()=>new THREE.Object3D(),[]);
 useFrame(()=>{

if(game.time<lastTime.current){serial.current=0;pool.forEach(p=>{p.life=0;});}lastTime.current=game.time;
  for(const effect of game.fx){
   if(effect.id<=serial.current)continue;
   serial.current=effect.id;
   const sparkle=effect.type==='coin'||effect.type==='checkpoint';
   const n=effect.type==='dust'?2:low?8:14;
   for(let i=0;i<n;i++){
    const p=pool[cursor.current++%count],a=i/n*Math.PI*2+effect.id*.7;
    Object.assign(p,{born:effect.time,life:sparkle?.65:.45,x:effect.x,y:effect.y+(sparkle?.2:.06),z:effect
.z,vx:Math.cos(a)*(sparkle?2:.9),vy:sparkle?1.6+(i%3)*.6:.35+(i%3)*.18,vz:Math.sin(a)*(sparkle?2:.9),size:
sparkle?.09:.15});
    p.color.set(sparkle?(i%2?'#fff0a5':'#ffca42'):effect.type==='hit'?'#f78b73':'#dfd3b3');
   }
  }
  pool.forEach((p,i)=>{const age=Math.max(0,game.time-p.born),live=age<p.life;
   dummy.position.set(p.x+p.vx*age,p.y+p.vy*age-(p.life>.5?2:0)*age*age,p.z+p.vz*age);
   const scale=live?p.size*(1-age/p.life):0;dummy.scale.setScalar(scale);dummy.rotation.set(age*4+i,age*3,
0);dummy.updateMatrix();mesh.current.setMatrixAt(i,dummy.matrix);mesh.current.setColorAt(i,p.color);
  });
  mesh.current.instanceMatrix.needsUpdate=true;if(mesh.current.instanceColor)mesh.current.instanceColor.ne
edsUpdate=true;
 });
 return <instancedMesh ref={mesh} args={[undefined,undefined,count]}
frustumCulled={false}><icosahedronGeometry args={[1,0]}/><meshBasicMaterial transparent opacity={.8}
depthWrite={false}/></instancedMesh>;
}
export function SunGlow({night=false}){
 const texture=useMemo(()=>{const
canvas=document.createElement('canvas');canvas.width=128;canvas.height=128;const ctx=canvas.getContext('2d
'),gradient=ctx.createRadialGradient(64,64,0,64,64,64);gradient.addColorStop(0,night?'#d6edff':'#fff4c9');
gradient.addColorStop(.12,night?'#bfdcff':'#ffefac');gradient.addColorStop(.3,'#ffffff50');gradient.addCol
orStop(1,'#ffffff00');ctx.fillStyle=gradient;ctx.fillRect(0,0,128,128);return new
THREE.CanvasTexture(canvas);},[night]);
 return <sprite position={[-45,45,-105]} scale={[42,42,1]}><spriteMaterial map={texture} transparent
opacity={night?.35:.7} depthWrite={false} blending={THREE.AdditiveBlending}/></sprite>;
}
export function FrameMonitor({game,onSlow,onStats}){
 const sampling=useRef({samples:0,elapsed:0,fps:null,downgrade:false});
 useFrame(({gl},dt)=>{
  if(document.hidden||game.status!=='playing')return;
  const next=performanceStep(sampling.current,dt,game.status==='playing');sampling.current=next;
  if(next.samples===0&&next.fps){onStats?.({fps:next.fps,calls:gl.info.render.calls,triangles:gl.info.rend
er.triangles});if(next.downgrade)onSlow?.();}
 });return null;
}

```

Code 09 / src/game.js

257 original lines | 14417 bytes | complete file

Game model, fixed-step rules, collisions, world records, stars, safe-run serialization and restoration.

```
import { movementVector } from './controls.js';
import { EXTRA_WORLDS } from './worlds.js';
// World-space units are metres. Player y is the sole of the character's feet.
export const PLATFORMS = [
  { id: 'start', x: 0, z: 2, w: 11, d: 12, y: 0 },
  { id: 'step', x: 0, z: -10, w: 7, d: 6, y: .4 },
  { id: 'grove', x: 4, z: -19, w: 9, d: 8, y: .7 },
  { id: 'checkpoint', x: 0, z: -30, w: 10, d: 9, y: 1 },
  { id: 'ferry', x: 0, z: -40, w: 4.5, d: 5, y: 1, moving: true },
  { id: 'ruins', x: -3, z: -49, w: 9, d: 8, y: 1 },
  { id: 'finish', x: 0, z: -61, w: 12, d: 12, y: 1.2 },
];
export const COINS = [
  [0, 1.1, 1], [0, 1.1, -1], [0, 1.1, -3],
  [0, 1.5, -8], [1.8, 1.5, -10], [0, 1.5, -12],
  [3, 1.8, -16.5], [5.8, 1.8, -18], [5.8, 1.8, -21], [2, 1.8, -22],
  [0, 2.1, -27], [-2.4, 2.1, -29], [0, 2.1, -31.5], [1.4, 2.1, -33],
  [-3, 2.1, -46.5], [-5.5, 2.1, -48], [-5.5, 2.1, -51], [-2, 2.1, -52],
  [-1, 2.3, -56.5], [0, 2.3, -58.5], [0, 2.3, -60.5], [3.7, 2.3, -62],
];
export const HAZARDS = [
  { x: 4, z: -20, y: .7, r: .9 },
  { x: -1.3, z: -49, y: 1, r: .8 },
  { x: 2.7, z: -59, y: 1.2, r: .8 },
];
export const ROTATORS = [
  { x: 0, z: -10, y: .4, length: 5.8, speed: .85, phase: 0 },
  { x: -3, z: -49.5, y: 1, length: 6.5, speed: -1, phase: .7 },
];
export const WORLDS = [
  {
    id: 0, name: 'Floating Gardens', subtitle: 'A path above the clouds',
    platforms: PLATFORMS, coins: COINS, hazards: HAZARDS, rotators: ROTATORS,
    spawn: [0, 0, 4], checkpoint: [0, 1, -29], flag: [-2.7, 1, -29], portal: [0, 1.2, -63.5],
    stages: [[-6, 'The first leap'], [-24, 'Spinning gardens'], [-35, 'The checkpoint'], [-44, 'The
drifting stone'], [-54, 'The final trial'], [-Infinity, 'The sky gate']],
  },
  {
    id: 1, name: 'Ember Canyon', subtitle: 'Across the crystal frontier',
    platforms: [
      { id: 'start', x: 0, z: 2, w: 10, d: 12, y: 0 },
      { id: 'step', x: -3, z: -10, w: 7, d: 6, y: .5 },
      { id: 'grove', x: -6, z: -20, w: 8, d: 9, y: 1 },
      { id: 'checkpoint', x: -1, z: -31, w: 10, d: 9, y: 1.2 },
      { id: 'ferry', x: 2, z: -41, w: 4.5, d: 5, y: 1.2, moving: true, amplitude: 2.5, rate: .7 },
      { id: 'ruins', x: 5, z: -50, w: 9, d: 8, y: 1.4 },
      { id: 'ferry2', x: 2, z: -60, w: 4.5, d: 5, y: 1.4, moving: true, amplitude: 1.7, rate: .9, phase: 1
 },
      { id: 'finish', x: -1, z: -70, w: 12, d: 12, y: 1.5 },
    ],
    coins: [[0,1.1,1],[0,1.1,-1],[0,1.1,-3],[-3,1.6,-8],[-5,1.6,-10],[-3,1.6,-12],[-6,2.1,-17],[-8,2.1,-20
],[-8,2.1,-23],[-4.5,2.1,-23],[-1,2.3,-28],[-3.4,2.3,-30],[-1,2.3,-33],[1,2.3,-34],[5,2.5,-47.5],[7.4,2.5,
-49],[7.4,2.5,-52],[3.8,2.5,-52],[-1,2.6,-66],[-1,2.6,-68],[-1,2.6,-70],[2,2.6,-71]],
    hazards: [{x:-5.4,z:-20,y:1,r:.9},{x:3.5,z:-50,y:1.4,r:.8},{x:1.8,z:-68,y:1.5,r:.8}],
    rotators:
[{x:-3,z:-10,y:.5,length:5.8,speed:-1.05,phase:.5},{x:5,z:-50,y:1.4,length:6.4,speed:1.15,phase:.2}],
    spawn: [0,0,4], checkpoint: [-1,1.2,-30], flag: [-3.7,1.2,-30], portal: [-1,1.5,-72.5],
    stages: [[-6,'Into the canyon'],[-25,'The crystal turn'],[-36,'Ember checkpoint'],[-45,'The first
crossing'],[-55,'The turning forge'],[-64,'The last crossing'],[-Infinity,'The ember gate']],
  },
];
WORLDS.push(...EXTRA_WORLDS);
export function getWorld(id = 0) { return WORLDS[id] || WORLDS[0]; }
export const progressKey = 'adventure-runner-worlds-v1';
export function readProgress(storage) {
 const empty=()=>({unlocked:0,completed:WORLDS.map(()=>false),played:WORLDS.map(()=>false),stars:WORLDS.ma
p(()=>0),tasks:WORLDS.map(()=>({coins:0,checkpoint:false}))});
 try {
  const v=JSON.parse(storage.getItem(progressKey));const result=empty();
  result.unlocked=Math.min(WORLDS.length-1,Math.max(0,Number.isInteger(v?.unlocked)?v.unlocked:0));
  for(const w of WORLDS){const i=w.id;result.completed[i]=v?.completed?.[i]===true;result.played[i]=v?.pla
```

```
yed?.[i]===true||result.completed[i];result.stars[i]=Math.max(result.completed[i]?1:0,Math.min(3,Number.is
Integer(v?.stars?.[i])?v.stars[i]:0));result.tasks[i]={coins:Math.min(w.coins.length,Math.max(0,Number.isI
nteger(v?.tasks?.[i]?.coins)?v.tasks[i].coins:0)),checkpoint:v?.tasks?.[i]?.checkpoint===true};}
  return result;
 }catch{return empty();}
}
function writeProgress(storage,p){try{storage.setItem(progressKey,JSON.stringify(p));}catch{}return p;}
export function starsForRun(coins,total,time=Infinity,hearts=0,par=60){
 const coinMedal=coins>=total?3:coins>=Math.ceil(total*.6)?2:1;
 if(!Number.isFinite(time))return coinMedal; // Legacy callers/saves preserve their original coin ratings.
 return 1+Number(coins>=Math.ceil(total*.6))+Number(time<=par&&hearts>=2);
}
export function runMedals(g){const world=getWorld(g.world),par=world.platforms.length*7;return
{time:g.time<=par,coins:g.coins.size>=Math.ceil(world.coins.length*.6),hearts:g.health>=2,par};}
export function recordRunProgress(storage,g){
 const p=readProgress(storage),i=g.world;if(!WORLDS[i]||g.status==='ready')return p;
 p.played[i]=true;p.tasks[i].coins=Math.max(p.tasks[i].coins,g.coins.size);p.tasks[i].checkpoint ||=
g.checkpoint;
 return writeProgress(storage,p);
}
export function completeWorld(storage,id,coins=0,time=Infinity,hearts=0){
 const p=readProgress(storage);if(!Number.isInteger(id)||!WORLDS[id])return p;
 p.unlocked=Math.max(p.unlocked,Math.min(WORLDS.length-1,id+1));p.completed[id]=true;p.played[id]=true;
 p.stars[id]=Math.max(p.stars[id],starsForRun(coins,WORLDS[id].coins.length,time,hearts,WORLDS[id].platfor
ms.length*7));
 p.tasks[id].coins=Math.max(p.tasks[id].coins,Math.min(WORLDS[id].coins.length,coins));
 return writeProgress(storage,p);
}
export const resumeKey='adventure-runner-resume-v1';
export function clearRun(storage){try{storage.setItem(resumeKey,'null');}catch{}}
export function saveRun(storage,g){
 if(!['playing','paused'].includes(g.status)||g.health<1)return null;
 const w=getWorld(g.world),safe=g.lastSafe||{x:w.spawn[0],y:w.spawn[1],z:w.spawn[2],ground:'start'};
 const data={version:1,world:g.world,time:g.time,health:g.health,coins:[...g.coins],checkpoint:g.checkpoin
t,deaths:g.deaths,safe:{...safe}};
 try{storage.setItem(resumeKey,JSON.stringify(data));return data;}catch{return null;}
}
export function readRun(storage){
 try{
  const v=JSON.parse(storage.getItem(resumeKey));if(v?.version!==1||!Number.isInteger(v.world)||!WORLDS[v.
world]||!Number.isFinite(v.time)||v.time<0||!Number.isInteger(v.health)||v.health<1||v.health>3||!Array.is
Array(v.coins)||!v.safe)return null;
  const w=getWorld(v.world),p=w.platforms.find(p=>p.id===v.safe.ground&&!p.moving);
  if(!p||![v.safe.x,v.safe.y,v.safe.z].every(Number.isFinite)||Math.abs(v.safe.x-p.x)>p.w/2+.18||Math.abs(
v.safe.z-p.z)>p.d/2+.18||Math.abs(v.safe.y-p.y)>.01)return null;
  return {...v,checkpoint:v.checkpoint===true,deaths:Number.isInteger(v.deaths)&&v.deaths>=0?v.deaths:0,co
ins:[...new Set(v.coins.filter(i=>Number.isInteger(i)&&i>=0&&i<w.coins.length))]};
 }catch{return null;}
}
export function restoreRun(storage){
 const v=readRun(storage);if(!v)return null;
 const g=createGame(v.world);Object.assign(g,{time:v.time,health:v.health,checkpoint:v.checkpoint,deaths:v
.deaths,coins:new Set(v.coins),x:v.safe.x,y:v.safe.y,z:v.safe.z,ground:v.safe.ground,lastSafe:{...v.safe},
invincible:2,status:'playing',toast:'Adventure restored · back on the trail',toastTime:3});return g;
}
export const bestKey = 'adventure-runner-best-v1';
export function platformAt(p, time) {
  return { ...p, x: p.x + (p.moving ? Math.sin(time * (p.rate ?? .8) + (p.phase ?? 0)) * (p.amplitude ??
3) : 0) };
}
export function createGame(world = 0) {
  return {
    world, x: 0, y: 0, z: 4, vy: 0, vx:0,vz:0, facing: Math.PI, ground: 'start',
    time: 0, health: 3, coins: new Set(), checkpoint: false, status: 'ready',
    invincible: 0, toast: '', toastTime: 0, jumpHeld: false, jumpBuffer: 0,
    coyote: .1, steps: 0, moving: false, deaths: 0, events: [],
jumpAge:0,jumpCut:false,landTime:-100,fade:0,fx:[],fxSerial:0,dustClock:0,
lastSafe:{x:0,y:0,z:4,ground:'start'},
  };
}
export function restart(g, world = g.world) {
  Object.assign(g, createGame(world));
  g.status = 'playing';
  g.events.push('start');
}
export function emitEffect(g,type,x=g.x,y=g.y,z=g.z){g.fx.push({id:++g.fxSerial,type,x,y,z,time:g.time});i
f(g.fx.length>32)g.fx.shift();}
export function respawn(g) {
  emitEffect(g,'hit');
```

```
  g.fade=.22;
  g.health--;
  g.deaths++;
  g.invincible = 2;
  const level = getWorld(g.world);
  [g.x, g.y, g.z] = g.checkpoint ? level.checkpoint : level.spawn;
  g.vy = 0;
  g.ground = g.checkpoint ? 'checkpoint' : 'start';
  g.lastSafe={x:g.x,y:g.y,z:g.z,ground:g.ground};
  g.coyote = .1;
  g.jumpBuffer = 0;
  g.facing = Math.PI;
  g.toast = g.health > 0 ? 'Safe landing · try that jump again' : 'The trail is calling. Try again.';
  g.toastTime = 2.5;
  g.events.push('hurt');
  if (g.health <= 0) g.status = 'gameover';
}
export function touchesRotator(g, r) {
  if (g.y > r.y + 1.05 || g.y + 1.5 < r.y + .35) return false;
  const angle = g.time * r.speed + r.phase;
  const dx = g.x - r.x, dz = g.z - r.z;
  const along = dx * Math.cos(angle) - dz * Math.sin(angle);
  const across = dx * Math.sin(angle) + dz * Math.cos(angle);
  return Math.abs(along) < r.length / 2 + .3 && Math.abs(across) < .48;
}
export function tick(g, input, dt) {
  if (g.status !== 'playing') return;
  const level = getWorld(g.world);
  dt = Math.max(0, Math.min(dt, .05));
  g.time += dt;
  g.invincible = Math.max(0, g.invincible - dt);
  g.toastTime = Math.max(0, g.toastTime - dt);
  g.fade=Math.max(0,g.fade-dt);
  g.jumpAge+=dt;
  const previousGround=g.ground;
  const carrier = level.platforms.find(p => p.id === g.ground && p.moving);
  if (carrier) g.x += platformAt(carrier, g.time).x - platformAt(carrier, g.time - dt).x;
  const movement=movementVector(input),dx=movement.x,dz=movement.z;
  g.moving=movement.strength>0;
  if(g.moving){g.steps+=dt*movement.strength;g.facing=Math.atan2(dx,dz);}
  g.vx=dx*6;g.vz=dz*6;
  g.coyote = g.ground ? .1 : Math.max(0, g.coyote - dt);
  g.jumpBuffer = input.jumpQueued || (input.jump && !g.jumpHeld) ? .14 : Math.max(0, g.jumpBuffer - dt);
  input.jumpQueued=false;
  g.jumpHeld = !!input.jump;
  if (g.jumpBuffer > 0 && g.coyote > 0) {
    g.vy = 9.7; g.ground = null; g.coyote = 0; g.jumpBuffer = 0;g.jumpAge=0;g.jumpCut=false;
    g.events.push('jump');
  }
  // Minimum launch time makes quick taps useful; holding retains the full jump envelope.
  if(!input.jump&&!g.jumpCut&&g.jumpAge>.14&&g.vy>0&&!g.ground){g.vy*=.55;g.jumpCut=true;}
  const impactVelocity=g.vy;
  const previousY = g.y, previousX = g.x;
  g.x += dx * 6 * dt; g.z += dz * 6 * dt;
  g.vy -= 19 * dt; g.y += g.vy * dt;
  // Railings block lateral movement at their visible height; jumping clears them.
  for (const p of level.platforms) {
    if (p.moving || Math.abs(g.z - p.z - p.d * .07) > p.d * .3 + .12
        || g.y > p.y + .95 || g.y + 1.8 < p.y) continue;
    for (const side of [-1, 1]) {
      const wall = p.x + side * (p.w / 2 - .35);
      const before = previousX - wall, after = g.x - wall;
      if (Math.abs(after) < .3 && Math.abs(before) >= Math.abs(after)) {
        g.x = wall + (before < 0 ? -.3 : .3);
      } else if (before * after < 0) g.x = wall + (before < 0 ? -.3 : .3);
    }
  }
  g.ground = null;
  if (g.vy <= 0) {
    for (const base of level.platforms) {
      const p = platformAt(base, g.time);
      if (Math.abs(g.x - p.x) < p.w / 2 + .18 && Math.abs(g.z - p.z) < p.d / 2 + .18
          && previousY >= p.y - .04 && g.y <= p.y) {
        g.y = p.y; g.vy = 0; g.ground = p.id;

+ if(!previousGround&&impactVelocity<-2){g.landTime=g.time;g.events.push('land');emitEffect(g,'land');}
        break;
      }
    }
```

```
  }
  for (let i = 0; i < level.coins.length; i++) {
    const c = level.coins[i];
    if (!g.coins.has(i) && Math.hypot(g.x - c[0], g.y + .8 - c[1], g.z - c[2]) < 1) {
      g.coins.add(i); g.events.push('coin');emitEffect(g,'coin',c[0],c[1],c[2]);
    }
  }
  if (!g.checkpoint && g.ground === 'checkpoint') {
    g.checkpoint = true; g.health = 3;
    g.toast = 'Checkpoint activated · all hearts restored'; g.toastTime = 3;
    g.events.push('checkpoint');g.fade=.12;emitEffect(g,'checkpoint');
  }
  const hitSpikes = level.hazards.some(h => Math.hypot(g.x - h.x, g.z - h.z) < h.r + .28
    && g.y < h.y + .65 && g.y + 1.5 > h.y);
  if (g.y < -9 || (g.invincible <= 0 && (hitSpikes || level.rotators.some(r => touchesRotator(g, r))))) {
    respawn(g); return;
  }
  if(g.ground&&g.moving){g.dustClock+=dt;if(g.dustClock>.12){g.dustClock=0;emitEffect(g,'dust');}}
  if(g.ground &&
!level.platforms.find(p=>p.id===g.ground)?.moving)g.lastSafe={x:g.x,y:g.y,z:g.z,ground:g.ground};
  if (g.ground === 'finish' && Math.hypot(g.x - level.portal[0], g.z - level.portal[2]) < 1.5) {
    g.status = 'won'; g.events.push('win');
  }
}
export function formatTime(seconds) {
  const cs = Math.floor(Math.max(0, Number.isFinite(seconds) ? seconds : 0) * 100);
  return `${Math.floor(cs / 6000).toString().padStart(2, '0')}:${(Math.floor(cs / 100) %
60).toString().padStart(2, '0')}.${(cs % 100).toString().padStart(2, '0')}`;
}
export function readBest(storage, world = 0) {
  try {
    const value = Number(storage.getItem(world === 0 ? bestKey : `${bestKey}-world-${world}`));
    return Number.isFinite(value) && value > 0 ? value : null;
  } catch { return null; }
}
export function saveBest(storage, time, world = 0) {
  const previous = readBest(storage, world);
  const best = previous ? Math.min(previous, time) : time;
  try { storage.setItem(world === 0 ? bestKey : `${bestKey}-world-${world}`, String(best)); } catch { /*
Private mode can deny storage. */ }
  return best;
}



export function landingPreview(g){
 if(g.ground)return null;
 const world=getWorld(g.world);let result=null;
 for(const base of world.platforms){
  const discriminant=g.vy*g.vy+38*(g.y-base.y);if(discriminant<0)continue;
  const t=(g.vy+Math.sqrt(discriminant))/19;if(t<0||t>1.6)continue;
  const p=platformAt(base,g.time+t),x=g.x+(g.vx||0)*t,z=g.z+(g.vz||0)*t;
  if(Math.abs(x-p.x)<p.w/2&&Math.abs(z-p.z)<p.d/2&&(!result||t<result.t))result={x,y:p.y+.035,z,t};
 }
 return result;
}
```

Code 10 / src/game.test.js

295 original lines | 23382 bytes | complete file

Node test suite, including full input-driven completions of every world.

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { createGame, restart, tick, PLATFORMS, ROTATORS, COINS, platformAt, touchesRotator, formatTime,
readBest, saveBest } from './game.js';
const frames = (g, input, n) => { for (let i = 0; i < n; i++) tick(g, input, 1 / 120); };
const started = () => { const g = createGame(); restart(g); return g; };
test('ready/paused states freeze physics, diagonal speed matches straight movement', () => {
  const g = createGame(); tick(g, { forward: true }, .05); assert.equal(g.z, 4);
  restart(g); frames(g, { forward: true }, 60); assert.ok(Math.abs(g.z - 1) < .01);
  g.status = 'paused'; const t = g.time; frames(g, { forward: true }, 120); assert.equal(g.time, t);
  const d = started(); frames(d, { forward: true, right: true }, 30); assert.ok(Math.abs(Math.hypot(d.x,
d.z - 4) - 1.5) < .01);
});
test('jump lands, held jump does not bounce, falling consumes hearts and ends the run', () => {
  const g = started(); frames(g, { jump: true }, 150); assert.equal(g.ground, 'start'); assert.equal(g.y,
0);
  for (let i = 0; i < 3; i++) { g.y = -11; tick(g, {}, 1 / 120); }
  assert.equal(g.health, 0); assert.equal(g.status, 'gameover'); assert.equal(g.deaths, 3);
});
test('coins can only be collected once and remain collected after a fall', () => {
  const g = started(); g.z = COINS[0][2]; frames(g, {}, 3); assert.equal(g.coins.size, 1);
  frames(g, {}, 120); assert.equal(g.coins.size, 1); g.y = -11; tick(g, {}, 1 / 120);
assert.equal(g.coins.size, 1);
});
test('checkpoint restores health and sets the new respawn location', () => {
  const g = started(); g.health = 1; g.z = -29; g.y = 1; frames(g, {}, 3);
  assert.ok(g.checkpoint); assert.equal(g.health, 3); g.y = -12; tick(g, {}, 1 / 120);
  assert.equal(g.z, -29); assert.equal(g.health, 2); assert.equal(g.ground, 'checkpoint');
});
test('spikes damage on contact, grace period prevents repeated damage, jumping clears them', () => {
  const g = started(); g.x = 4; g.z = -20; g.y = .7; frames(g, {}, 1); assert.equal(g.health, 2);
  g.x = 4; g.z = -20; g.y = .7; frames(g, {}, 1); assert.equal(g.health, 2);
  g.invincible = 0; g.y = 2; frames(g, {}, 1); assert.equal(g.health, 2);
});
test('rotating bar collider follows the visible angle and has jump clearance', () => {
  const g = started(), r = ROTATORS[0]; g.time = 0; g.x = 2; g.z = -10; g.y = .4;
  assert.equal(touchesRotator(g, r), true); g.y = 1.6; assert.equal(touchesRotator(g, r), false);
  g.y = .4; g.time = Math.PI / (2 * r.speed); assert.equal(touchesRotator(g, r), false);
  g.time = 0; frames(g, {}, 1); assert.equal(g.health, 2);
});
test('moving platform carries an idle rider without slipping', () => {
  const g = started(); g.z = -40; g.y = 1; g.ground = 'ferry'; frames(g, {}, 360);
  assert.equal(g.ground, 'ferry'); assert.ok(Math.abs(g.x - platformAt(PLATFORMS[4], g.time).x) < .01);
});
test('all island-to-island gaps fit within the jump envelope', () => {
  for (let i = 0; i < PLATFORMS.length - 1; i++) {
    const a = platformAt(PLATFORMS[i], 0), b = platformAt(PLATFORMS[i + 1], 1), g = started();
    g.x = a.x; g.z = a.z - a.d / 2 + .7; g.y = a.y; g.ground = a.id; g.invincible = 10;
    let landed = false;
    for (let n = 0; n < 180; n++) {
      tick(g, { jump: n === 0, forward: g.z > b.z + 1, left: g.x > b.x + .1, right: g.x < b.x - .1 }, 1 /
120);
      if (g.ground === b.id) { landed = true; break; }
    }
    assert.ok(landed, `${a.id} -> ${b.id}`);
  }
});
test('full unmodified run reaches victory with coins, all three hearts and an activated checkpoint', () =>
 {
  const g = started();
  // These waypoints steer only through ordinary movement and jump input. No teleports or immunity.
  const route = [
    [0, -3.4, false], [2.7, -7.5, true], [2.7, -12, true], [4, -16.5, true],
    [6.4, -21.7, false], [2, -22.4, false], [0, -27, true], [0, -33.8, false],
    [0, -38.5, true], ['ride', -41.8, false], [-3, -45.8, true], [-3, -47, false],
    [-5.6, -51.8, true], [-2.3, -53.02, true], [0, -56.5, true], [0, -63.5, false],
  ];
  for (let i = 0; i < route.length; i++) {
    let [x, z, jump] = route[i]; const ride = x === 'ride'; let reached = false;
    for (let n = 0; n < 600; n++) {
      if (ride) x = platformAt(PLATFORMS[4], g.time).x;
```

```
      const dx = x - g.x, dz = z - g.z;
      if (Math.hypot(dx, dz) < .18 && (!jump || g.ground) && (!ride || x < -1)) { reached = true; break; }
      tick(g, { left: dx < -.1, right: dx > .1, forward: dz < -.1, back: dz > .1, jump: jump && n < 90 },
1 / 120);
      assert.equal(g.health, 3, `damage on waypoint ${i}`);
      if (g.status === 'won') { reached = true; break; }
    }
    assert.ok(reached, `waypoint ${i} was not reachable`);
  }
  assert.equal(g.status, 'won'); assert.ok(g.checkpoint); assert.ok(g.coins.size >= 12);
assert.equal(g.deaths, 0);
  const t = g.time; frames(g, { forward: true }, 100); assert.equal(g.time, t);
});
test('restart resets everything, and local best time retains the fastest completed run', () => {
  const g = started(); g.z = -63.5; g.y = 1.2; frames(g, {}, 1); assert.equal(g.status, 'won');
  g.coins.add(0); restart(g); assert.equal(g.coins.size, 0); assert.equal(g.checkpoint, false);
assert.equal(g.time, 0);
  const memory = new Map(), storage = { getItem: k => memory.get(k), setItem: (k, v) => memory.set(k, v)
};
  assert.equal(readBest(storage), null); assert.equal(saveBest(storage, 30), 30);
assert.equal(saveBest(storage, 35), 30); assert.equal(saveBest(storage, 27), 27);
  assert.equal(formatTime(65.123), '01:05.12'); assert.equal(formatTime(59.999), '00:59.99');
});
test('blocked or corrupted storage cannot break the game', () => {
  const broken = { getItem() { throw Error('denied'); }, setItem() { throw Error('denied'); } };
  assert.equal(readBest(broken), null); assert.equal(saveBest(broken, 10), 10);
  assert.equal(readBest({ getItem: () => 'Infinity' }), null); assert.equal(readBest({ getItem: () => '-1'
 }), null);
});

test('world 2 has a different map, its own checkpoint, and two moving platforms', async () => {
  const { getWorld } = await import('./game.js');
  const a = getWorld(0), b = getWorld(1);
  assert.notDeepEqual(a.platforms, b.platforms); assert.equal(b.platforms.filter(p => p.moving).length,
2);
  const g = createGame(1); restart(g); [g.x, g.y, g.z] = b.checkpoint; g.health = 1;
  frames(g, {}, 1); assert.equal(g.health, 3); g.y = -10; frames(g, {}, 1);
  assert.deepEqual([g.x, g.y, g.z], b.checkpoint); assert.equal(g.health, 2);
});
test('world unlock and best records persist independently and tolerate blocked storage', async () => {
  const { completeWorld, readProgress } = await import('./game.js');
  const memory = new Map(), storage = { getItem: k => memory.get(k), setItem: (k, v) => memory.set(k, v)
};
  assert.equal(readProgress(storage).unlocked, 0);
  assert.equal(completeWorld(storage, 0).unlocked, 1); assert.equal(readProgress(storage).completed[0],
true);
  completeWorld(storage, 1); assert.deepEqual(readProgress(storage).completed.slice(0,2), [true, true]);
  saveBest(storage, 20, 0); saveBest(storage, 30, 1); assert.equal(readBest(storage, 0), 20);
assert.equal(readBest(storage, 1), 30);
  const blocked = {getItem(){throw Error('denied')},setItem(){throw Error('denied')}};
  assert.equal(completeWorld(blocked, 0).unlocked, 1);
});
test('world 2 completes through actual input with both ferries, no damage, and the correct portal', async
() => {
  const { getWorld } = await import('./game.js');
  const g = createGame(1); restart(g); const level = getWorld(1);
  const route = [[-3,-3.4,false],[-5.6,-7.5,true],[-5.6,-12,true],[-6,-17,true],[-8,-23,false],[-4.6,-24,f
alse],[-1,-28,true],[1,-34.8,false],[2,-39.1,true],['ride',-42.8,false],[5,-46.5,true],[5,-47,false],[7.6,
-52.2,true],[4,-53.1,true],[2,-58.4,true],['ride2',-61.8,false],[-1,-65.7,true],[-1,-72.5,false]];
  for (let i = 0; i < route.length; i++) {
    let [x,z,jump] = route[i]; const ride=x==='ride'||x==='ride2'; const
p=level.platforms[x==='ride2'?6:4]; let reached=false;
    for(let n=0;n<1000;n++) {
      if(ride)x=platformAt(p,g.time).x; const dx=x-g.x,dz=z-g.z;
      if(Math.hypot(dx,dz)<.18&&(!jump||g.ground)&&(!ride||(i===9?x>3:x<1.5))){reached=true;break}
      tick(g,{left:dx<-.1,right:dx>.1,forward:dz<-.1,back:dz>.1,jump:jump&&n<90},1/120);
      assert.equal(g.health,3,`world 2 damage at waypoint ${i}`);
      if(g.status==='won'){reached=true;break}
    }
    assert.ok(reached,`world 2 waypoint ${i}`);
  }
  assert.equal(g.status,'won'); assert.equal(g.checkpoint,true); assert.equal(g.deaths,0);
assert.ok(g.coins.size>=10);
});

function runWorld(world){
 const g=createGame(world.id);restart(g);
 function go(x,z,jump=false,moving=null){let reached=false;for(let n=0;n<1400;n++){
   const targetX=moving?platformAt(moving,g.time).x:x;const dx=targetX-g.x,dz=z-g.z;
```

```
    if(Math.hypot(dx,dz)<.17&&(!jump||g.ground)){reached=true;break}
    tick(g,{left:dx<-.08,right:dx>.08,forward:dz<-.08,back:dz>.08,jump:jump&&n<90},1/120);
   if(g.status==='won')return true;if(g.health!==3)throw Error(`damage at target ${x},${z} time ${g.time}`);
  }
  if(!reached)throw Error(`unreachable ${x},${z} at ${g.x},${g.z} on ${g.ground}`);
  tick(g,{},1/120);return true;
  }
  for(let i=0;i<world.platforms.length;i++){
   const p=world.platforms[i],next=world.platforms[i+1];
   if(i){go(p.x,p.z+p.d/2-.55,true);if(g.ground!==p.id)throw Error(`landed on ${g.ground}, expected ${p.id}`);}
   if(!next){go(world.portal[0],world.portal[2]);break;}
   if(p.moving){go(p.x,p.z-p.d/2+.55,false,p);}
   else{
    const front=i?p.z+p.d/2-.55:g.z;
    go(p.x+3.5,front);go(p.x+3.5,p.z-p.d/2+.55);go(next.x,p.z-p.d/2+.55);
   }
  }
  if(g.status!=='won')throw Error('not won');return g;
}

for (const id of Array.from({length:10},(_,i)=>i+2)) {
  test(`world ${id+1} full route reaches its portal without damage, teleports, or invincibility`,async()=>{
    const {getWorld}=await import('./game.js');
    const g=runWorld(getWorld(id));assert.equal(g.status,'won');assert.equal(g.deaths,0);assert.ok(g.checkpoint);assert.equal(g.health,3);
  });
}
test('twelve routes and themes are distinct, valid and independently recorded',async()=>{
 const {WORLDS,readProgress,completeWorld}=await import('./game.js');
 const {getTheme}=await import('./worlds.js');
 assert.equal(WORLDS.length,12);
 assert.equal(new Set(WORLDS.map(w=>JSON.stringify(w.platforms))).size,12);
 assert.equal(new Set(WORLDS.map(w=>getTheme(w).name)).size,12);
 const memory=new Map(),storage={getItem:k=>memory.get(k),setItem:(k,v)=>memory.set(k,v)};
 // Existing two-world saves migrate without losing completion or best times.
 storage.setItem('adventure-runner-worlds-v1',JSON.stringify({unlocked:1,completed:[true,true]}));
 assert.equal(readProgress(storage).completed.length,12);assert.ok(readProgress(storage).completed[0]);assert.ok(readProgress(storage).completed[1]);
 for(const w of WORLDS){completeWorld(storage,w.id);saveBest(storage,30+w.id,w.id);assert.equal(readBest(storage,w.id),30+w.id);assert.ok(w.platforms.some(p=>p.id==='checkpoint'));assert.ok(w.coins.length>10);}

assert.ok(readProgress(storage).completed.every(Boolean));assert.equal(readProgress(storage).unlocked,11);
 completeWorld(storage,99);assert.equal(readProgress(storage).completed.length,12);
});

test('star ratings persist as the best completed run and never downgrade',async()=>{
 const {completeWorld,readProgress,starsForRun,recordRunProgress}=await import('./game.js');
 const m=new Map(),s={getItem:k=>m.get(k),setItem:(k,v)=>m.set(k,v)};
 assert.equal(starsForRun(0,22),1);assert.equal(starsForRun(14,22),2);assert.equal(starsForRun(22,22),3);
 completeWorld(s,0,14);assert.equal(readProgress(s).stars[0],2);
 completeWorld(s,0,2);assert.equal(readProgress(s).stars[0],2);
 completeWorld(s,0,22);assert.equal(readProgress(s).stars[0],3);
 const g=createGame(3);restart(g);g.coins.add(0);g.checkpoint=true;recordRunProgress(s,g);
 const p=readProgress(s);assert.ok(p.played[3]);assert.equal(p.completed[3],false);assert.equal(p.stars[3],0);assert.equal(p.tasks[3].coins,1);assert.ok(p.tasks[3].checkpoint);
});
test('continue restores time, coins, hearts, world and last safe island across reload',async()=>{
 const {saveRun,readRun,restoreRun,getWorld,clearRun}=await import('./game.js');
 const m=new Map(),s={getItem:k=>m.get(k),setItem:(k,v)=>m.set(k,v)};
 const g=createGame(6);restart(g);const cp=getWorld(6).checkpoint;[g.x,g.y,g.z]=cp;frames(g,{},2);g.time=45.75;g.health=2;g.coins.add(2);g.coins.add(3);g.deaths=1;g.status='paused';
 assert.ok(saveRun(s,g));assert.equal(readRun(s).world,6);
 const restored=restoreRun(s);assert.equal(restored.world,6);assert.equal(restored.time,45.75);assert.equal(restored.health,2);assert.equal(restored.coins.size,g.coins.size);assert.equal(restored.checkpoint,true);assert.equal(restored.status,'playing');assert.equal(restored.deaths,1);assert.equal(restored.ground,'checkpoint');assert.ok(restored.invincible>0);
 clearRun(s);assert.equal(readRun(s),null);
});
test('airborne or moving-platform autosave returns to the last solid ground safely',async()=>{
 const {saveRun,restoreRun}=await import('./game.js');const m=new Map(),s={getItem:k=>m.get(k),setItem:(k,v)=>m.set(k,v)};
 const g=createGame();restart(g);frames(g,{},1);const safe={...g.lastSafe};frames(g,{jump:true,forward:true},30);assert.equal(g.ground,null);saveRun(s,g);
 let r=restoreRun(s);assert.deepEqual([r.x,r.y,r.z],[safe.x,safe.y,safe.z]);
 g.ground='ferry';g.x=1;g.y=1;g.z=-40;saveRun(s,g);r=restoreRun(s);assert.equal(r.ground,'start');assert.equal(r.z,safe.z);
```

```
});
test('bad saves and unavailable browser storage do not break continue',async()=>{
 const {saveRun,readRun,restoreRun,resumeKey}=await import('./game.js');
 const denied={getItem(){throw Error('denied')},setItem(){throw Error('denied')}};
 const g=createGame();restart(g);assert.equal(saveRun(denied,g),null);assert.equal(restoreRun(denied),null);
 const m=new Map(),s={getItem:k=>m.get(k),setItem:(k,v)=>m.set(k,v)};saveRun(s,g);
 const data=JSON.parse(s.getItem(resumeKey));data.safe.ground='ferry';s.setItem(resumeKey,JSON.stringify(data));assert.equal(readRun(s),null);
 data.safe.ground='start';data.safe.x=9999;s.setItem(resumeKey,JSON.stringify(data));assert.equal(readRun(s),null);
 data.world=999;s.setItem(resumeKey,JSON.stringify(data));assert.equal(readRun(s),null);
});

test('joystick dead zone removes drift and diagonal movement never exceeds full speed',async()=>{
 const {stickVector,movementVector}=await import('./controls.js');
 assert.equal(stickVector(2,1,48).x,0);assert.equal(stickVector(2,1,48).z,0);
 const full=stickVector(100,-100,48);assert.ok(Math.abs(Math.hypot(full.x,full.z)-1)<.0001);assert.ok(Math.abs(Math.hypot(full.thumbX,full.thumbY)-48)<.0001);
 const precise=stickVector(24,0,48);assert.ok(precise.x>0&&precise.x<.5);
 assert.deepEqual(stickVector(NaN,0),{x:0,z:0,thumbX:0,thumbY:0});
 const mixed=movementVector({right:true,axisX:1,axisZ:-1});assert.ok(Math.hypot(mixed.x,mixed.z)<=1.00001);
});
test('two touch pointers can steer and jump independently, including cancellation',async()=>{
 const {createTouchController}=await import('./controls.js');const input={axisX:0,axisZ:0,jump:false},c=createTouchController(input);
 assert.ok(c.beginStick(1));c.moveStick(1,.5,-.8);assert.ok(c.beginJump(2));assert.equal(input.jump,true);
 assert.equal(c.beginStick(3),false);c.endStick(3);assert.equal(input.axisZ,-.8);
 c.endJump(2);assert.equal(input.jump,false);assert.equal(input.axisZ,-.8);
 c.beginJump(4);c.endStick(1);assert.equal(input.axisX,0);assert.equal(input.axisZ,0);assert.equal(input.jump,true);
 c.reset();assert.equal(input.jump,false);assert.equal(input.axisX,0);assert.ok(c.beginStick(6));
});
test('analogue speed supports precise walking, running and simultaneous jumping',()=>{
 const slow=started(),fast=started();frames(slow,{axisZ:-.3},60);frames(fast,{axisZ:-1},60);
 assert.ok(Math.abs((4-slow.z)-.9)<.001);assert.ok(Math.abs((4-fast.z)-3)<.001);
 tick(slow,{axisZ:-.3,jump:true},1/120);assert.ok(slow.y>0);assert.equal(slow.ground,null);
 const stopped=slow.z;frames(slow,{axisZ:0,axisX:0},10);assert.equal(slow.z,stopped);
});
test('landing preview predicts a real surface and hides while grounded',async()=>{
 const {landingPreview}=await import('./game.js');const g=started();assert.equal(landingPreview(g),null);
 frames(g,{jump:true,axisZ:-1},20);const target=landingPreview(g);assert.ok(target);assert.ok(Number.isFinite(target.x));assert.ok(target.t>0);
 const endY=target.y-.035;let actual=false;for(let i=0;i<180;i++){tick(g,{axisZ:-1},1/120);if(g.ground){actual=true;break;}}
 assert.ok(actual);assert.ok(Math.abs(g.y-endY)<.01);
});

test('a quick jump tap between physics frames is buffered and fires exactly once',async()=>{
 const {createTouchController}=await import('./controls.js');const input={axisX:0,axisZ:0,jump:false,jumpQueued:false};const c=createTouchController(input),g=started();
 c.beginJump(2);c.endJump(2);assert.equal(input.jump,false);assert.equal(input.jumpQueued,true);
 tick(g,input,1/120);assert.ok(g.vy>0);assert.equal(input.jumpQueued,false);assert.equal(g.ground,null);
 frames(g,input,150);assert.equal(g.ground,'start');assert.equal(g.events.filter(e=>e==='jump').length,1);
 c.beginJump(3);c.reset();assert.equal(input.jumpQueued,false);
});


test('holding jump yields a higher arc than a quick tap, with one launch per press',()=>{
 const tap=started(),hold=started();let tapPeak=0,holdPeak=0;
 for(let i=0;i<160;i++){tick(tap,{jump:i===0},1/120);tick(hold,{jump:i<80},1/120);tapPeak=Math.max(tapPeak,tap.y);holdPeak=Math.max(holdPeak,hold.y);}
 assert.ok(holdPeak>tapPeak+.5);assert.equal(tap.events.filter(e=>e==='jump').length,1);assert.equal(hold.events.filter(e=>e==='jump').length,1);
});
test('coyote time and a buffered landing press are both usable',()=>{
 const g=started();g.z=-4.16;frames(g,{forward:true},2);assert.equal(g.ground,null);tick(g,{jump:true},1/120);assert.ok(g.vy>0);
 const b=started();b.y=.06;b.ground=null;b.coyote=0;b.vy=-3;tick(b,{jumpQueued:true},1/120);frames(b,{},5);assert.ok(b.vy>0);assert.equal(b.ground,null);
});
test('running, pickup, landing and respawn produce bounded lightweight effects',()=>{
 const g=started();g.z=1;frames(g,{forward:true},20);assert.ok(g.fx.some(e=>e.type==='coin'));assert.ok(g.fx.some(e=>e.type==='dust'));
 frames(g,{jump:true},90);frames(g,{},90);assert.ok(g.events.includes('land'));g.y=-11;tick(g,{},1/120);assert.ok(g.fade>0);assert.equal(g.health,2);
 for(let i=0;i<1000;i++)tick(g,{right:i%100<50,left:i%100>=50},1/120);assert.ok(g.fx.length<=32);
});
```

```
test('settings survive reload and device/slow-frame detection reduces quality',async()=>{
 const {loadSettings,storeSettings,deviceProfile,performanceStep}=await import('./preferences.js');const m=new Map(),s={getItem:k=>m.get(k),setItem:(k,v)=>m.set(k,v)};
 storeSettings(s,{muted:true,haptics:false});assert.deepEqual(loadSettings(s),{muted:true,haptics:false});assert.equal(deviceProfile({deviceMemory:2},true).quality,'low');assert.equal(deviceProfile({hardwareConcurrency:2}).pixelRatio,.85);assert.equal(deviceProfile({hardwareConcurrency:12,deviceMemory:16}).quality,'high');
 let state={samples:0,elapsed:0};for(let i=0;i<85;i++)state=performanceStep(state,.05,true);assert.equal(state.downgrade,true);assert.ok(state.fps<30);
 let fast={samples:0,elapsed:0};for(let i=0;i<245;i++)fast=performanceStep(fast,1/60,true);assert.equal(fast.downgrade,false);assert.ok(fast.fps>59);
});
test('haptics are guarded, optional and specific to game events',async()=>{
 const {feedback}=await import('./preferences.js');const calls=[],nav={vibrate:pattern=>{calls.push(pattern);return true;}};feedback('jump',nav);feedback('coin',nav);feedback('hurt',nav);assert.equal(calls.length,3);feedback('jump',nav,false);assert.equal(calls.length,3);assert.equal(feedback('jump',{}),false);
});
test('new celebration ratings use time, coin target and hearts without downgrading old records',async()=>{
 const {starsForRun,completeWorld,readProgress}=await import('./game.js');assert.equal(starsForRun(14,22,30,3,49),3);assert.equal(starsForRun(14,22,70,3,49),2);assert.equal(starsForRun(14,22,30,1,49),2);assert.equal(starsForRun(2,22,70,1,49),1);
 const m=new Map(),s={getItem:k=>m.get(k),setItem:(k,v)=>m.set(k,v)};completeWorld(s,0,22);completeWorld(s,0,2,100,1);assert.equal(readProgress(s).stars[0],3);
});


test('audio and music stay lazy until a gesture, and music stops when muted or paused',async()=>{
 const audio=await import('./audio.js');assert.equal(audio.audioStatus().created,false);audio.enableSound(false);assert.equal(audio.audioStatus().created,false);
 const events=[];const param={setValueAtTime(){},linearRampToValueAtTime(){},exponentialRampToValueAtTime(){},setTargetAtTime(v){events.push(['gain',v]);}};
 globalThis.window={AudioContext:class{state='running';currentTime=0;destination={};createGain(){return {gain:param,connect(){},disconnect(){}}}createOscillator(){events.push(['note']);return {frequency:param,connect(){},disconnect(){},start(){},stop(){}}}}};
 audio.unlockAudio();assert.equal(audio.audioStatus().created,true);audio.enableSound(true);audio.setMusicPlaying(true);assert.ok(audio.audioStatus().scheduled);assert.ok(events.some(e=>e[0]==='note'));audio.enableSound(false);assert.equal(audio.audioStatus().scheduled,false);audio.setMusicPlaying(false);delete globalThis.window;
});

test('static scenery baking preserves colors and local transforms including moving-parent instances',async()=>{
 const THREE=await import('three');const {bakeIsland}=await import('./batchGeometry.js');
 const root=new THREE.Group();root.position.set(40,3,-20);
 const child=new THREE.Mesh(new THREE.BoxGeometry(2,2,2),new THREE.MeshStandardMaterial({color:'#ff0000'}));child.position.x=4;root.add(child);
 const instances=new THREE.InstancedMesh(new THREE.BoxGeometry(),new THREE.MeshStandardMaterial({color:'#00ff00'}),2);
 instances.setMatrixAt(0,new THREE.Matrix4().makeTranslation(-3,0,0));instances.setMatrixAt(1,new THREE.Matrix4().makeTranslation(-6,0,0));root.add(instances);
 const animated=new THREE.Mesh(new THREE.PlaneGeometry(),new THREE.ShaderMaterial());root.add(animated);
 const baked=bakeIsland(root);assert.equal(baked.originals.length,2);assert.ok(!baked.originals.includes(animated));
 baked.geometry.computeBoundingBox();assert.equal(baked.geometry.boundingBox.max.x,5);assert.equal(baked.geometry.boundingBox.min.x,-6.5);
 assert.equal(baked.geometry.getAttribute('position').count,108);assert.equal(baked.geometry.getAttribute('color').getX(0),1);
 assert.equal(baked.geometry.getAttribute('color').getY(36),1);assert.equal(child.visible,true);baked.geometry.dispose();
});

test('touch profile retains readable resolution and reacts before frame rate reaches 30',async()=>{
 const {deviceProfile,performanceStep}=await import('./preferences.js');assert.equal(deviceProfile({hardwareConcurrency:6},true).pixelRatio,1.25);
 let state={samples:0,elapsed:0};for(let i=0;i<95;i++)state=performanceStep(state,1/45);assert.equal(state.downgrade,true);assert.ok(state.fps>44&&state.fps<46);
});
```

Code 11 / src/Icon.jsx

5 original lines | 1102 bytes | complete file

Reusable SVG HUD icons.

```
1  import React from 'react';
2  export default function Icon({name,size=20,...props}){
3   const paths={map:<><path d="m3 5 6-2 6 2 6-2v16l-6 2-6-2-6 2z"/><path d="M9
+  3v16m6-14v16"/></>,sound:<><path d="m11 4-5 4H3v8h3l5 4z"/><path d="M15 8a6 6 0 0 1 0 8m3-11a10 10 0 0 1 0
+   14"/></>,muted:<><path d="m11 4-5 4H3v8h3l5 4z"/><path d="m16 9 6 6m0-6-6 6"/></>,pause:<><path d="M8
+  5v14m8-14v14"/></>,play:<path d="m8 4 12 8-12 8z"/>,expand:<path d="M9 3H3v6m12-6h6v6M3
+  15v6h6m12-6v6h-6"/>,heart:<path d="M20.8 4.7a5.5 5.5 0 0 0-7.8 0L12 5.8l-1.1-1.1a5.5 5.5 0 0 0-7.8 7.8L12
+  21l8.8-8.5a5.5 5.5 0 0 0 0-7.8z"/>,flag:<><path d="M5 22V3m0 0c5-4 9 4 14 0v11c-5 4-9-4-14
+  0"/></>,check:<path d="m4 12 5 5L21 5"/>,sun:<><circle cx="12" cy="12" r="4"/><path d="M12 2v2m0 16v2M2
+  12h2m16 0h2M5 5l1 1m12 12 1 1M5 19l1-1M18 6l1-1"/></>,chevron:<path d="m6 9 6 6 6-6"/>};
4   return <svg width={size} height={size} viewBox="0 0 24 24" fill={name==='heart'?'currentColor':'none'}
+  stroke="currentColor" strokeWidth="1.8" strokeLinecap="round" strokeLinejoin="round" aria-hidden="true"
+  {...props}>{paths[name]||paths.check}</svg>;
5  }
```

Code 12 / src/main.jsx

190 original lines | 23009 bytes | complete file

Application shell, lifecycle, keyboard input, menu/map, HUD snapshots, settings, save triggers and event dispatch.

```
import React, { Component, Suspense, lazy, useCallback, useEffect, useRef, useState } from 'react';
import { createRoot } from 'react-dom/client';
import { Canvas } from '@react-three/fiber';
import { WORLDS, getWorld, createGame, restart, formatTime, readBest, saveBest, readProgress,
completeWorld, recordRunProgress, saveRun, readRun, restoreRun, clearRun, starsForRun,runMedals } from
'./game';
import { enableSound, playSound, unlockAudio,setMusicPlaying } from './audio';
import './style.css';
import MobileControls from './MobileControls';
import Icon from './Icon';
import {loadSettings,storeSettings,deviceProfile,feedback} from './preferences';
import {PCFSoftShadowMap} from 'three';
import { getTheme } from './worlds';
const Scene = lazy(() => import('./Scene'));
const storage = { getItem: key => window.localStorage.getItem(key), setItem: (key, value) =>
window.localStorage.setItem(key, value) };
const input = { forward: false, back: false, left: false, right: false, jump:
false,jumpQueued:false,axisX:0,axisZ:0 };
const clearInput = () => Object.keys(input).forEach(key => { input[key] = key.startsWith('axis')?0:false;
});
const getSnapshot = game => ({ world: game.world, status: game.status, coins: game.coins.size, time:
game.time, health: game.health, checkpoint: game.checkpoint, z: game.z, toast: game.toastTime > 0 ?
game.toast : '', deaths: game.deaths,fade:game.fade });
class SceneBoundary extends Component {
  state = { error: false };
  static getDerivedStateFromError() { return { error: true }; }
  render() {
    return this.state.error ? <div className="graphics-error"><h2>The 3D world couldn’t
start</h2><p>Enable hardware acceleration in your browser, then reload.</p><button onClick={() =>
location.reload()}>Reload game</button></div> : this.props.children;
  }
}
function CoinIcon() { return <span className="coin-icon" aria-hidden="true">★</span>; }
function WorldRoute({ world }) {
  const level = getWorld(world);
  const path = level.platforms.map(p => `${p.x},${p.z}`).join(' ');
  return <svg className="route-map" viewBox={`-15 ${level.portal[2]-10} 32 ${25-level.portal[2]}`}
aria-label={`World ${world+1} route, ${level.platforms.length} islands`} role="img">
    <polyline points={path} fill="none" stroke={getTheme(level).accent} strokeWidth=".7"
strokeDasharray="1.4 1.4"/>
    {level.platforms.map(p => <rect key={p.id} x={p.x-p.w/2} y={p.z-p.d/2} width={p.w} height={p.d} rx="1"
 fill={p.moving ? '#ffdc76' : getTheme(level).ground} stroke="#f6f4c255" strokeWidth=".35"/>)}
    <circle cx={level.checkpoint[0]} cy={level.checkpoint[2]} r="1.6" fill="#ff7862"/>
    <circle cx={level.portal[0]} cy={level.portal[2]} r="2.2" fill="#91efff"/>
  </svg>;
}
function App() {
  const game = useRef(createGame()).current;
  const [view, setView] = useState(() => getSnapshot(game));
  const [best, setBest] = useState(() => readBest(storage));
  const [preferences,setPreferences]=useState(()=>loadSettings(storage));
  const [sound,setSound]=useState(()=>!loadSettings(storage).muted);
  const feedbackSettings=useRef(preferences);feedbackSettings.current=preferences;
  const [taskOpen,setTaskOpen]=useState(false);
  const [hitId,setHitId]=useState(0);
  const [performanceMode,setPerformanceMode]=useState(false);
  const [fps,setFps]=useState(null);
  const [renderCalls,setRenderCalls]=useState(null);
  const [device]=useState(()=>deviceProfile(navigator,matchMedia('(any-pointer: coarse)').matches));
  const [atlasOpen, setAtlasOpen] = useState(false);
  const [resumeData,setResumeData]=useState(()=>readRun(storage));
  const [mapFilter,setMapFilter]=useState('all');
  const [progress, setProgress] = useState(() => readProgress(storage));
  const [mobile,setMobile]=useState(()=>matchMedia('(any-pointer: coarse), (max-width: 900px), (max-width:
 950px) and (max-height: 500px)').matches);
  useEffect(()=>{const query=matchMedia('(any-pointer: coarse), (max-width: 900px), (max-width: 950px) and
 (max-height: 500px)');const changed=()=>setMobile(query.matches);query.addEventListener('change',changed)
;return()=>query.removeEventListener('change',changed);},[]);
  const [quality,setQuality]=useState(()=>device.quality);
  const slowFrames=useCallback(()=>{setPerformanceMode(true);setQuality('low');},[]);
  const reportFrames=useCallback(value=>{setFps(Math.round(value.fps));setRenderCalls(value.calls);},[]);
```

```
  const [cameraMode, setCameraMode] = useState('follow');
  const [loading, setLoading] = useState(true);
  const [newBest, setNewBest] = useState(false);
  const [fullscreen, setFullscreen] = useState(false);
  const lastAutosave=useRef(0);
  const lastUpdate = useRef(0), savedRun = useRef(false), loaded = useRef(false), panelRef = useRef();
  const captureRun=useCallback(()=>{const data=saveRun(storage,game);if(data)setResumeData(data);setProgre
ss(recordRunProgress(storage,game));lastAutosave.current=performance.now();},[game]);
  const sync = useCallback(() => {
    const events = game.events.splice(0);
    events.forEach(event=>{playSound(event);feedback(event,navigator,feedbackSettings.current.haptics);});
    if(events.includes('hurt'))setHitId(n=>n+1);
    if (game.status === 'won' && !savedRun.current) {
      savedRun.current = true;
      const old = readBest(storage, game.world);
      setNewBest(!old || game.time < old);
      setBest(saveBest(storage, game.time, game.world));
      recordRunProgress(storage,game);setProgress(completeWorld(storage,
game.world,game.coins.size,game.time,game.health));clearRun(storage);setResumeData(null);
    }
    if(game.status==='playing'&&(performance.now()-lastAutosave.current>1800||events.some(e=>['checkpoint'
,'start'].includes(e))))captureRun();
    if(game.status==='gameover'&&events.includes('hurt')){recordRunProgress(storage,game);clearRun(storage
);setResumeData(null);}
    if (!loaded.current) { loaded.current = true; setLoading(false); }
    if ((game.status === 'playing' && performance.now() - lastUpdate.current > 75) || events.length) {
      lastUpdate.current = performance.now(); setView(getSnapshot(game));
    }
  }, [game,captureRun]);
  const begin = useCallback((world = game.world) => {
    setAtlasOpen(false); unlockAudio(); clearInput(); restart(game, world); setBest(readBest(storage,
world)); savedRun.current = false;
    setNewBest(false); captureRun();setView(getSnapshot(game));
  }, [game,captureRun]);
  const pause = useCallback(() => {
    if (game.status === 'playing') {game.status = 'paused';captureRun();}
    else if (game.status === 'paused') { game.status = 'playing';setAtlasOpen(false); unlockAudio(); }
    clearInput(); setView(getSnapshot(game));
  }, [game,captureRun]);
  const openMap=useCallback(()=>{if(!loaded.current)return;if(game.status==='playing')pause();setProgress(
recordRunProgress(storage,game));setMapFilter('all');setAtlasOpen(true);clearInput();},[game,pause]);
  const continueAdventure=()=>{const restored=restoreRun(storage);if(!restored)return;Object.assign(game,r
estored);setAtlasOpen(false);savedRun.current=false;setNewBest(false);clearInput();unlockAudio();setBest(r
eadBest(storage,game.world));setView(getSnapshot(game));};
  useEffect(() => {
    const map = { KeyW: 'forward', ArrowUp: 'forward', KeyS: 'back', ArrowDown: 'back', KeyA: 'left',
ArrowLeft: 'left', KeyD: 'right', ArrowRight: 'right', Space: 'jump' };
    const held = new Set();
    const updateKeys = () => { Object.values(map).forEach(action => { input[action] = [...held].some(code
=> map[code] === action); }); };
    const down = event => {
      if (event.target instanceof HTMLSelectElement) return;
      if (map[event.code]) { event.preventDefault(); if (game.status === 'playing') {
held.add(event.code);if(event.code==='Space'&&!event.repeat)input.jumpQueued=true; updateKeys(); } }
      if ((event.code === 'Escape' || event.code === 'KeyP') && !event.repeat) {
held.clear();if(atlasOpen){setAtlasOpen(false);if(game.status==='paused')pause();}else pause(); }
      if(event.code==='KeyM'&&!event.repeat)openMap();
      if (event.code === 'KeyR' && !event.repeat && ['won', 'gameover'].includes(game.status)) begin();
    };
    const up = event => { if (map[event.code]) { event.preventDefault(); held.delete(event.code);
updateKeys(); } };
    const blur = () => { held.clear(); clearInput(); if (game.status === 'playing') { game.status =
'paused';captureRun(); setView(getSnapshot(game)); } };
    const visibility = () => { if (document.hidden) blur(); };
    const pagehide=()=>{saveRun(storage,game);recordRunProgress(storage,game);};
    const full = () => setFullscreen(!!document.fullscreenElement);
    window.addEventListener('keydown', down); window.addEventListener('keyup', up);
window.addEventListener('blur', blur);
    document.addEventListener('visibilitychange', visibility);
document.addEventListener('fullscreenchange', full);window.addEventListener('pagehide',pagehide);
    return () => { window.removeEventListener('keydown', down); window.removeEventListener('keyup', up);
window.removeEventListener('blur', blur); document.removeEventListener('visibilitychange', visibility);
document.removeEventListener('fullscreenchange', full);window.removeEventListener('pagehide',pagehide); };
  }, [game, pause, begin,atlasOpen,openMap,captureRun]);
  useEffect(()=>{enableSound(sound);setPreferences(old=>({...old,muted:!sound}));},[sound]);
  useEffect(()=>{storeSettings(storage,preferences);},[preferences]);

useEffect(()=>{setMusicPlaying(view.status==='playing');return()=>setMusicPlaying(false);},[view.status]);
  useEffect(()=>{const suppress=e=>{if(e.touches?.length>1||e.type==='gesturestart')e.preventDefault();};d
```

```
+ ocument.addEventListener('gesturestart',suppress,{passive:false});document.addEventListener('touchmove',su
+ ppress,{passive:false});return()=>{document.removeEventListener('gesturestart',suppress);document.removeEv
+ entListener('touchmove',suppress);};},[]);
117   useEffect(() => {
118     if (view.status !== 'playing') panelRef.current?.querySelector('button:not([hidden])')?.focus();
119   }, [view.status, loading,atlasOpen]);
120   const toggleSound = () => { unlockAudio(); setSound(value => !value); };
121   const toggleFullscreen = async () => {
122     try { if (document.fullscreenElement) await document.exitFullscreen(); else await
+ document.documentElement.requestFullscreen(); }
123     catch { game.toast = 'Fullscreen unavailable · you can keep playing here'; game.toastTime = 3; }
124   };
125   const level = getWorld(view.world);
126   const selectWorld = id => {if(atlasOpen&&id===game.world&&game.status==='paused'){pause();return;}if(gam
+ e.status==='paused')captureRun();setAtlasOpen(false); clearInput(); Object.assign(game, createGame(id));
+ savedRun.current=false; setBest(readBest(storage,id)); setNewBest(false); setView(getSnapshot(game)); };
127   const menu = view.status !== 'playing';
128   const won = view.status === 'won';
129   const medals=runMedals(game);
130   const stars=starsForRun(view.coins,level.coins.length,view.time,view.health,medals.par);
131   const visibleWorlds=WORLDS.filter(w=>mapFilter==='all'||(mapFilter==='played'?progress.played[w.id]:prog
+ ress.completed[w.id]));
132   const stage = level.stages.find(([z]) => view.z > z)?.[1];
133   return <main data-fps={fps??''} data-draw-calls={renderCalls??''} data-quality={quality}
+ data-auto-performance={performanceMode}>
134     <SceneBoundary><Canvas shadows={quality === 'high'} dpr={quality==='high'?[1,1.5]:performanceMode ?
+ (device.low ? .85 : 1) : Math.min(1.25,device.pixelRatio)} camera={{ fov: 57, near: .1, far: 210 }}
+ gl={{antialias:!device.touch&&!device.low,powerPreference:'high-performance'}}
+ onCreated={({gl})=>{gl.shadowMap.type=PCFSoftShadowMap;}}>
135       <Suspense fallback={null}><Scene key={view.world} game={game} input={input} onStep={sync}
+ quality={quality} cameraMode={cameraMode} mobile={mobile} onSlow={slowFrames} onStats={reportFrames}
+ /></Suspense>
136     </Canvas></SceneBoundary>
137     <header className="topbar">
138       <div className="wordmark"><span>ADVENTURE</span><b>RUNNER <i>3D</i></b><small>WORLD {view.world+1} ·
+  {level.name}</small></div>
139       <div className="hud" aria-label="Game statistics">
140         <div className="coin-stat"><CoinIcon /><strong key={view.coins}
+ className="coin-count">{view.coins}</strong><small>/ {level.coins.length}</small></div>
141         <div className="time-stat"><span
+ className="stat-label">TIME</span><strong>{formatTime(view.time)}</strong></div>
142         <div key={hitId} className={`hearts ${hitId?'heart-hit':''}`} aria-label={`${view.health} of 3
+ hearts`}>{[1, 2, 3].map(n => <span key={n} className={n <= view.health ? '' : 'empty'}><Icon name="heart"
+ size={21}/></span>)}</div>
143       </div>
144       <div className="tools"><button className="map-button" title="Map (M)" aria-label="Open world map"
+ disabled={loading} onClick={openMap}><Icon name="map" size={17}/><span>MAP</span></button><button
+ title={sound ? 'Mute sound' : 'Enable sound'} aria-label={sound ? 'Mute sound' : 'Enable sound'}
+ onClick={toggleSound}><Icon name={sound?'sound':'muted'}/></button><button title={fullscreen ? 'Exit
+ fullscreen' : 'Fullscreen'} aria-label={fullscreen ? 'Exit fullscreen' : 'Fullscreen'}
+ onClick={toggleFullscreen}><Icon name="expand"/></button><button title="Pause (Esc)"
+ aria-label={view.status === 'paused' ? 'Resume game' : 'Pause game'} disabled={!['playing',
+ 'paused'].includes(view.status)} onClick={pause}><Icon
+ name={view.status==='paused'?'play':'pause'}/></button></div>
145     </header>
146     {!menu && <>
147       <div className="trail-progress"><span>{stage}</span><div><i style={{ width: `${Math.min(100,
+ Math.max(0, (level.spawn[2] - view.z) / (level.spawn[2] - level.portal[2]) * 100))}%` }} /></div><b
+ className={view.checkpoint ? 'lit' : ''} title={view.checkpoint ? 'Checkpoint active' : 'Reach the
+ checkpoint'}>⚑</b></div>
148       <section className={`task-panel ${taskOpen?'expanded':''}`}><button className="task-toggle"
+ aria-expanded={taskOpen} aria-controls="world-task-list" onClick={()=>setTaskOpen(value=>!value)}><Icon
+ name="flag" size={15}/><span>World
+ tasks</span><b>{Number(view.coins>=Math.ceil(level.coins.length*.6))+Number(view.checkpoint)} / 3</b><Icon
+  name="chevron" size={15}/></button>{taskOpen&&<div id="world-task-list" className="task-list"><span
+ className={view.coins>=Math.ceil(level.coins.length*.6)?'done':''}>★ Collect coins
+ {Math.min(view.coins,Math.ceil(level.coins.length*.6))} / {Math.ceil(level.coins.length*.6)}</span><span
+ className={view.checkpoint?'done':''}>⚑ Activate checkpoint</span><span>◈ Reach the
+ portal</span></div>}</section>
149       {mobile&&<div className="rotate-hint">Rotate for a wider view</div>}
150
151       <div className="desktop-help"><kbd>WASD</kbd> move <kbd>SPACE</kbd> jump <kbd>ESC</kbd> pause
+ <span>Follow the coins to the portal</span></div>
152       {mobile&&<MobileControls input={input}/>}
153     </>}
154     {view.fade>0&&!menu&&<div className="respawn-fade" style={{opacity:Math.min(.75,view.fade/.22)}}
+ aria-hidden="true"/>}
155     {view.toast && !menu && <div className="toast" role="status">✦ {view.toast}</div>}
156     {menu && <div className={`menu-layer ${won ? 'won' : ''}`}><section ref={panelRef}
```

```
+  className={`menu-card ${atlasOpen?'atlas-card':''}`} role="dialog" aria-modal="true"
+  aria-labelledby="menu-title" onKeyDown={event => {
157        if (event.key !== 'Tab') return;
158        const controls = panelRef.current.querySelectorAll('button:not(:disabled):not([hidden]), select');
159        const first = controls[0], last = controls[controls.length - 1];
160        if (event.shiftKey && document.activeElement === first) { event.preventDefault(); last.focus(); }
161        else if (!event.shiftKey && document.activeElement === last) { event.preventDefault();
+  first.focus(); }
162      }}>
163        <div className="chapter-label">WORLD {String(view.world+1).padStart(2,'0')} <span>•</span>
+  {level.name.toUpperCase()}</div>
164        {loading ? <><h1 id="menu-title">Building your world…</h1><p>Warming up the sky gate.</p><div
+  className="loading-bar" /></> : <>
165          <div className="menu-emblem">{won ? '★' : view.status === 'paused' ? 'II' : view.status ===
+  'gameover' ? '♥' : '✦'}</div>
166          <h1 id="menu-title">{atlasOpen ? 'Choose your world' : won ? 'Adventure complete!' : view.status
+  === 'paused' ? 'Take a breather.' : view.status === 'gameover' ? 'Another adventure?' : 'The sky is
+  calling.'}</h1>
167          <p>{atlasOpen ? `Explore ${WORLDS.length} worlds. Each has its own route, scenery and personal
+  best. All are available to play.` : won ? `You made it through ${level.name.toLowerCase()}.${view.world <
+  WORLDS.length-1 ? ` Next stop: ${WORLDS[view.world+1].name}.` : ' You reached the final world!'}` :
+  view.status === 'paused' ? 'Your progress is safe. The clock is paused.' : view.status === 'gameover' ?
+  'Every great explorer takes a few tumbles.' : 'Leap across floating islands, dodge the traps, and find
+  your way to the ancient sky gate.'}</p>
168          {won && !atlasOpen && <><div className="stars" aria-label={`${stars} of 3 stars`}>{[1, 2, 3].map(n
+   => <span key={n} className={n <= stars ? '' : 'empty'}>★</span>)}</div><div className="victory-burst"
+  aria-hidden="true">✦ ✧ ✦</div><div className="run-medals"><span className={medals.coins?'earned':''}>★
+  Coin target</span><span className={medals.time?'earned':''}>◷ Under {medals.par}s</span><span
+  className={medals.hearts?'earned':''}>♥ 2+ hearts</span></div><div className="results"><div><small>YOUR
+  TIME</small><b>{formatTime(view.time)}</b></div><div><small>COINS FOUND</small><b>{view.coins} /
+  {level.coins.length}</b></div><div><small>FALLS / HITS</small><b>{view.deaths}</b></div></div>{newBest &&
+  <div className="record">✦ NEW PERSONAL BEST</div>}</>}
169          {view.status==='ready'&&resumeData&&!atlasOpen&&<button className="continue-button"
+  onClick={continueAdventure}>▶ CONTINUE ADVENTURE <small>World {resumeData.world+1} ·
+  {getWorld(resumeData.world).name} · {formatTime(resumeData.time)}</small></button>}
170          <button className="primary" hidden={atlasOpen} onClick={view.status === 'paused' ? pause : () =>
+  begin(won && view.world < WORLDS.length-1 ? view.world+1 : game.world)}>{view.status === 'ready' ?
+  (resumeData?'START NEW RUN':'PLAY ADVENTURE') : view.status === 'paused' ? 'RESUME ADVENTURE' : won &&
+  view.world < WORLDS.length-1 ? `ENTER WORLD ${view.world+2}` : 'PLAY AGAIN'} <span>▶</span></button>
171          {view.status === 'paused' && !atlasOpen && <button className="secondary" onClick={() =>
+  begin()}>Restart from the beginning</button>}
172          {won && !atlasOpen && <button className="secondary" onClick={() => selectWorld(view.world)}>Replay
+   this world / World select</button>}
173          {view.status === 'ready' && !atlasOpen && <button className="atlas-toggle"
+  onClick={openMap}>Choose world <span>{WORLDS.length} destinations →</span></button>}
174          {view.status === 'paused' && !atlasOpen && <button className="secondary" onClick={openMap}>Map &
+  tasks / Choose world</button>}
175          {atlasOpen && <><div className="atlas-summary">{progress.completed.filter(Boolean).length} /
+  {WORLDS.length} worlds complete <button
+  onClick={()=>{setAtlasOpen(false);if(game.status==='paused')pause();}}>{view.status==='paused'?'Resume
+  game':'Back'}</button></div><nav className="map-filters" aria-label="Filter
+  maps">{['all','played','completed'].map(filter=><button key={filter} aria-pressed={mapFilter===filter}
+  onClick={()=>setMapFilter(filter)}>{filter==='all'?'All
+  worlds':filter==='played'?'Played':'Completed'}</button>)}</nav><div className="world-select"
+  aria-label="Choose a world">{visibleWorlds.map(world => <button key={world.id} className={`world-option
+  ${view.world===world.id?'selected':''} ${world.id===1?'ember':''}`}
+  style={{'--world-accent':getTheme(world).accent,'--world-ground':getTheme(world).ground}}
+  aria-pressed={view.world===world.id} onClick={() => selectWorld(world.id)}><WorldRoute
+  world={world.id}/><span><small>WORLD {String(world.id+1).padStart(2,'0')}
+  {progress.completed[world.id]?'✓':''}</small><b>{world.name}</b><span className="map-stars"
+  aria-label={`${progress.stars[world.id]} stars earned`}>{[1,2,3].map(n=><i key={n}
+  className={n<=progress.stars[world.id]?'earned':''}>★</i>)}</span><em>{`${world.platforms.length} islands
+  · ${world.platforms.filter(p=>p.moving).length} moving`}</em><em
+  className="map-tasks">{progress.tasks[world.id].coins>=Math.ceil(world.coins.length*.6)?'✓':'○'} Coins {Ma
+  th.min(progress.tasks[world.id].coins,Math.ceil(world.coins.length*.6))}/{Math.ceil(world.coins.length*.6)
+  } · {progress.tasks[world.id].checkpoint?'✓':'○'} Flag · {progress.completed[world.id]?'✓':'○'}
+  Gate</em>{view.status==='paused'&&world.id===view.world&&<em className="current-run">▶ Resume this
+  run</em>}</span></button>)}</div>{!visibleWorlds.length&&<p
+  className="empty-maps">{mapFilter==='played'?'Choose any world and start an adventure.':'Finish a world to
+   earn your first star.'}</p>}</>}
176          {!atlasOpen && ['ready', 'paused'].includes(view.status) && <>
177            <div className="instructions">{mobile?<><span>Left thumb: drag the joystick to
+  move</span><span>Right thumb: tap JUMP while steering</span><span>Hold JUMP for longer leaps · drag gently
+   for precise steps</span></>:<><span><kbd>W A S D</kbd> or arrows to move</span><span><kbd>SPACE</kbd> to
+  jump over gaps & red bars</span></>}<span>⚑ Checkpoint restores your three hearts</span></div>
178            <div className="settings"><label>Graphics<select aria-label="Graphics quality" value={quality}
+  onChange={event=>{setQuality(event.target.value);setPerformanceMode(false);}}><option value="high">High ·
+  shadows</option><option value="low">Smooth · no shadows</option></select></label><label>Camera<select
+  aria-label="Camera view" value={cameraMode} onChange={event => setCameraMode(event.target.value)}><option
```

```
+ value="follow">Follow</option><option value="wide">Wide</option></select></label><button
+ className="sound-setting" onClick={toggleSound} aria-pressed={sound}>Sound {sound ? 'ON' :
+ 'OFF'}</button></div><div className="extras-settings"><button aria-pressed={preferences.haptics}
+ onClick={()=>setPreferences(old=>({...old,haptics:!old.haptics}))}>Haptics
+ {preferences.haptics?'ON':'OFF'}</button><span>{performanceMode?'Adaptive performance
+ mode':device.low?'Low-power device detected':'Adaptive quality enabled'}{fps?` · ${fps}
+ fps`:''}</span></div>
179          </>}
180          {best && !atlasOpen && <div className="best">PERSONAL BEST
+ <strong>{formatTime(best)}</strong></div>}
181        </>}
182      </section><span className="menu-footnote">A real-time 3D adventure · Keyboard & touch
+ controls</span></div>}
183      <div className="sr-only" aria-live="polite">{view.checkpoint ? 'Checkpoint activated.' : ''}{won ?
+ `Level completed. ${view.coins} coins collected.` : ''}</div>
184    </main>;
185  }
186  createRoot(document.getElementById('root')).render(<App />);
187
188
189
190
```

Code 13 / src/MobileControls.jsx

20 original lines | 2988 bytes | complete file

Floating touch joystick, pointer capture and jump button.

```
import React,{useEffect,useMemo,useRef,useState} from 'react';
import {createTouchController,stickVector} from './controls';
import {unlockAudio} from './audio';
export default function MobileControls({input}){
 const controller=useMemo(()=>createTouchController(input),[input]);
 const zone=useRef(),anchor=useRef(null),thumb=useRef();
 const
paintThumb=(x=0,y=0)=>{if(thumb.current)thumb.current.style.transform=`translate3d(${x}px,${y}px,0)`;};
 const [origin,setOrigin]=useState(null),[jumping,setJumping]=useState(false);
 useEffect(()=>{const reset=()=>{controller.reset();anchor.current=null;paintThumb();setOrigin(null);setJu
mping(false);};window.addEventListener('blur',reset);window.addEventListener('orientationchange',reset);re
turn()=>{controller.reset();window.removeEventListener('blur',reset);window.removeEventListener('orientati
onchange',reset);};},[controller]);
 const move=event=>{if(!anchor.current)return;const v=stickVector(event.clientX-anchor.current.x,event.cli
entY-anchor.current.y,48);if(controller.moveStick(event.pointerId,v.x,v.z))paintThumb(v.thumbX,v.thumbY);}
;
 const release=event=>{controller.endStick(event.pointerId);if(!input.axisX&&!input.axisZ){anchor.current=
null;paintThumb();setOrigin(null);}};
 const jumpEnd=event=>{controller.endJump(event.pointerId);setJumping(input.jump);};
 return <div className="mobile-controls" aria-label="Touch game controls">
  <div ref={zone} className="move-zone" role="group" aria-label="Floating movement joystick"
onContextMenu={e=>e.preventDefault()}
onPointerDown={event=>{event.preventDefault();if(!controller.beginStick(event.pointerId))return;const rect
=zone.current.getBoundingClientRect();anchor.current={x:event.clientX,y:event.clientY};setOrigin({x:Math.m
ax(67,Math.min(rect.width-67,event.clientX-rect.left)),y:Math.max(67,Math.min(rect.height-67,event.clientY
-rect.top))});paintThumb();unlockAudio();event.currentTarget.setPointerCapture(event.pointerId);}}
onPointerMove={move} onPointerUp={release} onPointerCancel={release} onLostPointerCapture={release}>
   <div className={`joystick ${origin?'steering floating':'resting'}`}
style={origin?{left:origin.x,top:origin.y}:undefined}><span className="stick-orbit"/><span
className="stick-cross x"/><span className="stick-cross y"/><span ref={thumb}
className="stick-thumb"/><span className="stick-arrow">^</span></div><span className="control-caption
stick-caption">DRAG ANYWHERE HERE</span>
  </div>
  <div className="jump-group"><button className={`jump-pad ${jumping?'jumping':''}`} aria-label="Jump"
aria-pressed={jumping} onContextMenu={e=>e.preventDefault()} onPointerDown={event=>{event.preventDefault()
;if(!controller.beginJump(event.pointerId))return;unlockAudio();event.currentTarget.setPointerCapture(even
t.pointerId);setJumping(true);}} onPointerUp={jumpEnd} onPointerCancel={jumpEnd}
onLostPointerCapture={jumpEnd}><span>↑</span><b>JUMP</b></button><span className="control-caption">HOLD
FOR LONGER LEAPS</span></div>
 </div>;
}

```

Code 14 / src/preferences.js

22 original lines | 1313 bytes | complete file

Settings storage, device profile, vibration guards and frame-rate policy.

```
export const SETTINGS_KEY='adventure-runner-settings-v1';
export function loadSettings(storage){
 try{const s=JSON.parse(storage.getItem(SETTINGS_KEY));return
{muted:s?.muted===true,haptics:s?.haptics!==false};}catch{return {muted:false,haptics:true};}
}
export function
storeSettings(storage,settings){try{storage.setItem(SETTINGS_KEY,JSON.stringify(settings));}catch{}}
export function deviceProfile(nav={},touch=false){
 const low=(Number.isFinite(nav.deviceMemory)&&nav.deviceMemory<=4)||(Number.isFinite(nav.hardwareConcurre
ncy)&&nav.hardwareConcurrency<=4)||nav.connection?.saveData===true;
 return {low,touch,quality:low||touch?'low':'high',pixelRatio:low ? .85 : touch ? 1.25 : 1.5};
}
export function feedback(event,nav=globalThis.navigator,enabled=true){
 if(!enabled||typeof nav?.vibrate!=='function')return false;
 const pattern={jump:12,coin:7,hurt:[30,35,45],checkpoint:[12,25,18]};
 if(!pattern[event])return false;
 try{return nav.vibrate(pattern[event]);}catch{return false;}
}
export function performanceStep(state,delta,playing=true){
 if(!playing||delta<=0||delta>.5)return state;
 const samples=state.samples+1,elapsed=state.elapsed+delta;
 if(elapsed<2)return {...state,samples,elapsed};
 const fps=samples/elapsed;return {samples:0,elapsed:0,fps,downgrade:fps<50};
}

```

Code 15 / src/Scene.jsx

326 original lines | 34288 bytes | complete file

Scene composition, geometry, character/camera, platform motion, portal/water shaders and frame simulation.

```
import React, { createContext, useContext, useLayoutEffect, useMemo, useRef, memo } from 'react';
import { useFrame } from '@react-three/fiber';
import * as THREE from 'three';
import {mergeGeometries} from 'three/addons/utils/BufferGeometryUtils.js';
import { getWorld, platformAt, tick, landingPreview } from './game';
import {ParticleEffects,SunGlow,FrameMonitor} from './Effects';
import { getTheme, THEMES } from './worlds';
import StaticIsland from './StaticIsland';


const LowDetail=createContext(false);
const WorldTheme = createContext(THEMES.garden);
function paint(color,t) {
 const map={'#78bd39':t.ground,'#7abd43':t.ground,'#79bc45':t.ground,'#e8c785':t.path,'#8dcc44':t.ground,'
#fff9df':t.name==='snow'?'#ffffff':t.accent,'#ffcd45':t.accent,'#6aba37':t.foliage,'#489c3d':t.foliage,'#a
e742f':t.name==='snow'?'#87a9ba':'#996e52','#bd873f':t.name==='snow'?'#abcad4':'#b98a60'};
 return map[color]||color;
}
const rand = seed => { const n = Math.sin(seed * 127.1 + 311.7) * 43758.5453; return n - Math.floor(n); };
function Box({ at = [0, 0, 0], size = [1, 1, 1], color = '#af8c61', ...props }) {
  const theme = useContext(WorldTheme), ember = theme.name === 'ember' || theme.name === 'volcano';
  color = paint(color, theme);
  return <mesh position={at} scale={size} castShadow receiveShadow {...props}><boxGeometry
/><meshStandardMaterial color={color} roughness={.83} /></mesh>;
}
function Cylinder({ at, args, color, ...props }) {
  return <mesh position={at} castShadow receiveShadow {...props}><cylinderGeometry args={args}
/><meshStandardMaterial color={color} roughness={.7} /></mesh>;
}
function InstanceSet({ geometry, color, transforms, roughness = .8 }) {
  const ref = useRef();
  useLayoutEffect(() => {
    const dummy = new THREE.Object3D();
    transforms.forEach((t, i) => {
      dummy.position.set(...t.p); dummy.rotation.set(...(t.r || [0, 0, 0]));
      dummy.scale.set(...(t.s || [1, 1, 1])); dummy.updateMatrix(); ref.current.setMatrixAt(i,
dummy.matrix);
    });
    ref.current.instanceMatrix.needsUpdate = true;
    ref.current.computeBoundingSphere();
  }, [transforms]);
  const theme = useContext(WorldTheme), ember = theme.name === 'ember' || theme.name === 'volcano';
  color = paint(color, theme);
  return <instancedMesh ref={ref} args={[geometry, undefined, transforms.length]} castShadow
receiveShadow><meshStandardMaterial color={color} roughness={roughness} /></instancedMesh>;
}
function Rock({ w, d, seed = 1 }) {
  const theme = useContext(WorldTheme), ember = theme.name === 'ember' || theme.name === 'volcano';
  const geo = useMemo(() => {
    const vertices = [], colors = [], palette = theme.rock.map(c => new THREE.Color(c));
    const rings = [0, 1, 2].map((ring) => Array.from({ length: 8 }, (_, i) => {
      const a = i / 8 * Math.PI * 2;
      const scale = [1, .8, .18][ring];
      return [Math.cos(a) * w * .6 * scale, [-.35, -2.1, -5.2][ring] + (rand(seed + i * 2 + ring) - .5) *
.5, Math.sin(a) * d * .6 * scale];
    }));
    function triangle(a, b, c, shade) { vertices.push(...a, ...b, ...c); for (let j = 0; j < 3; j++)
colors.push(shade.r, shade.g, shade.b); }
    for (let r = 0; r < 2; r++) for (let i = 0; i < 8; i++) {
      const j = (i + 1) % 8, col = palette[Math.floor(rand(seed + i * 3 + r) * palette.length)];
      triangle(rings[r][i], rings[r + 1][i], rings[r][j], col);
      triangle(rings[r][j], rings[r + 1][i], rings[r + 1][j], palette[(i + r) % palette.length]);
    }
    const geometry = new THREE.BufferGeometry();
    geometry.setAttribute('position', new THREE.Float32BufferAttribute(vertices, 3));
    geometry.setAttribute('color', new THREE.Float32BufferAttribute(colors, 3));
    geometry.computeVertexNormals(); return geometry;
  }, [w, d, seed, theme]);
  return <mesh geometry={geo} castShadow receiveShadow><meshStandardMaterial vertexColors flatShading
side={THREE.DoubleSide} /></mesh>;
}
```

```
function Pine({ at, scale = 1 }) {
  const theme=useContext(WorldTheme), kind=theme.plant,low=useContext(LowDetail);
  if(kind==='cactus')return <group position={at} scale={scale}><Cylinder at={[0,1.2,0]}
args={[.29,.34,2.4,7]} color={theme.foliage}/>{[-1,1].map((v,i)=><group key={v}><Cylinder
at={[v*.46,1+i*.5,0]} args={[.15,.15,.8,6]} color={theme.foliage} rotation={[0,0,Math.PI/2]}/><Cylinder
at={[v*.78,1.3+i*.5,0]} args={[.16,.18,.8,6]} color={theme.foliage}/></group>)}</group>;
  if(kind==='mushroom')return <group position={at} scale={scale}><Cylinder at={[0,.8,0]}
args={[.24,.36,1.6,7]} color="#e8dbd4"/><mesh position={[0,1.5,0]} scale={[1.2,.6,1.2]}
castShadow><sphereGeometry args={[1,10,7,0,Math.PI*2,0,Math.PI/2]}/><meshStandardMaterial
color={theme.foliage} emissive={theme.foliage} emissiveIntensity={.13}/></mesh>{[-.55,0,.55].map(x=><mesh
key={x} position={[x,1.94-Math.abs(x)*.22,.27]} scale={[.15,.045,.15]}><sphereGeometry
args={[1,6,4]}/><meshStandardMaterial color="#fff3de"/></mesh>)}</group>;
  if(kind==='palm')return <group position={at} scale={scale}><Cylinder at={[0,1.3,0]}
args={[.16,.28,2.6,6]} color="#987146"/>{Array.from({length:6},(_,i)=><mesh key={i}
position={[Math.cos(i*Math.PI/3)*.7,2.65,Math.sin(i*Math.PI/3)*.7]} rotation={[0,-i*Math.PI/3,.15]}
scale={[1.2,.12,.35]} castShadow><octahedronGeometry/><meshStandardMaterial
color={theme.foliage}/></mesh>)}</group>;
  if(kind==='broadleaf')return <group position={at} scale={scale}><Cylinder at={[0,1,0]}
args={[.18,.28,2,6]} color="#805b43"/>{[-1,0,1].map((v,i)=><mesh key={i}
position={[v*.5,2.1+(i===1?.6:0),v*.17]} scale={[.85,.9,.85]} castShadow><icosahedronGeometry
args={[1,1]}/><meshStandardMaterial color={i%2?theme.foliage:'#c45f3b'}/></mesh>)}</group>;
  if(['crystal','basalt','beacon'].includes(kind))return <group position={at}
scale={scale}>{[-1,0,1].map((v,i)=><mesh key={i} position={[v*.35,.75+(i===1?.55:0),v*.2]}
scale={[.4,.95+(i===1?.65:0),.4]} rotation={[0,i*.8,v*-.25]}
castShadow>{kind==='beacon'?<boxGeometry/>:<octahedronGeometry/>}<meshStandardMaterial
color={kind==='basalt'?'#3f354d':theme.foliage} emissive={theme.accent}
emissiveIntensity={kind==='basalt'?.12:.35} roughness={.35}/></mesh>)}{kind==='beacon'&&<mesh
position={[0,2,0]}><sphereGeometry args={[.18,8,6]}/><meshBasicMaterial
color={theme.accent}/></mesh>}</group>;
  return <group position={at} scale={scale}><Cylinder at={[0,.7,0]} args={[.17,.27,1.5,5]} color="#88552d"
/>{(low?[{y:1.8,r:1.1,h:2.4},{y:2.9,r:.65,h:1.6}]:[{y:1.6,r:1.15,h:1.8},{y:2.4,r:.88,h:1.65},{y:3.1,r:.6,h
:1.5}]).map((v,i)=><group key={i}><mesh position={[0,v.y,0]} rotation={[0,i*.4,0]}
castShadow><coneGeometry args={[v.r,v.h,5]}/><meshStandardMaterial
color={kind==='snowpine'?'#537f94':theme.foliage} flatShading/></mesh>{kind==='snowpine'&&<mesh
position={[0,v.y+.2,0]} rotation={[0,i*.4,0]} castShadow><coneGeometry
args={[v.r*.92,v.h*.83,5]}/><meshStandardMaterial color="#f1fbff" flatShading/></mesh>}</group>)}</group>;
}
function Flowers({ p, seed }) {
  const data = useMemo(() => {
    const petals = [], centers = [], grass = [], leaves = [];
    for (let i = 0; i < 8; i++) {
      const x = (i % 2 ? 1 : -1) * (p.w * .27 + rand(seed + i) * p.w * .14);
      const z = (rand(seed + i * 5) - .5) * p.d * .83;
      centers.push({ p: [x, .15, z], s: [.08, .08, .08] });
      for (let j = 0; j < 5; j++) petals.push({ p: [x + Math.cos(j * 1.256) * .105, .13, z + Math.sin(j *
1.256) * .105], s: [.11, .035, .07], r: [0, -j * 1.256, 0] });
      for (let j = 0; j < 3; j++) grass.push({ p: [x + .4, .16, z + j * .1], s: [.13, .45 + j * .08, .1],
r: [.2, j * 1.2, .4] });
    }
    for (let i = 0; i < 3; i++) for (let j = 0; j < 5; j++) leaves.push({ p: [p.w * (.2 - i * .27), -.2 -
j * .38, p.d * .48], s: [.25, .37, .12], r: [0, 0, (j % 2 ? 1 : -1) * .55] });
    return { petals, centers, grass, leaves };
  }, [p, seed]);
  const sphere = useMemo(() => new THREE.IcosahedronGeometry(1, 0), []);
  const blade = useMemo(() => new THREE.ConeGeometry(1, 1, 3), []);
  return <><InstanceSet geometry={sphere} color="#fff9df" transforms={data.petals} /><InstanceSet
geometry={sphere} color="#ffcd45" transforms={data.centers} /><InstanceSet geometry={blade}
color="#6aba37" transforms={data.grass} /><InstanceSet geometry={sphere} color="#489c3d"
transforms={data.leaves} /></>;
}
function Fence({ x, z, length, rotate = false }) {
  return <group position={[x, 0, z]} rotation={[0, rotate ? Math.PI / 2 : 0, 0]}>
    {[-length / 2, 0, length / 2].map((v, i) => <Box key={i} at={[0, .55, v]} size={[.22, 1.1, .24]}
color="#ae742f" />)}
    {[.4, .8].map(y => <Box key={y} at={[0, y, 0]} size={[.13, .14, length]} color="#bd873f" />)}
  </group>;
}
function Island({ p, index, game }) {
  const low=useContext(LowDetail);
  const ref = useRef();
  useFrame(() => { if (p.moving) ref.current.position.x = platformAt(p, game.time).x; });
  return <group ref={ref} position={[p.x, p.y, p.z]}><StaticIsland enabled={low}>
    <Rock w={p.w} d={p.d} seed={index + 10} />
    <Box at={[0, -.21, 0]} size={[p.w, .42, p.d]} color={p.moving ? '#caab6c' : '#78bd39'} />
    <Box at={[0, .014, 0]} size={[Math.min(2.8, p.w * .45), .035, p.d - .12]} color={p.moving ? '#ae7c3e'
: '#e8c785'} />
    {p.moving ? <>{[-1.6, -.8, 0, .8, 1.6].map(x => <Box key={x} at={[x, .05, 0]} size={[.07, .07, p.d]}
color="#8b6139" />)}<Box at={[0, -.4, 0]} size={[p.w + .2, .2, 1]} color="#889aa3" /></> : <>
      <Flowers p={p} seed={index * 7} />
```

```
      {[0, 2, 3].includes(index) && <group position={[0, .03, -p.d / 2 + .8]}>
        {[-.6, -.2, .2, .6].map(z => <Box key={z} at={[0, 0, z]} size={[3, .06, .35]} color="#c3924f" />)}
        {[-1.28, 1.28].map(x => <Box key={x} at={[x, .045, 0]} size={[.12, .04, 1.7]} color="#9b6b3d" />)}
      </group>}
      {[-1, 1].map((side) => <group key={side}><Fence x={side * (p.w / 2 - .35)} z={p.d * .07} length={p.d * .6} /><Box at={[side * (p.w / 2 - .15), -.1, -p.d * .35]} size={[.5, .4, .9]} color="#8dcc44" /></group>)}
      {index !== 1 && <Pine at={[(index % 2 ? -1 : 1) * (p.w / 2 - 1.25), 0, -p.d / 2 + 1.3]} scale={.7 + (index % 3) * .15} />}
      {[0, 1].map(i => <mesh key={i} position={[(i ? -1 : 1) * (p.w * .34), .15, p.d * .3]} scale={[.45, .35, .4]} rotation={[0, index, .1]} castShadow><dodecahedronGeometry /><meshStandardMaterial color="#99a1a0" flatShading /></mesh>)}
    </>}
  </StaticIsland></group>;
}
const starShape = new THREE.Shape();
for (let i = 0; i < 10; i++) { const a = i * Math.PI / 5 + Math.PI / 2, r = i % 2 ? .12 : .25; i ? starShape.lineTo(Math.cos(a) * r, Math.sin(a) * r) : starShape.moveTo(Math.cos(a) * r, Math.sin(a) * r); }
starShape.closePath();
const starGeometry = new THREE.ShapeGeometry(starShape);
const coinGeometry=(()=>{
 const pieces=[];
 const add=(geometry,color)=>{geometry=geometry.toNonIndexed();const c=new THREE.Color(color),values=[];for(let i=0;i<geometry.attributes.position.count;i++)values.push(c.r,c.g,c.b);geometry.setAttribute('color',new THREE.Float32BufferAttribute(values,3));pieces.push(geometry);};
 const body=new THREE.CylinderGeometry(.36,.36,.1,12);body.rotateX(Math.PI/2);add(body,'#ffc83a');
 for(const side of [-1,1]){const ring=new THREE.TorusGeometry(.3,.033,4,12);ring.translate(0,0,side*.06);add(ring,'#ffeb91');const star=starGeometry.clone();if(side<0)star.rotateY(Math.PI);star.translate(0,0,side*.067);add(star,'#fff1aa');}
 const result=mergeGeometries(pieces,false);pieces.forEach(p=>p.dispose());return result;
})();
function Coin({at,index,game}){
 const ref=useRef();useFrame(()=>{ref.current.visible=!game.coins.has(index);ref.current.rotation.y=game.time*2.3+index*.5;ref.current.position.y=at[1]+Math.sin(game.time*3+index)*.1;});
 return <mesh ref={ref} position={at} geometry={coinGeometry} dispose={null}><meshStandardMaterial vertexColors metalness={.12} roughness={.32}/></mesh>;
}
function Limb({ at, color, leg = false, limbRef }) {
  return <group position={at} ref={limbRef}><mesh position={[0, -.2, 0]} castShadow><capsuleGeometry args={[leg ? .115 : .095, .25, 3, 6]} /><meshStandardMaterial color={color} /></mesh>{leg ? <><Cylinder at={[0, -.39, 0]} args={[.115, .115, .13, 6]} color="#fff9e8" /><Box at={[0, -.49, .06]} size={[.26, .2, .41]} color="#224faa" /><Box at={[0, -.58, .08]} size={[.27, .07, .43]} color="#f8f1db" /></> : <mesh position={[0, -.4, 0]} castShadow><sphereGeometry args={[.13, 8, 6]} /><meshStandardMaterial color="#ffbe83" /></mesh>}</group>;
}
function Character({ game }) {
  const shape=useMemo(()=>new THREE.Vector3(1,1,1),[]);
  const ref = useRef(), leftArm = useRef(), rightArm = useRef(), leftLeg = useRef(), rightLeg = useRef();
  useFrame((_, dt) => {
    ref.current.position.set(game.x, game.y, game.z);
    const sinceLanding=game.time-game.landTime;
    const squash=game.ground&&sinceLanding<.18?Math.sin(sinceLanding/.18*Math.PI)*.18:0;
    const stretch=!game.ground?THREE.MathUtils.clamp(game.vy*.014,-.05,.14):0;
    ref.current.scale.lerp(shape.set(1+squash-stretch*.45,1-squash+stretch,1+squash-stretch*.45),1-Math.exp(-dt*20));
    ref.current.visible = game.invincible <= 0 || Math.floor(game.invincible * 9) % 2 === 0;
    let difference = THREE.MathUtils.euclideanModulo(game.facing - ref.current.rotation.y + Math.PI, Math.PI * 2) - Math.PI;
    ref.current.rotation.y += difference * Math.min(1, dt * 17);
    const swing = game.moving && game.ground ? Math.sin(game.steps * 17) * .75 : 0;
    leftLeg.current.rotation.x = game.ground ? swing : -.5;
    rightLeg.current.rotation.x = game.ground ? -swing : .45;
    leftArm.current.rotation.x = -swing * .8 - (game.ground ? 0 : .65);
    rightArm.current.rotation.x = swing * .8 - (game.ground ? 0 : .65);
  });
  return <group ref={ref}>
    <Limb at={[-.17, .59, 0]} color="#275bad" leg limbRef={leftLeg} /><Limb at={[.17, .59, 0]} color="#275bad" leg limbRef={rightLeg} />
    <mesh position={[0, .89, 0]} scale={[1, 1, .72]} castShadow><capsuleGeometry args={[.28, .26, 4, 8]} /><meshStandardMaterial color="#f04735" /></mesh>
    <Limb at={[-.35, 1.05, 0]} color="#f86a43" limbRef={leftArm} /><Limb at={[.35, 1.05, 0]} color="#f86a43" limbRef={rightArm} />
    <mesh position={[0, 1.36, .015]} scale={[1, 1.03, 1]} castShadow><sphereGeometry args={[.28, 12, 8]} /><meshStandardMaterial color="#ffc28b" /></mesh>
    <mesh position={[0, 1.44, -.045]} castShadow><sphereGeometry args={[.284, 12, 8]} /><meshStandardMaterial color="#533020" /></mesh>
    <mesh position={[0, 1.56, 0]} castShadow><sphereGeometry args={[.31, 12, 8, 0, Math.PI * 2, 0, Math.PI / 2]} /><meshStandardMaterial color="#ed342c" /></mesh>
```

```
    <mesh position={[0, 1.55, .17]} scale={[1, .18, 1]} castShadow><sphereGeometry args={[.31, 10, 6]} />
<meshStandardMaterial color="#e7362b" /></mesh>
    <mesh position={[0, 1.88, 0]}><sphereGeometry args={[.045, 8, 5]} /><meshStandardMaterial color="#c52525" /></mesh>
    {[-1, 1].map(side => <group key={side}><mesh position={[side * .275, 1.36, .005]}><sphereGeometry args={[.065, 7, 5]} /><meshStandardMaterial color="#f7ad74" /></mesh><mesh position={[side * .095, 1.39, .265]}><sphereGeometry args={[.035, 6, 5]} /><meshStandardMaterial color="#392b25" /></mesh><Box at={[side * .18, 1.02, -.19]} size={[.075, .5, .06]} color="#2679c9" /></group>)}
    <Box at={[0, .91, -.29]} size={[.46, .48, .21]} color="#1387dd" /><Box at={[0, .78, -.415]} size={[.35, .19, .1]} color="#1266bc" /><Box at={[0, 1.05, -.411]} size={[.13, .06, .025]} color="#f3b85e" />
    <mesh rotation={[-Math.PI / 2, 0, 0]} position={[0, .025, 0]}><circleGeometry args={[.4, 16]} /><meshBasicMaterial color="#213c4d" transparent opacity={.16} depthWrite={false} /></mesh>
  </group>;
}
function Rotator({ r, game }) {
  const bar = useRef();
  useFrame(() => { bar.current.rotation.y = game.time * r.speed + r.phase; });
  return <group position={[r.x, r.y, r.z]}>
    <Cylinder at={[0, .12, 0]} args={[.65, .8, .24, 8]} color="#78848a" />
    <group ref={bar} position={[0, .65, 0]}><mesh rotation={[0, 0, Math.PI / 2]} castShadow><cylinderGeometry args={[.2, .2, r.length, 12]} /><meshStandardMaterial color="#f23c34" metalness={.3} roughness={.32} /></mesh></group>
    <Box at={[0, .65, 0]} size={[.85, .95, .85]} color="#78858d" />
    <Cylinder at={[0, 1.15, 0]} args={[.36, .36, .1, 8]} color="#a5b1b6" />
    {[-1, 1].map(side => <mesh key={side} position={[side * .28, .7, .432]}><sphereGeometry args={[.055, 6, 4]} /><meshStandardMaterial color="#d5e1e2" metalness={.6} /></mesh>)}
  </group>;
}
function Checkpoint({ game }) {
 const low=useContext(LowDetail);
  const level = getWorld(game.world);
  const flag = useRef(), glow = useRef();
  useFrame(() => { flag.current.rotation.y = Math.sin(game.time * 3) * .1; glow.current.material.opacity = game.checkpoint ? .5 + Math.sin(game.time * 3) * .1 : .15; });
  return <group position={level.flag}>
    <Cylinder at={[0, .08, 0]} args={[1.1, 1.2, .16, 12]} color="#a8afb0" /><Cylinder at={[0, .24, 0]} args={[.83, .93, .18, 12]} color="#d9d6ba" />
    <Cylinder at={[0, 1.6, 0]} args={[.06, .07, 2.8, 8]} color="#de9d35" /><mesh position={[0, 3.07, 0]}><sphereGeometry args={[.13, 8, 6]} /><meshStandardMaterial color="#ffe16f" metalness={.5} roughness={.3} /></mesh>
    <group ref={flag} position={[.64, 2.6, 0]}><Box size={[1.2, .76, .045]} color="#f54a37" />{Array.from({ length: 6 }, (_, i) => <Box key={i} at={[(i % 3) * .23 - .3, Math.floor(i / 3) * .23 - .11, .03]} size={[.22, .22, .014]} color={(i + Math.floor(i / 3)) % 2 ? '#fff4cf' : '#f54a37'} />)}</group>
    <mesh ref={glow} rotation={[-Math.PI / 2, 0, 0]} position={[0, .35, 0]}><ringGeometry args={[.68, .9, 40]} /><meshBasicMaterial color="#ffe15b" transparent opacity={.3} depthWrite={false} /></mesh>
    {!low&&<pointLight position={[0, 1, 0]} color="#ffcf41" intensity={1.5} distance={5} />}
  </group>;
}
const portalVertex = `varying vec2 vUv; void main(){vUv=uv;gl_Position=projectionMatrix*modelViewMatrix*vec4(position,1.);}`;
const portalFragment = `uniform float time;varying vec2 vUv;void main(){vec2 p=(vUv-.5)*2.;float r=length(p);if(r>1.)discard;float a=atan(p.y,p.x);float spiral=pow(.5+.5*sin(a*3.+r*17.-time*3.),7.);float edge=pow(r,7.);vec3 c=mix(vec3(.025,.29,.87),vec3(.15,.87,1.),.4+r*.4);c+=spiral*.75+edge*.8;float center=exp(-r*9.);gl_FragColor=vec4(c+center, .93);}`;
function Portal({ game }) {
 const low=useContext(LowDetail);
  const level = getWorld(game.world);
  const mat = useRef(), crystal = useRef(), particles = useRef();
  const uniforms = useMemo(() => ({ time: { value: 0 } }), []);
  const sparkleGeometry=useMemo(()=>new THREE.IcosahedronGeometry(.035,0),[]);
  const stars = useMemo(() => Array.from({ length: 24 }, (_, i) => { const a = i * 2.399; return [Math.cos(a) * (1.4 + rand(i) * 1.3), 1.5 + rand(i + 15) * 3, Math.sin(a) * .7]; }), []);
  useFrame((state, dt) => { mat.current.uniforms.time.value += dt; crystal.current.rotation.y += dt; particles.current.rotation.y += dt * .15; });
  return <group position={level.portal}>
    <Box at={[0, .12, 1.1]} size={[4.1, .24, 2.2]} color="#c3c0a6" />
    <Box at={[0, .27, .65]} size={[3.6, .3, 1.6]} color="#dfd4b2" />
    {[-1, 1].map(side => <group key={side}>{[0, 1, 2, 3].map(i => <Box key={i} at={[side * 1.5, .5 + i * .6, 0]} size={[.65, .57, .9]} color={i % 2 ? '#9594ae' : '#aaabc2'} rotation={[0, 0, side * -.025]} />)}</group>)}
    {[0, 1, 2, 3, 4].map(i => { const a = (i / 4) * Math.PI; return <Box key={i} at={[Math.cos(a) * 1.5, 2.3 + Math.sin(a) * 1.45, 0]} size={[.77, .68, .95]} color={i % 2 ? '#b7b6c8' : '#9699b5'} rotation={[0, 0, a - Math.PI / 2]} />; })}
    <mesh position={[0, 1.95, .12]} scale={[1.25, 1.68, 1]}><circleGeometry args={[1, 64]} /><shaderMaterial ref={mat} uniforms={uniforms} vertexShader={portalVertex} fragmentShader={portalFragment} transparent side={THREE.DoubleSide} /></mesh>
    <mesh position={[0, 1.95, .1]} scale={[1.23, 1.65, 1]}><torusGeometry args={[1, .055, 8, 48]}
```

```
/><meshBasicMaterial color="#9cffff" /></mesh>
    <mesh ref={crystal} position={[0, 3.95, .53]} scale={[.26, .4, .18]}><octahedronGeometry
/><meshStandardMaterial color="#6ff5ff" emissive="#1cb4ee" emissiveIntensity={1.5} metalness={.2}
roughness={.15} /></mesh>
    <group ref={particles}><InstanceSet geometry={sparkleGeometry} color="#b7ffff"
transforms={stars.map(p=>({p}))}/></group>
    <mesh position={[0,1.95,-.02]} scale={[1.7,2.2,1]}><circleGeometry args={[1,48]}/><meshBasicMaterial
color="#40d7ff" transparent opacity={.11} blending={THREE.AdditiveBlending} depthWrite={false}
side={THREE.DoubleSide}/></mesh>{!low&&<pointLight position={[0, 2, 1]} color="#55e8ff" intensity={6}
distance={7} />}
    {[-1, 1].map(side => <group key={side} position={[side * 2.7, 0, .8]}><Cylinder at={[0, .55, 0]}
args={[.3, .43, 1.1, 6]} color="#aaa8a0" /><mesh position={[0, 1.45, 0]}><sphereGeometry args={[.2, 7, 6]}
 /><meshBasicMaterial color="#ffcd43" /></mesh><mesh position={[0, 1.64, 0]}><coneGeometry args={[.18,
.55, 6]} /><meshBasicMaterial color="#ffeb86" /></mesh></group>)}
  </group>;
}
const waterFrag = `uniform float time; uniform vec3 baseColor; varying vec2 vUv;void main(){float
wave=.5+.5*sin(vUv.x*50.+sin(vUv.y*9.-time*4.)*.9);vec3 c=mix(baseColor,vec3(1.),pow(wave,5.));float
alpha=smoothstep(0.,.12,vUv.x)*smoothstep(1.,.88,vUv.x)*(.45+vUv.y*.4);gl_FragColor=vec4(c,alpha);}`;
function Waterfall({at,width=2,height=13}) {
 const theme=useContext(WorldTheme),mat=useRef();
 const uniforms=useMemo(()=>({time:{value:0},baseColor:{value:new
THREE.Color(theme.water).convertLinearToSRGB()}}),[theme]);
 useFrame((_,dt)=>{mat.current.uniforms.time.value+=dt;});
 return <mesh position={at}><planeGeometry args={[width,height]}/><shaderMaterial ref={mat}
uniforms={uniforms} vertexShader={portalVertex} fragmentShader={waterFrag} transparent
side={THREE.DoubleSide} depthWrite={false}/></mesh>;
}
function Castle() {
 const low=useContext(LowDetail);
  return <group position={[-24, 5, -51]} scale={1.4}><StaticIsland enabled={low}>
    <Rock w={11} d={10} seed={88} /><Box at={[0, -.15, 0]} size={[11, .4, 10]} color="#79bc45" />
    <Box at={[0, 2.1, 0]} size={[5, 4.2, 3.5]} color="#e4d8ba" /><Box at={[0, 4.1, 0]} size={[5.5, .45,
3.9]} color="#c8c3b1" />
    {[[-3, 0, 2], [3, 0, 2], [-2.5, 0, -2], [2.5, 0, -2]].map((at, i) => <group key={i}
position={at}><Cylinder at={[0, 2.6 + i * .2, 0]} args={[.8, .85, 5.2 + i * .4, 8]} color="#ede1c5"
/><mesh position={[0, 5.8 + i * .4, 0]}><coneGeometry args={[1.16, 2, 8]} /><meshStandardMaterial
color="#ed795c" /></mesh><Box at={[0, 4, .79]} size={[.22, .8, .06]} color="#657c8b" /><Cylinder at={[0,
7.1 + i * .4, 0]} args={[.035, .035, 1, 5]} color="#dbc285" /><Box at={[.24, 7.3 + i * .4, 0]} size={[.5,
.3, .035]} color="#ffb85e" /></group>)}
    <Box at={[0, 1.1, 1.78]} size={[1.3, 2.2, .05]} color="#566573" />
    <Pine at={[4.2, 0, 3]} scale={.7} /><Pine at={[-4.1, 0, 1.2]} scale={.75} />
    <Waterfall at={[1.6, -6.2, 5.05]} width={2.1} height={12} /></StaticIsland>
  </group>;
}
function Landmark() {
 const theme=useContext(WorldTheme);
 const wind=useRef();useFrame((_,dt)=>{if(wind.current)wind.current.rotation.z+=dt*.6;});
 if(theme.name==='garden')return <Castle/>;
 return <group position={[-25,5,-48]}><Rock w={16} d={14} seed={71}/><Box at={[0,-.2,0]} size={[16,.4,14]}
 color={theme.ground}/>
 {theme.name==='desert'?<><mesh position={[0,3,0]} rotation={[0,Math.PI/4,0]}><coneGeometry
args={[6,6,4]}/><meshStandardMaterial color="#dfb772" flatShading/></mesh><mesh position={[5,1.6,2]}
rotation={[0,Math.PI/4,0]}><coneGeometry args={[3,3.2,4]}/><meshStandardMaterial
color="#cda166"/></mesh></>:theme.name==='ocean'?<><Cylinder at={[0,3,0]} args={[.9,1.3,6,12]}
color="#fff5df"/><Cylinder at={[0,5,0]} args={[.93,.93,1,12]} color="#e8795e"/><mesh
position={[0,6.6,0]}><sphereGeometry args={[.5,10,7]}/><meshBasicMaterial color="#ffed9b"/></mesh><Pine
at={[-4,0,2]} scale={1.2}/></>:theme.name==='mushroom'?<Pine at={[0,0,0]}
scale={4}/>:theme.name==='autumn'?<><Box at={[0,2,0]} size={[3,4,3]} color="#c5a57a"/><mesh
position={[0,4.8,0]}><coneGeometry args={[2.5,2,4]}/><meshStandardMaterial color="#a55f4f"/></mesh><group
ref={wind} position={[0,3,1.7]}>{[0,Math.PI/2].map(r=><Box key={r} size={[.3,6,.15]} rotation={[0,0,r]}
color="#f4dfad"/>)}</group></>:['snow','aurora'].includes(theme.name)?<><mesh
position={[0,4,0]}><coneGeometry args={[5,8,5]}/><meshStandardMaterial color="#8bacbf"
flatShading/></mesh><mesh position={[0,6.1,0]}><coneGeometry args={[2.8,4.3,5]}/><meshStandardMaterial
color="#f4fcff" flatShading/></mesh><Pine at={[-5,0,3]}
scale={1.5}/></>:theme.name==='jungle'?<>{[0,1,2,3].map(i=><Box key={i} at={[0,i*.8+.4,0]}
size={[7-i*1.6,.8,6-i*1.3]} color="#74896b"/>)}</>:<>{[-4,-2,0,2,4].map((x,i)=><group key={i}
position={[x,0,-1]}><Box at={[0,2+i%3,0]} size={[1.4,4+i%3*2,1.4]} color={theme.rock[0]}/><mesh
position={[0,5+i%3,0]}><octahedronGeometry args={[.7]}/><meshStandardMaterial color={theme.accent}
emissive={theme.accent} emissiveIntensity={1.1}/></mesh></group>)}</>}
 <Waterfall at={[3,-7,7.05]} width={2.5} height={14}/></group>;
}
function Weather({game}) {
 const theme=useContext(WorldTheme),ref=useRef();
 const count=theme.weather==='rain'?150:85;
 const positions=useMemo(()=>new
Float32Array(Array.from({length:count*3},(_,i)=>(rand(i+45)-.5)*(i%3===1?18:36))),[count]);
 useFrame((state,dt)=>{if(!ref.current)return;ref.current.position.set(game.x,game.y+6,game.z-12);const
attr=ref.current.geometry.attributes.position;for(let i=0;i<count;i++){attr.array[i*3+1]-=dt*(theme.weathe
```

```
r==='rain'?11:theme.weather==='embers'?-1.2:1.7);if(attr.array[i*3+1]<-9)attr.array[i*3+1]=9;if(attr.array
[i*3+1]>9)attr.array[i*3+1]=-9;}attr.needsUpdate=true;});
 if(!theme.weather||theme.weather==='none')return null;
 return <points ref={ref}><bufferGeometry><bufferAttribute attach="attributes-position"
args={[positions,3]}/></bufferGeometry><pointsMaterial
color={theme.weather==='snow'?'#ffffff':theme.accent} size={theme.weather==='rain'?.065:.11} transparent
opacity={.75} depthWrite={false} sizeAttenuation/></points>;
}
function Background() {
 const low=useContext(LowDetail);
 const drift=useRef();
 useFrame(({clock})=>{if(drift.current)drift.current.position.x=Math.sin(clock.elapsedTime*.025)*3;});
 const theme=useContext(WorldTheme);
 const sphere=useMemo(()=>new THREE.IcosahedronGeometry(1,2),[]);
 const clouds=useMemo(()=>Array.from({length:low?24:55},(_,i)=>({p:[(rand(i+6)-.5)*135,-9+rand(i+9)*5,25-r
and(i+21)*180],s:[3+rand(i+3)*6,1+rand(i)*2,2+rand(i+4)*3]})),[low]);
 const high=useMemo(()=>Array.from({length:low?10:24},(_,i)=>({p:[(rand(i+29)-.5)*200,19+rand(i+81)*8,-35-
rand(i+2)*150],s:[5+rand(i)*8,1.4+rand(i+3)*3,2+rand(i+4)*4]})),[low]);
 const uniforms=useMemo(()=>({top:{value:new
THREE.Color(theme.skyTop).convertLinearToSRGB()},bottom:{value:new
THREE.Color(theme.skyBottom).convertLinearToSRGB()}}),[theme]);
 return <><mesh><sphereGeometry args={[190,24,16]}/><shaderMaterial side={THREE.BackSide}
uniforms={uniforms} vertexShader={`varying vec3 v;void
main(){v=position;gl_Position=projectionMatrix*modelViewMatrix*vec4(position,1.);}`}
fragmentShader={`uniform vec3 top;uniform vec3 bottom;varying vec3 v;void main(){float
h=clamp(normalize(v).y*.8+.1,0.,1.);gl_FragColor=vec4(mix(bottom,top,h),1.);}`}
depthWrite={false}/></mesh>
 <group ref={drift}><InstanceSet geometry={sphere} color={theme.cloud} transforms={clouds}/><InstanceSet
geometry={sphere} color={theme.cloud} transforms={high}/></group><SunGlow
night={['moon','aurora','storm'].includes(theme.name)}/><Landmark/>
 {Array.from({length:low?6:12},(_,i)=>{const
x=(i%2?-1:1)*(19+rand(i+2)*25),z=4-i*11,y=-3+rand(i+4)*10,s=3+rand(i+6)*4;return <group key={i}
position={[x,y,z]}><StaticIsland enabled={low}><Rock w={s} d={s} seed={i+50}/><Box at={[0,-.2,0]}
size={[s,.4,s]} color={theme.ground}/><Pine at={[0,0,-.3]} scale={.5+rand(i)*.4}/>{i%3===0&&<Waterfall
at={[.7,-5.5,s/2+.02]} width={1.1} height={11}/>}</StaticIsland></group>;})}
 {theme.name==='moon'&&<group position={[25,26,-90]}><mesh><sphereGeometry
args={[10,24,16]}/><meshStandardMaterial color="#d4c7e7"/></mesh><mesh
rotation={[1.2,.1,.3]}><torusGeometry args={[14,.4,8,64]}/><meshBasicMaterial
color="#a8c7f2"/></mesh></group>}
 {theme.name==='aurora'&&[0,1,2].map(i=><mesh key={i} position={[i*18-18,23+i*3,-70-i*8]}
rotation={[0,0,i*.1-.2]}><planeGeometry args={[100,8]}/><meshBasicMaterial color={i%2?'#80a4ff':'#59f7bc'}
 transparent opacity={.15} depthWrite={false} side={THREE.DoubleSide}/></mesh>)}
 </>;
}
function Sign({ at, children }) {
  const texture = useMemo(() => {
    const canvas = document.createElement('canvas'); canvas.width = 512; canvas.height = 160;
    const ctx = canvas.getContext('2d'); ctx.fillStyle = '#99602e'; ctx.fillRect(0, 0, 512, 160);
ctx.fillStyle = '#b88142'; ctx.fillRect(8, 8, 496, 144); ctx.fillStyle = '#fff6d7'; ctx.font = 'bold 54px
sans-serif'; ctx.textAlign = 'center'; ctx.textBaseline = 'middle'; ctx.fillText(children, 256, 82);
    const t = new THREE.CanvasTexture(canvas); t.colorSpace = THREE.SRGBColorSpace; return t;
  }, [children]);
  return <group position={at}><Box at={[0, .6, 0]} size={[.15, 1.2, .15]} color="#885b32" /><mesh
position={[0, 1.2, 0]}><boxGeometry args={[1.7, .55, .12]} /><meshStandardMaterial map={texture}
/></mesh></group>;
}
function Celebration({ game }) {
  const level = getWorld(game.world);
  const ref = useRef();
  const transforms = useMemo(() => Array.from({ length: 65 }, (_, i) => ({ p: [(rand(i + 4) - .5) * 10, 2
+ rand(i) * 6, level.portal[2] + (rand(i + 2) - .5) * 6], s: [.07, .15, .035], r: [i, i * 2, i * 3] })),
[]);
  const geometry = useMemo(() => new THREE.BoxGeometry(), []);
  useFrame((_, dt) => { ref.current.visible = game.status === 'won'; if (game.status === 'won')
ref.current.rotation.y += dt * .2; });
  return <group ref={ref}><InstanceSet geometry={geometry} transforms={transforms} color="#ffdd5e"
/></group>;
}
function LandingMarker({game}){
 const ref=useRef();useFrame(()=>{const p=landingPreview(game);ref.current.visible=!!p&&game.status==='pla
ying';if(p)ref.current.position.set(p.x,p.y,p.z);});
 return <group ref={ref}><mesh rotation={[-Math.PI/2,0,0]}><ringGeometry
args={[.33,.44,28]}/><meshBasicMaterial color="#a4ffce" transparent opacity={.9}
depthWrite={false}/></mesh><mesh rotation={[-Math.PI/2,0,0]}><circleGeometry
args={[.3,20]}/><meshBasicMaterial color="#53caa5" transparent opacity={.22}
depthWrite={false}/></mesh></group>;
}
function Scene({ game, input, onStep, quality, cameraMode, mobile, onSlow, onStats }) {
  const level = getWorld(game.world), theme = getTheme(level);
```

```
284    const cameraReady = useRef(false), previousStatus = useRef(game.status), accumulator = useRef(0);
285    const target = useMemo(() => new THREE.Vector3(), []);
286    const cameraHeight=useRef(game.y);
287    useFrame(({ camera }, delta) => {
288      if (game.status === 'playing') {
289        accumulator.current += Math.min(delta, .1);
290        while (accumulator.current >= 1 / 120) { tick(game, input, 1 / 120); accumulator.current -= 1 / 120;
 +   }
291      } else accumulator.current = 0;
292      if (previousStatus.current !== 'playing' && game.status === 'playing' && game.time < .2)
 + cameraReady.current = false;
293      previousStatus.current = game.status;
294      const wide=cameraMode==='wide',portrait=camera.aspect<.85;
295      const index=level.platforms.findIndex(p=>game.z>p.z-p.d/2-1.5),next=level.platforms[Math.min(level.pla
 + tforms.length-1,Math.max(0,index)+1)];
296      const lookX=mobile?game.x*.76+next.x*.24:game.x*.93;
297      const floor=game.lastSafe?.y??0;
298      cameraHeight.current=THREE.MathUtils.lerp(cameraHeight.current,floor+Math.max(0,game.y-floor)*.35,1-Ma
 + th.exp(-delta*5));
299      const height=cameraHeight.current;
300      const speed=Math.min(1,Math.hypot(game.vx,game.vz)/6);const fov=(mobile?(portrait?68:61):57)+(game.gro
 + und&&speed>.8?1.8:0);camera.fov=THREE.MathUtils.lerp(camera.fov,fov,1-Math.exp(-delta*5));camera.updatePro
 + jectionMatrix();
301      target.set(mobile?game.x*.86+next.x*.14:game.x*.85,height+(wide?8.2:mobile?6.2:4.9),game.z+(wide?12:mo
 + bile?10:8));
302      if(!cameraReady.current){camera.position.copy(target);cameraReady.current=true;}
303      camera.position.lerp(target,1-Math.exp(-delta*6));
304      camera.lookAt(lookX,height+.8,game.z-(wide?4:mobile?5.4:4.5));
305      onStep();
306    }, -2);
307    return <LowDetail.Provider value={quality==='low'}><WorldTheme.Provider value={theme}>
308      <fog attach="fog" args={[theme.skyBottom, 50, 170]} />
309      <hemisphereLight args={[theme.cloud, theme.rock[2], 2.1]} />
310      <directionalLight position={[-18, 28, 14]} intensity={3.2} color="#fff2c8" castShadow={quality ===
 + 'high'} shadow-mapSize={[2048, 2048]} shadow-camera-left={-32} shadow-camera-right={30}
 + shadow-camera-top={25} shadow-camera-bottom={-145} shadow-camera-far={210} shadow-radius={3}
 + shadow-blurSamples={8} shadow-bias={-.0005} shadow-normalBias={.035} />
311      <Background /><Weather game={game}/><ParticleEffects game={game} low={quality==='low'}/><FrameMonitor
 + game={game} onSlow={onSlow} onStats={onStats}/>
312      {level.platforms.map((p, i) => <Island key={p.id} p={p} index={i} game={game} />)}
313      {level.coins.map((at, i) => <Coin key={i} at={at} index={i} game={game} />)}
314      {level.rotators.map((r, i) => <Rotator key={i} r={r} game={game} />)}
315      {level.hazards.map((h, i) => <group key={i} position={[h.x, h.y, h.z]}><Box at={[0, .05, 0]}
 + size={[1.65, .1, 1.4]} color="#687782" />{[-.5, 0, .5].flatMap(x => [-.3, .3].map(z => <mesh
 + key={`${x},${z}`} position={[x, .39, z]} castShadow><coneGeometry args={[.2, .7, 7]}
 + /><meshStandardMaterial color="#c4d7e2" metalness={.75} roughness={.25} /></mesh>))}</group>)}
316      <Sign at={[-3.1, 0, 3.2]}>START ↑</Sign><Sign at={[2.5, 0, -2.9]}>JUMP ↑</Sign><Sign
 + at={[level.checkpoint[0]+3.3,level.checkpoint[1],level.checkpoint[2]-3]}>WAIT · JUMP</Sign>
317      <Checkpoint game={game} /><Portal game={game} /><Character game={game} /><LandingMarker
 + game={game}/><Celebration game={game} />
318    </WorldTheme.Provider></LowDetail.Provider>;
319  }
320
321
322  export default memo(Scene);
323
324
325
326
```

Code 16 / src/StaticIsland.jsx

23 original lines | 863 bytes | complete file

Create/dispose baked scenery in Smooth mode and restore originals when quality changes.

```
import React, { useLayoutEffect, useRef } from 'react';
import * as THREE from 'three';
import { bakeIsland } from './batchGeometry';

export default function StaticIsland({ enabled, children }) {
  const root = useRef();
  useLayoutEffect(() => {
    if (!enabled) return;
    const batch = bakeIsland(root.current);
    if (!batch) return;
    const material = new THREE.MeshStandardMaterial({ vertexColors: true, roughness: .9, side:
THREE.DoubleSide });
    const mesh = new THREE.Mesh(batch.geometry, material);
    batch.originals.forEach(object => { object.visible = false; });
    root.current.add(mesh);
    return () => {
      mesh.removeFromParent();
      batch.originals.forEach(object => { object.visible = true; });
      batch.geometry.dispose();
      material.dispose();
    };
  }, [enabled]);
  return <group ref={root}>{children}</group>;
}
```

Code 17 / src/style.css

37 original lines | 29737 bytes | complete file

Responsive styling, safe areas, visual feedback and touch layouts.

```
1  :root{font-family:Inter,ui-sans-serif,system-ui,-apple-system,BlinkMacSystemFont,'Segoe UI',sans-serif;col
+  or:#fff7df;background:#6bb7f3;font-synthesis:none}*{box-sizing:border-box}body{margin:0;overscroll-behavio
+  r:none}button,select{font:inherit}button{cursor:pointer;touch-action:manipulation}button,a{-webkit-tap-hig
+  hlight-color:transparent}button:focus-visible,select:focus-visible{outline:3px solid #fff6b5;outline-offse
+  t:4px}button:disabled{opacity:.4;cursor:default}main{height:100dvh;min-height:420px;position:relative;over
+  flow:hidden;user-select:none}canvas{display:block;touch-action:none}.topbar{position:absolute;top:0;left:0
+  ;width:100%;display:flex;align-items:flex-start;justify-content:space-between;gap:18px;padding:23px 28px;p
+  ointer-events:none}.topbar>*{pointer-events:auto}.wordmark{font-weight:1000;line-height:1.05;letter-spacin
+  g:-.7px;font-size:21px;transform:rotate(-3deg);filter:drop-shadow(0 3px 0 #693c23);text-shadow:2px 0
+  #623918,-2px 0 #623918,0 -2px #623918,0 2px #623918;color:#ffe07c}.wordmark>span{display:block}.wordmark
+  b{display:block;color:#e9fcff;font-size:20px}.wordmark
+  i{font-style:normal;color:#8ee7ff}.hud{display:flex;align-items:center;gap:22px;border:1px solid
+  #ffffff55;box-shadow:0 3px 0 #17436d3d,0 5px 20px
+  #24537020;background:#123b60d9;backdrop-filter:blur(12px);padding:13px 22px;border-radius:16px;font-varian
+  t-numeric:tabular-nums}.coin-stat{display:flex;gap:8px;align-items:center}.coin-stat
+  strong{font-size:23px}.coin-stat
+  small{font-size:12px;color:#c1d8e6}.coin-icon{width:25px;height:29px;background:#ffc83f;border:3px solid #
+  ffdf72;border-radius:50%;display:inline-flex;align-items:center;justify-content:center;color:#e89613;font-
+  size:17px;box-shadow:inset 0 0 0 1px #eba421,0 2px 0 #b27a21}.time-stat{border-inline:1px solid
+  #ffffff26;padding:0 20px;text-align:center;line-height:1.15}.stat-label{display:block;font-size:8px;letter
+  -spacing:2px;color:#aec6d7;margin-bottom:4px}.time-stat strong{font-size:17px;letter-spacing:.5px;font-wei
+  ght:650}.hearts{display:flex;color:#ff7f70;gap:4px;font-size:25px;text-shadow:0 2px 0
+  #793633}.empty{opacity:.22}.tools{display:flex;gap:8px}.tools button{border:1px solid #ffffff88;color:#fff
+  df1;background:#174776b8;border-radius:12px;width:42px;height:42px;font-size:21px;box-shadow:0 3px 0
+  #16446d3d;backdrop-filter:blur(12px)}.tools button:hover{background:#2a648b}.trail-progress{position:absol
+  ute;bottom:28px;left:50%;transform:translateX(-50%);width:190px;text-align:center;color:#294c66;font-size:
+  11px;font-weight:700;text-shadow:0 1px 2px
+  #fff8}.trail-progress>div{height:5px;background:#1c557a40;border:1px solid
+  #fff7;border-radius:10px;margin-top:8px;overflow:hidden}.trail-progress
+  i{display:block;height:100%;background:#fff1ad;border-radius:10px;transition:width .15s}.trail-progress
+  b{position:absolute;bottom:-4px;left:51%;font-size:17px;color:#567889}.trail-progress b.lit{color:#e85135}
+  .desktop-help{position:absolute;bottom:28px;left:28px;display:flex;align-items:center;gap:7px;font-size:10
+  px;font-weight:600;color:#284f68;text-shadow:0 1px 2px #fff9}.desktop-help span{display:none}.desktop-help
+   kbd{font:inherit;font-size:9px;border:1px solid #fff7;background:#e8f7ff9e;border-radius:5px;padding:5px
+  6px;margin-left:7px;color:#254e68}.desktop-help kbd:first-child{margin-left:0}.menu-layer{position:absolut
+  e;inset:0;display:flex;align-items:center;justify-content:center;background:linear-gradient(180deg,#154d70
+  30,#0d3b5540);padding:100px 20px 50px;overflow:auto;backdrop-filter:blur(2px)}.menu-card{width:410px;max-w
+  idth:100%;background:linear-gradient(155deg,#173e5cf5,#112f47f7);border:1px solid
+  #c4edff55;border-radius:22px;padding:26px 30px;text-align:center;box-shadow:0 12px 55px #0c2d4d55,0 4px 0
+  #0d2a42aa}.chapter-label{font-size:9px;letter-spacing:1.8px;color:#b5d5df;font-weight:700}.chapter-label
+  span{color:#ffc950;padding:0
+  7px}.menu-emblem{width:49px;height:49px;display:grid;place-items:center;margin:21px auto
+  10px;color:#ffdb6c;font-size:34px;filter:drop-shadow(0 2px 8px
+  #ffda3830)}h1{font-size:27px;letter-spacing:-.8px;line-height:1.2;margin:8px 0
+  12px;font-weight:850}.menu-card p{font-size:12px;line-height:1.8;color:#b7d0e0;margin:0 auto
+  23px;max-width:310px}.primary{border:1px solid #ffe38d;box-shadow:0 4px 0 #b77724,0 7px 20px
+  #0b253b55;background:linear-gradient(#ffdc72,#ffc342);color:#59401d;width:100%;padding:15px 18px;border-ra
+  dius:10px;letter-spacing:1px;font-size:12px;font-weight:850;display:flex;justify-content:center;align-item
+  s:center;gap:18px}.primary:hover{background:#ffe38e}.primary:active{transform:translateY(2px);box-shadow:0
+   2px 0 #b77724}.primary span{font-size:12px}.secondary{display:block;width:100%;background:none;border:non
+  e;color:#9ebed1;font-size:11px;padding:19px 0
+  2px}.secondary:hover{color:white}.instructions{display:flex;flex-direction:column;gap:8px;margin:26px 0
+  19px;color:#a9c6d8;font-size:10px}.instructions kbd{font-family:inherit;color:#f6e6ad;font-weight:700;font
+  -size:9px}.settings{display:flex;align-items:end;justify-content:center;gap:10px;border-top:1px solid
+  #ffffff16;padding-top:17px}.settings label{font-size:9px;color:#95b6cc;text-align:left;flex:1}.settings
+  select{display:block;margin-top:6px;width:100%;font-size:10px;border:1px solid
+  #accadd44;color:#d4e6ee;background:#173951;border-radius:6px;padding:7px
+  5px;cursor:pointer}.sound-setting{font-size:9px;background:transparent;color:#d7e9ec;border:1px solid
+  #accadd44;border-radius:6px;padding:8px 7px;white-space:nowrap;height:29px}.best{margin-top:20px;font-size
+  :8px;letter-spacing:1.5px;color:#8dafc5}.best strong{color:#ffe099;margin-left:12px;font-size:12px;letter-
+  spacing:.6px;font-variant-numeric:tabular-nums}.menu-footnote{position:absolute;bottom:18px;left:0;width:1
+  00%;text-align:center;font-size:9px;color:#244f6a;text-shadow:0 1px 2px white;font-weight:600}.toast{posit
+  ion:absolute;top:110px;left:50%;transform:translateX(-50%);background:#fff0b9f5;color:#5e461c;border:1px
+  solid #ffdd83;border-radius:30px;box-shadow:0 5px 15px #173e6126;padding:13px 22px;font-size:12px;font-wei
+  ght:650;text-align:center;max-width:90%;width:max-content;pointer-events:none}.stars{display:flex;justify-
+  content:center;gap:8px;font-size:37px;color:#ffd15b;text-shadow:0 3px 0 #af742d;margin:9px 0
+  20px}.results{display:flex;justify-content:space-between;gap:15px;margin:0 0 20px}.results
+  small{display:block;font-size:8px;color:#89afc4;letter-spacing:.7px;margin-bottom:8px}.results b{font-size
+  :17px;font-weight:650;font-variant-numeric:tabular-nums}.record{font-size:9px;letter-spacing:1.8px;color:#
+  ffe295;margin:-3px 0 22px}.mobile-controls{display:none}.sr-only{position:absolute;width:1px;height:1px;ov
+  erflow:hidden;clip:rect(0,0,0,0);white-space:nowrap}.graphics-error{position:absolute;inset:0;z-index:20;b
```

```
+  ackground:#163953;padding:30px;display:grid;place-content:center;text-align:center}.graphics-error
+  p{color:#b9d3df}.graphics-error button{border:0;border-radius:8px;padding:15px;background:#ffcf61}.loading
+  -bar{height:4px;border-radius:5px;background:linear-gradient(90deg,#3c7188,#ffe28a,#3c7188);background-siz
+  e:200% 100%;animation:loading 1s linear infinite;margin:25px 0}.menu-card:has(.loading-bar){padding:45px
+  30px}@keyframes loading{to{background-position:200% 0}}
2 @media(pointer:coarse),(max-width:700px),(max-width:950px) and (max-height:500px){.mobile-controls{display
+  :flex;position:absolute;bottom:37px;left:23px;right:23px;justify-content:space-between;align-items:flex-en
+  d;pointer-events:none}.touch{pointer-events:auto;touch-action:none;user-select:none;border:1px solid #ffff
+  ff96;color:#fff9e7;background:#17446bb8;backdrop-filter:blur(10px);width:51px;height:51px;border-radius:14
+  px;font-size:26px;box-shadow:0 3px 0 #14385630}.touch:active{background:#568bb2}.dpad{display:grid;grid-te
+  mplate:repeat(2,51px)/repeat(3,51px);gap:5px}.up{grid-column:2}.left{grid-row:2;grid-column:1}.down{grid-r
+  ow:2;grid-column:2}.right{grid-row:2;grid-column:3}.jump{width:80px;height:80px;border-radius:50%;border:3
+  px solid
+  #ffeaa4;background:#ffd064e8;color:#695025;font-size:12px;font-weight:900;letter-spacing:1px;box-shadow:0
+  4px 0 #b57c3b66}.jump:active{background:#ffe6a2}.desktop-help{display:none}.trail-progress{bottom:17px;wid
+  th:130px;font-size:9px}.trail-progress b{font-size:14px}}
3 @media(max-width:700px){.topbar{padding:17px
+  17px;gap:10px;flex-wrap:wrap}.wordmark{font-size:15px;letter-spacing:-.3px}.wordmark
+  b{font-size:15px}.tools{gap:5px}.tools
+  button{width:34px;height:34px;border-radius:10px;font-size:17px}.hud{order:3;margin:0
+  auto;gap:15px;padding:9px 15px;border-radius:12px}.coin-stat{gap:6px}.coin-stat
+  strong{font-size:19px}.coin-stat
+  small{font-size:10px}.coin-icon{width:21px;height:24px;font-size:14px}.time-stat{padding:0 14px}.time-stat
+   strong{font-size:14px}.stat-label{font-size:7px}.hearts{font-size:22px;gap:3px}.menu-layer{padding:135px
+  20px 35px}.menu-card{width:365px;padding:23px 24px;border-radius:18px}.menu-emblem{margin:12px auto
+  8px;width:38px;height:38px;font-size:30px}h1{font-size:25px}.menu-card
+  p{font-size:11px;margin-bottom:20px}.instructions{font-size:9px;margin:23px 0
+  16px}.chapter-label{font-size:8px}.settings{gap:7px}.settings
+  select{font-size:9px}.sound-setting{font-size:8px}.toast{top:145px;font-size:10px;padding:12px
+  17px}.menu-footnote{font-size:8px;bottom:11px}.results{gap:9px}.results small{font-size:7px}.results
+  b{font-size:15px}}
4 @media(max-height:690px){.menu-layer{align-items:flex-start}.menu-card{padding:18px
+  24px}.menu-emblem{display:none}h1{margin-top:15px;font-size:23px}.menu-card
+  p{font-size:11px;margin-bottom:15px}.instructions{margin:20px 0 13px;gap:6px}.settings{padding-top:12px}.b
+  est{margin-top:12px}.primary{padding:13px}.menu-footnote{display:none}}
5 @media(max-height:500px) and
+  (min-width:701px){.menu-layer{padding-top:83px}.menu-card{width:620px}.menu-card p{max-width:480px;margin-
+  bottom:12px}.instructions{flex-direction:row;justify-content:center;font-size:8px;margin:18px 0 12px}.sett
+  ings{max-width:390px;margin:auto}.chapter-label{font-size:8px}.best{margin-top:10px}.mobile-controls{botto
+  m:15px}.touch{width:43px;height:43px}.dpad{grid-template:repeat(2,43px)/repeat(3,43px)}.jump{width:65px;he
+  ight:65px}}
6 @media(prefers-reduced-motion:reduce){.loading-bar{animation:none}.trail-progress i{transition:none}}
7 .wordmark small{display:block;color:#f8f0cd;font-size:8px;font-weight:650;letter-spacing:.35px;margin-top:
+  7px;text-shadow:0 1px 1px
+  #22394c;filter:none;transform:rotate(3deg)}.world-select{display:flex;gap:9px;margin:22px -6px 0}.world-op
+  tion{display:flex;flex:1;align-items:center;gap:7px;text-align:left;background:#2c60793d;border:1px solid
+  #81a8bd44;border-radius:9px;padding:9px;color:#bed3df;min-width:0}.world-option.selected{border-color:#ffd
+  a78;background:#8e78401a}.world-option.ember{background:#9f566a26}.world-option:disabled{opacity:.55}.worl
+  d-option>span{min-width:0}.world-option
+  small{font-size:7px;letter-spacing:1px;color:#e6d8a8;display:block}.world-option
+  b{display:block;font-size:10px;line-height:1.4;margin:5px 0}.world-option em{display:block;font-size:7px;l
+  ine-height:1.5;color:#91b2c8;font-style:normal}.route-map{width:27px;height:84px;flex-shrink:0}.world-sele
+  ct+.instructions{margin-top:18px}.menu-card:has(.world-select){margin-block:auto;padding-top:21px}.menu-ca
+  rd:has(.world-select) .menu-emblem{height:25px;margin:12px auto
+  8px;font-size:27px}.menu-card:has(.world-select) h1{font-size:25px}.menu-card:has(.world-select)
+  p{margin-bottom:19px}.menu-card:has(.world-select)
+  .instructions{font-size:9px;gap:6px}.menu-card:has(.world-select)
+  .settings{padding-top:13px}.menu-card:has(.world-select)
+  .best{margin-top:13px}@media(max-width:700px){.world-option{padding:7px;gap:5px}.world-option
+  b{font-size:9px}.world-option em{font-size:6px}.route-map{width:22px;height:70px}.world-select{margin-top:
+  17px}.menu-card:has(.world-select){padding:20px}.menu-card:has(.world-select)
+  h1{font-size:23px}.menu-card:has(.world-select) .menu-emblem{display:none}.menu-card:has(.world-select)
+  .instructions{font-size:8px}.wordmark small{font-size:6px}}@media(max-height:700px){.menu-card:has(.world-
+  select){margin-block:0}.menu-card:has(.world-select)
+  .menu-emblem{display:none}.menu-card:has(.world-select) h1{font-size:21px}.menu-card:has(.world-select)
+  .instructions{display:none}.world-select{margin-top:15px}.route-map{height:60px}}
8 [hidden]{display:none!important}.atlas-toggle{display:flex;justify-content:space-between;width:100%;border
+  :1px solid #9fc4d144;border-radius:9px;background:#315e7740;color:#e0eff1;padding:12px
+  13px;font-size:11px;margin-top:18px}.atlas-toggle
+  span{font-size:9px;color:#a5c4d3}.atlas-card{width:760px;max-height:calc(100dvh -
+  145px);overflow:auto;scrollbar-color:#7399b0 #1a3d56}.atlas-card .menu-emblem{display:none}.atlas-card .wo
+  rld-select{display:grid;grid-template-columns:repeat(3,minmax(0,1fr));gap:10px;max-height:49dvh;overflow:a
+  uto;padding:4px;margin:15px -5px 0;overscroll-behavior:contain;scrollbar-color:#7399b0 #173b52}.atlas-card
+   .world-option{min-height:108px;border-color:#8ba5bb55;border-left:4px solid
+  var(--world-ground);padding:10px;background:linear-gradient(120deg,#244962,#1a354f)}.atlas-card
+  .world-option.selected{border-color:var(--world-accent);background:#34566a}.atlas-card .world-option
+  b{font-size:12px;margin:6px 0}.atlas-card .world-option
+  small{font-size:8px;color:var(--world-accent)}.atlas-card .world-option em{font-size:8px}.atlas-card .rout
+  e-map{width:36px;height:86px}.atlas-summary{display:flex;justify-content:space-between;align-items:center;
```

```
+ font-size:10px;color:#a9c8d5;margin-top:18px}.atlas-summary button{border:1px solid
+ #9fc4d144;background:#315e7740;color:#edf5ed;border-radius:6px;padding:7px 13px;font-size:10px}.atlas-card
+  .menu-emblem+h1{margin-top:17px}.atlas-card p{max-width:500px}.atlas-card
+ h1{font-size:27px}.menu-card:not(.atlas-card)
+ .instructions{margin-top:18px}.menu-layer:has(.atlas-card){align-items:center}.atlas-card
+ .best{display:none}@media(max-width:700px){.atlas-card{width:100%;max-height:calc(100dvh -
+ 165px);padding:19px!important}.atlas-card
+ .world-select{grid-template-columns:repeat(2,minmax(0,1fr));max-height:48dvh;gap:8px}.atlas-card
+ .world-option{min-height:99px;padding:7px;gap:6px}.atlas-card .world-option b{font-size:10px}.atlas-card
+ .world-option em{font-size:7px}.atlas-card .route-map{width:24px;height:76px}.atlas-card
+ .chapter-label{font-size:7px}.atlas-card h1{font-size:24px!important}.atlas-card
+ p{font-size:10px}.menu-layer:has(.atlas-card){padding-inline:13px}}@media(max-height:600px){.atlas-card
+ .world-select{max-height:39dvh}.atlas-card .chapter-label{display:none}.atlas-card h1{margin:3px 0
+ 8px}.atlas-card p{margin-bottom:7px}.atlas-card .atlas-summary{margin-top:8px}.atlas-card
+ .world-select{margin-top:8px}}
9 .tools .map-button{width:57px;font-size:11px;font-weight:800;letter-spacing:1px}.map-filters{display:flex;
+ gap:8px;margin:13px 0 4px}.map-filters button{border:1px solid
+ #9ac7d344;border-radius:18px;background:#28526b55;color:#a9c8d9;padding:7px
+ 14px;font-size:10px}.map-filters button[aria-pressed=true]{background:#f8d37a;color:#4c432c;border-color:#
+ ffe7a5}.map-stars{display:flex;gap:3px;margin:3px 0 6px;font-size:15px;line-height:1}.map-stars
+ i{font-style:normal;color:#647e94}.map-stars i.earned{color:#ffda69;text-shadow:0 1px #b17a29}.atlas-card
+ .world-option{min-height:132px}.atlas-card .world-option
+ .map-tasks{font-size:7px;line-height:1.7;margin-top:6px;color:#b7d3dd}.atlas-card .world-option
+ .current-run{color:#ffe08b;font-size:9px;margin-top:6px}.empty-maps{padding:20px 0;color:#a9c8d5}.continue
+ -button{display:block;width:100%;background:linear-gradient(#73cba7,#45ae8d);color:#103e38;border:1px
+ solid #a4efd2;box-shadow:0 3px 0 #226852;border-radius:10px;padding:13px;font-size:11px;font-weight:850;le
+ tter-spacing:.5px;margin-bottom:14px}.continue-button small{display:block;font-size:9px;letter-spacing:0;f
+ ont-weight:500;margin-top:6px}.task-hud{position:absolute;left:28px;bottom:80px;background:#133d59c9;borde
+ r:1px solid #ffffff35;border-radius:10px;padding:13px 16px;display:flex;flex-direction:column;gap:6px;colo
+ r:#bed4de;font-size:10px;pointer-events:none;backdrop-filter:blur(8px)}.task-hud
+ b{font-size:8px;letter-spacing:1.3px;color:#ffda83;margin-bottom:3px}.task-hud
+ .done{color:#9be4ba}.atlas-card .world-select{max-height:42dvh}@media(max-width:700px){.tools
+ .map-button{width:42px;font-size:9px}.tools{gap:4px}.topbar{padding-inline:13px}.tools
+ button{width:31px}.wordmark{font-size:13px}.wordmark
+ b{font-size:13px}.task-hud{left:17px;bottom:167px;font-size:8px;gap:4px;padding:9px 12px}.task-hud
+ b{font-size:7px}.atlas-card .world-option{min-height:133px}.map-filters{gap:6px}.map-filters
+ button{padding:6px 10px;font-size:9px}.atlas-card .world-option .map-tasks{font-size:6px}.atlas-card
+ .world-select{max-height:40dvh}.map-stars{font-size:13px}.continue-button{font-size:10px}.continue-button
+ small{font-size:8px}}
10 .topbar{z-index:3}.menu-layer{z-index:2}.toast{z-index:1}.graphics-error{z-index:10}
11 /* Thumb controls reserve the bottom corners and stay independent for multi-touch. */
12 .coin-count{animation:coin-pop .22s ease-out}@keyframes coin-pop{50%{transform:scale(1.2);color:#ffe49a}}
13 @media(pointer:coarse),(max-width:700px),(max-width:950px) and (max-height:500px){
14  main{min-height:0}.mobile-controls{display:flex;z-index:1;left:max(18px,env(safe-area-inset-left));right:
+ max(18px,env(safe-area-inset-right));bottom:max(20px,env(safe-area-inset-bottom));align-items:flex-end;gap
+ :20px;pointer-events:none}.stick-group,.jump-group{display:flex;flex-direction:column;align-items:center;g
+ ap:9px;pointer-events:none}.joystick{position:relative;width:154px;height:154px;border-radius:50%;backgrou
+ nd:radial-gradient(circle,#153a5866 0%,#153a582b 65%,#ffffff0a 66%);border:1px solid
+ #f3ffff80;box-shadow:inset 0 0 0 8px #c6ecff0d,0 3px 20px #14384a20;pointer-events:auto;touch-action:none;
+ -webkit-touch-callout:none;user-select:none;backdrop-filter:blur(3px)}.joystick.steering{background:#153a5
+ 855;border-color:#dffbff}.stick-orbit{position:absolute;inset:24%;border:1px dashed #e0f7ff45;border-radiu
+ s:50%;pointer-events:none}.stick-cross{position:absolute;background:#d9f4ff30;pointer-events:none}.stick-c
+ ross.x{width:54%;height:1px;left:23%;top:50%}.stick-cross.y{height:54%;width:1px;top:23%;left:50%}.stick-t
+ humb{position:absolute;width:62px;height:62px;left:calc(50% - 31px);top:calc(50% -
+ 31px);border-radius:50%;border:2px solid
+ #c6f3ffcc;background:linear-gradient(145deg,#789faebd,#254f6ecf);box-shadow:0 4px 12px #102d4b50,inset 0
+ 2px 0 #ffffff55;pointer-events:none;will-change:transform}.joystick:not(.steering)
+ .stick-thumb{transition:transform .1s ease-out}.stick-arrow{position:absolute;top:7px;left:calc(50% -
+ 7px);font-size:24px;color:#e0f8ffc2;pointer-events:none}.jump-pad{width:102px;height:102px;border:3px
+ solid #fff1b6;border-radius:50%;background:linear-gradient(#ffdf80,#f1bb45);color:#705125;box-shadow:0 5px
+  0 #8c6c444f,0 6px 20px #17344b30;pointer-events:auto;touch-action:none;-webkit-touch-callout:none;user-se
+ lect:none;display:flex;align-items:center;justify-content:center;flex-direction:column;gap:0}.jump-pad>spa
+ n{font-size:32px;line-height:1.1;font-weight:600}.jump-pad>b{font-size:12px;letter-spacing:1.6px}.jump-pad
+ .jumping{transform:translateY(3px);background:#ffe9a3;box-shadow:0 2px 0
+ #8c6c444f}.control-caption{font-size:8px;letter-spacing:1.2px;font-weight:750;color:#e7f7fa;text-shadow:0
+ 1px 5px #18394a,0 1px 1px #18394a;pointer-events:none}.task-hud{display:none}.trail-progress{bottom:max(22
+ px,env(safe-area-inset-bottom));width:100px;font-size:8px}.topbar{padding-top:max(12px,env(safe-area-inset
+ -top));gap:9px}.hud{padding:7px 13px;gap:9px}.time-stat{padding:0 11px}.wordmark small{max-width:110px;ove
+ rflow:hidden;white-space:nowrap;text-overflow:ellipsis}.menu-footnote{display:none}.desktop-help{display:n
+ one}.toast{font-size:10px;top:132px}.menu-layer{padding-bottom:max(20px,env(safe-area-inset-bottom));}.men
+ u-card .instructions{line-height:1.5;font-size:9px}.mobile-controls~*{-webkit-user-select:none}
15 }
16 @media(max-width:360px){.joystick{width:134px;height:134px}.stick-thumb{width:56px;height:56px;left:calc(5
+ 0% - 28px);top:calc(50% - 28px)}.jump-pad{width:94px;height:94px}.mobile-controls{left:14px;right:14px}.co
+ ntrol-caption{font-size:7px}.wordmark{font-size:11px}.wordmark b{font-size:12px}.tools
+ .map-button{width:40px}.tools button{width:29px}}
17 @media(pointer:coarse) and (orientation:landscape),(max-width:950px) and (max-height:500px){
18  main{min-height:0}.topbar{flex-wrap:nowrap;padding:10px max(14px,env(safe-area-inset-right)) 0
+ max(14px,env(safe-area-inset-left));align-items:flex-start}.hud{order:0;padding:7px
```

```
+ 13px;margin:0;gap:9px}.wordmark{font-size:12px}.wordmark b{font-size:12px}.tools
+ button{height:34px;width:34px}.tools .map-button{width:46px}.mobile-controls{left:max(24px,env(safe-area-i
+ nset-left));right:max(24px,env(safe-area-inset-right));bottom:max(12px,env(safe-area-inset-bottom))}.joyst
+ ick{width:126px;height:126px}.stick-thumb{width:52px;height:52px;left:calc(50% - 26px);top:calc(50% - 26px
+ )}.jump-pad{width:88px;height:88px}.control-caption{font-size:7px}.trail-progress{bottom:16px}.task-hud,.d
+ esktop-help{display:none}.menu-layer{padding:76px 18px
+ 12px;align-items:flex-start}.menu-card{width:510px;padding:15px 24px;max-height:calc(100dvh -
+ 88px);overflow:auto}.menu-card h1{font-size:22px}.menu-card
+ p{max-width:440px;font-size:10px;margin-bottom:12px}.menu-card .instructions{display:none}.menu-card
+ .settings{padding-top:10px}.menu-card .atlas-toggle{margin-top:13px}.menu-card
+ .menu-emblem{display:none}.menu-card
+ .best{margin-top:10px}.toast{top:80px}.atlas-card{max-height:calc(100dvh - 88px)}.atlas-card
+ .world-select{max-height:38dvh;grid-template-columns:repeat(3,minmax(0,1fr))}
19 }
20 @media(prefers-reduced-motion:reduce){.coin-count{animation:none}.stick-thumb{transition:none!important}}
21
22 /* Final mobile layout: a floating stick zone, unobstructed route and collapsible tasks. */
23 html,body,#root{width:100%;height:100%;overflow:hidden;touch-action:none;-webkit-text-size-adjust:100%}but
 + ton,svg,canvas{-webkit-touch-callout:none}.tools
 + button{display:flex;align-items:center;justify-content:center;gap:5px}.tools button
 + svg{flex-shrink:0}.tools .map-button{width:64px;font-size:10px}.hearts
 + span{display:flex;align-items:center}.heart-hit{animation:heart-shake .35s ease-out}@keyframes heart-shake
 + {20%,60%{transform:translateX(-4px)}40%,80%{transform:translateX(4px)}}.respawn-fade{position:absolute;ins
 + et:0;z-index:5;background:#e8f6ff;pointer-events:none;transition:opacity .07s linear}.task-panel{position:
 + absolute;z-index:1;top:116px;left:26px;width:220px;background:#143e58dc;border:1px solid
 + #d6edf544;border-radius:11px;backdrop-filter:blur(8px);box-shadow:0 4px 16px #113e5520}.task-toggle{displa
 + y:flex;gap:9px;align-items:center;border:0;background:none;color:#d2e9ed;width:100%;padding:12px;font-size
 + :11px}.task-toggle b{margin-left:auto;color:#ffda8a;font-size:10px}.task-toggle
 + svg:last-child{transition:transform .15s}.task-panel.expanded .task-toggle
 + svg:last-child{transform:rotate(180deg)}.task-list{display:flex;flex-direction:column;gap:10px;padding:0
 + 15px 15px;color:#aac8d5;font-size:11px}.task-list .done{color:#9de2be}.rotate-hint{display:none}.extras-se
 + ttings{display:flex;align-items:center;gap:12px;margin-top:13px;text-align:left}.extras-settings
 + button{border:1px solid #accadd44;border-radius:6px;padding:7px;background:transparent;color:#cce1e5;font-
 + size:9px;white-space:nowrap}.extras-settings
 + span{font-size:8px;color:#8fb3c7}.run-medals{display:flex;justify-content:center;gap:11px;margin:0 0
 + 20px;font-size:9px;color:#799bae}.run-medals .earned{color:#f3dc91}.victory-burst{font-size:24px;letter-sp
 + acing:15px;color:#fada7a;animation:victory-float 2s ease-in-out infinite;margin:-5px 0 13px}.won
 + .menu-card{border-color:#e4cb7c88;box-shadow:0 12px 60px #f4cd5d2a,0 4px 0 #0d2a42aa}@keyframes
 + victory-float{50%{transform:translateY(-5px);opacity:.7}}
24 @media(any-pointer:coarse),(max-width:900px),(max-width:950px) and (max-height:500px){
25  main{min-height:0}.mobile-controls{display:block;position:absolute;inset:0;z-index:1;pointer-events:none}
 + .move-zone{position:absolute;left:0;bottom:0;width:50%;height:44%;min-height:220px;max-height:390px;pointe
 + r-events:auto;touch-action:none;user-select:none;-webkit-touch-callout:none}.move-zone .joystick{position:
 + absolute;width:138px;height:138px;left:88px;top:auto;bottom:36px;transform:translateX(-50%);pointer-events
 + :none;background:radial-gradient(circle,#153a5838,#153a581c 65%,#ffffff08 66%);border:1px solid
 + #f3ffff65}.move-zone .joystick.floating{bottom:auto;transform:translate(-50%,-50%);opacity:1}.move-zone
 + .joystick.resting{opacity:.65}.move-zone .stick-thumb{width:58px;height:58px;left:calc(50% -
 + 29px);top:calc(50% - 29px)}.move-zone .stick-caption{position:absolute;left:17px;bottom:max(15px,env(safe-
 + area-inset-bottom));font-size:7px;letter-spacing:.6px}.jump-group{position:absolute;right:max(18px,env(saf
 + e-area-inset-right));bottom:max(14px,env(safe-area-inset-bottom));gap:10px}.jump-pad{width:100px;height:10
 + 0px;pointer-events:auto}.jump-group .control-caption{font-size:7px;letter-spacing:.55px}.topbar{display:gr
 + id;grid-template-columns:minmax(80px,1fr)
 + auto;gap:12px;align-items:start;padding:max(12px,env(safe-area-inset-top))
 + max(14px,env(safe-area-inset-right)) 0 max(14px,env(safe-area-inset-left));z-index:3}.wordmark{font-size:1
 + 2px;letter-spacing:-.4px;max-width:114px}.wordmark b{font-size:12px}.wordmark
 + small{font-size:6px;max-width:110px}.tools{gap:6px;justify-self:end}.tools
 + button{width:36px;height:36px;border-radius:11px}.tools .map-button{width:58px;font-size:9px}.tools button
 +  svg{width:17px;height:17px}.hud{grid-column:1/-1;order:3;justify-self:center;margin:0;padding:8px
 + 15px;gap:12px;border-radius:14px;box-shadow:0 3px 0 #17436d26}.hearts{gap:4px}.hearts
 + svg{width:18px;height:18px}.time-stat{padding:0 12px}.time-stat strong{font-size:14px}.coin-stat
 + strong{font-size:19px}.stat-label{font-size:7px}.coin-stat small{font-size:10px}.coin-icon{width:22px;heig
 + ht:24px;font-size:14px}.task-panel{top:130px;left:14px;width:190px;z-index:1;background:#123b55c7}.task-to
 + ggle{padding:9px 11px;font-size:10px;gap:7px}.task-list{font-size:10px;gap:8px;padding:0 12px
 + 12px}.task-panel:not(.expanded){width:164px}.task-toggle b{font-size:9px}.trail-progress{left:50%;bottom:2
 + 3px;width:84px;font-size:7px;z-index:0}.trail-progress>span{display:none}.trail-progress>div{height:4px}.t
 + rail-progress b{font-size:12px}.rotate-hint{display:block;position:absolute;left:50%;bottom:69px;transform
 + :translateX(-50%);width:95px;color:#f4fbfb;text-shadow:0 1px 4px #173b50;text-align:center;font-size:8px;l
 + ine-height:1.5;pointer-events:none}.toast{top:175px;font-size:10px;padding:10px
 + 15px}.menu-layer{padding:130px 15px
 + max(16px,env(safe-area-inset-bottom));align-items:center}.menu-card{width:370px;max-height:calc(100dvh -
 + 146px);overflow:auto;padding:22px}.menu-card .menu-emblem{display:none}.menu-card
 + h1{font-size:24px}.menu-card p{font-size:11px;margin-bottom:17px}.menu-card
 + .instructions{font-size:9px;margin:17px 0 15px}.menu-card
 + .settings{padding-top:13px}.atlas-card{max-height:calc(100dvh - 146px)}.atlas-card
 + .world-select{max-height:35dvh}.desktop-help,.menu-footnote{display:none}
26 }
27 @media(max-width:380px){.tools{gap:4px}.tools button{width:32px;height:34px}.tools
 + .map-button{width:51px}.wordmark{font-size:11px}.wordmark
 + b{font-size:11px}.task-panel{top:129px}.move-zone .joystick{left:79px;width:126px;height:126px}.jump-group
```

```
+ {right:14px}.jump-pad{width:94px;height:94px}.jump-group
+ .control-caption{font-size:6px}.rotate-hint{font-size:7px;width:78px}.move-zone
+ .stick-caption{font-size:6px}}
@media(min-width:701px) and (min-height:600px) and (max-width:900px){.topbar{grid-template-columns:1fr
+ auto 1fr}.hud{grid-column:2;grid-row:1;order:0}.tools{grid-column:3}.task-panel{top:91px}.menu-layer{paddi
+ ng-top:105px}.atlas-card{width:700px}.atlas-card .world-select{max-height:47dvh}.move-zone .joystick{width
+ :154px;height:154px;left:110px;bottom:47px}.jump-group{right:36px;bottom:28px}.jump-pad{width:112px;height
+ :112px}.trail-progress{bottom:30px}}
@media(orientation:landscape) and (max-height:550px){.topbar{display:flex;flex-wrap:nowrap;align-items:sta
+ rt;padding-top:max(10px,env(safe-area-inset-top))}.hud{order:0;padding:6px
+ 13px;gap:10px;margin:0}.wordmark{font-size:11px}.wordmark b{font-size:11px}.task-panel{top:72px;left:15px}
+ .task-panel:not(.expanded){width:160px}.task-list{font-size:9px}.move-zone{height:68%;min-height:190px;max
+ -height:290px}.move-zone .joystick{width:114px;height:114px;left:85px;bottom:24px}.move-zone
+ .stick-thumb{width:50px;height:50px;left:calc(50% - 25px);top:calc(50% - 25px)}.jump-group{right:max(22px,
+ env(safe-area-inset-right));bottom:max(12px,env(safe-area-inset-bottom))}.jump-pad{width:86px;height:86px}
+ .rotate-hint{display:none}.trail-progress{bottom:17px}.toast{top:83px}.menu-layer{padding:73px 17px
+ 12px;align-items:flex-start}.menu-card{max-height:calc(100dvh - 85px);width:520px;padding:16px
+ 24px}.menu-card .instructions{display:none}.menu-card p{max-width:450px;font-size:10px}.menu-card
+ h1{font-size:22px}.atlas-card .world-select{max-height:34dvh}}
@media(prefers-reduced-motion:reduce){.heart-hit,.victory-burst{animation:none}}
/* Avoid continuously compositing backdrop blur over a moving WebGL canvas on phones. */
@media(any-pointer:coarse),(max-width:900px){
 .hud,.task-panel,.tools button,.mobile-controls
+ *{backdrop-filter:none!important;-webkit-backdrop-filter:none!important}
 .hud{background:#194768f2}.task-panel{background:#194768ed}
 .stick-thumb{will-change:transform;transition:none!important}
 .joystick{box-shadow:inset 0 0 0 1px #e2f8ff44}
}
```

Code 18 / src/worlds.js

52 original lines | 5305 bytes | complete file

Ten additional deterministic routes and twelve theme palettes.

```
// Explicit route layouts and theme palettes for the twelve-world adventure.
// Generated islands keep a bounded jump envelope; layouts are deterministic.
const specs = [
  ['Frostfall Peaks','Snowy passes and frozen falls',[0,-2,-4,-1,2,4,1,-2,0],[3,6],'snow'],
  ['Sunstone Dunes','Ancient ruins above the desert',[0,3,6,3,0,-3,-5,-2,1],[2,6],'desert'],
  ['Emerald Canopy','A winding trail through the jungle',[0,-3,-6,-3,0,3,6,3,0,-3],[3,7],'jungle'],
  ['Coral Coast','Palm islands over a turquoise sea',[0,2,4,2,-1,-4,-2,1,3],[2,5],'ocean'],
  ['Amberwood Valley','Golden forests and autumn leaves',[0,-2,1,4,1,-2,-5,-2,0,3],[3,7],'autumn'],
  ['Moonstone Orbit','A silver trail beneath the stars',[0,3,1,-2,-5,-2,1,4,2,0],[2,6,8],'moon'],
  ['Mushroom Hollow','Giant mushrooms and glowing spores',[0,-3,-1,2,5,2,-1,-4,-1,2,0],[3,7],'mushroom'],
  ['Thunderhead Isles','Storm clouds and electric beacons',[0,2,-1,-4,-1,2,5,2,-1,1],[2,5,8],'storm'],
  ['Obsidian Forge','Black rock and rivers of fire',[0,-3,-6,-3,0,3,1,-2,-5,-2,0],[3,6,9],'volcano'],
  ['Aurora Summit','The final climb under northern lights',[0,3,6,3,0,-3,-6,-3,0,3,0],[2,5,8],'aurora'],
];
function makeWorld([name, subtitle, layout, ferries, theme], offset) {
  const id = offset + 2, cpIndex = Math.floor(layout.length / 2);
  // Keep the checkpoint on solid ground, even if a route lists that index as a ferry.
  const movingIndices = ferries.filter(i => i !== cpIndex);
  let z = 2;
  const platforms = layout.map((x, i) => {
    const moving = movingIndices.includes(i), finish = i === layout.length-1;
    const d = i === 0 || finish ? 12 : moving ? 6 : 8 + (id + i) % 2;
    if (i) {
      const prevD = i === 1 ? 12 : movingIndices.includes(i-1) ? 6 : 8 + (id+i-1)%2;
      z -= prevD/2 + d/2 + 2.2 + ((id+i)%3)*.2;
    }
    return {id:i===0?'start':finish?'finish':i===cpIndex?'checkpoint':`island-${i}`,x,z,w:finish?12:moving
?5.5:10,d,y:Math.min(2.2, i*.18),moving,amplitude:moving?1.1:0,rate:.55+(id%4)*.08,phase:id*.4};
  });
  const cp = platforms[cpIndex], end = platforms.at(-1);
  const coins = platforms.flatMap((p,i) => p.moving ? [] :
[[p.x,p.y+1.1,p.z+p.d*.28],[p.x,p.y+1.1,p.z],[p.x,p.y+1.1,p.z-p.d*.28]]);
  const dangers = platforms.filter((p,i) => i>0 && i<platforms.length-1 && !p.moving &&
p.id!=='checkpoint');
  const rotators = dangers.filter((_,i)=>i%2===0).map((p,i)=>({x:p.x,z:p.z,y:p.y,length:4.6+(id%3)*.2,spee
d:(i%2?-1:1)*(.65+id*.035),phase:id*.3+i}));
  const hazards = dangers.filter((_,i)=>i%2===1).map(p=>({x:p.x-1.7,z:p.z,y:p.y,r:.8}));
  return {id,name,subtitle,theme,platforms,coins,hazards,rotators,spawn:[0,0,4],checkpoint:[cp.x,cp.y,cp.z
+1],flag:[cp.x-2.7,cp.y,cp.z+1],portal:[end.x,end.y,end.z-2.5],stages:platforms.slice(1).map((p,i)=>[p.z+p
.d/2+2,`${name} · crossing ${i+1}`]).concat([[-Infinity,`${name} · final gate`]])};
}
export const EXTRA_WORLDS = specs.map(makeWorld);
const theme = (name, ground, path, rock, skyTop, skyBottom, plant, foliage, water, cloud, accent,
weather='none') => ({name,ground,path,rock,skyTop,skyBottom,plant,foliage,water,cloud,accent,weather});
export const THEMES = {
  garden:theme('garden','#78bd39','#e8c785',['#a18160','#b9956d','#c6a47d','#887968','#a5947d'],'#217aef',
'#c4e6ff','pine','#38a854','#219feb','#f3faff','#ffda70'),
  ember:theme('ember','#bc6951','#efbc83',['#8b5364','#b06c70','#c3847a','#63516d','#9c7280'],'#332066','#
faa478','crystal','#ba88ed','#f44b15','#c5a6cb','#d5aaff','embers'),
  snow:theme('snow','#edf8ff','#b5ddec',['#8babc1','#bed9e5','#c9e8f2','#7f96b6','#b5c8dd'],'#3c83be','#e4
f6ff','snowpine','#97c2d3','#84d1f5','#f5fcff','#a6eaff','snow'),
  desert:theme('desert','#ddaf65','#f5d594',['#b5804c','#d4a36e','#e9b773','#a17551','#d4b27a'],'#348bd6',
'#ffe2b4','cactus','#59996a','#eec36a','#fff4dd','#ffd35f','dust'),
  jungle:theme('jungle','#347e46','#a9a96c',['#526756','#79946a','#8aa178','#4b6154','#a2b391'],'#326f79',
'#a4dbbc','palm','#16795b','#51c5b0','#d6ece1','#a1f282','spores'),
  ocean:theme('ocean','#e9d7a3','#fff0c3',['#6b9f9c','#87b8b0','#a8c9bb','#588a91','#d0d6b5'],'#1689cb','#
a9f2f6','palm','#2fb284','#16c3d7','#f5ffff','#6eeff1'),
  autumn:theme('autumn','#c0934d','#e8cb85',['#836953','#a48c60','#c9ac76','#78604e','#b69b74'],'#617cb1',
'#f6d1a5','broadleaf','#ed9845','#9dc0c4','#f4e7d3','#ffd07b','leaves'),
  moon:theme('moon','#9babc6','#cbd9e6',['#5c6383','#939cba','#afb5d0','#535d7c','#7e8bab'],'#090f31','#55
5c96','crystal','#99bdff','#a2baff','#747697','#c5dcff','stars'),
  mushroom:theme('mushroom','#708795','#a9b9af',['#5d617e','#7e7ca0','#a19db8','#4f5c77','#898bad'],'#282b
65','#b19cc8','mushroom','#f488ba','#9ee0e0','#c4b7dc','#ffb0e3','spores'),
  storm:theme('storm','#65778c','#a5b3bc',['#3e4b65','#67728a','#8997aa','#333e59','#7b8999'],'#172039','#
859bad','beacon','#a8c7ff','#668ff5','#8a9db0','#87c8ff','rain'),
  volcano:theme('volcano','#553e49','#aa6f54',['#2b2939','#524252','#6d4a58','#292535','#7d5661'],'#2a1a33
','#e38663','basalt','#e28b61','#ff5718','#aa777f','#ffb056','embers'),
  aurora:theme('aurora','#d6e8ed','#a8d5de',['#627d9e','#93b6c9','#bfdbe2','#4a6689','#9bb5d0'],'#071e3a',
'#4e8a9e','snowpine','#9fcbdf','#61e6ce','#a5d3dc','#7effd8','snow'),
};
export const getTheme = world => THEMES[world.theme || (world.id===1?'ember':'garden')];
```